	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	René Adrián Dávila Pérez	
Asignatura:	Programación Orientada a Objetos	
Grupo:	7	
No de Práctica(s):	10	
Integrante(s):	120002850	323064039
	323071378	323214636
	426059576	426058744
Semestre:	2027-1	
Fecha de entrega:	27-Octubre-2026	
Obervaciones:		

CALIFICACIÓN: _____

Índice

1. Introducción **2**

1.1. Planteamiento del problema 2

1.2. Motivación 2

1.3. Objetivos 2

2. Marco Teórico **2**

3. Desarrollo **3**

3.1. Práctica 5: encapsulamiento 4

3.2. Práctica 6: organización por paquetes 8

3.3. Práctica 7: jerarquía de cuadriláteros 39

3.4. Práctica 8: contratos e interfaz gráfica 48

3.5. Proyecto fase 2: sistema hospitalario 55

4. Resultados **85**

5. Conclusiones **86**

1. Introducción

1.1. Planteamiento del problema

Las prácticas 5 a 8 y el proyecto en su segunda fase contienen distintas formas de organizar objetos. El problema consiste en representar sus clases y relaciones mediante UML, conservando lo que existe en sus fuentes y evitando vínculos que el programa no establece. Se toma como referencia el apartado de UML del manual de laboratorio y la imagen proporcionada para esta actividad [1, 2].

1.2. Motivación

Un listado de archivos permite localizar una clase, pero no explica por sí solo qué objetos conserva, de qué tipo hereda o qué contrato implementa. Los diagramas facilitan reconocer esas diferencias y comparar programas que emplean encapsulamiento, paquetes, herencia e interfaces.

1.3. Objetivos

El objetivo es construir diagramas de clases con atributos, operaciones, visibilidad y relaciones verificables. También se busca justificar las multiplicidades y conservar versiones editables, acompañadas de vistas que permitan leer todos los miembros sin reducirlos a un tamaño ilegible.

2. Marco Teórico

El manual describe el diagrama de clases mediante tres elementos: clases, cardinalidad y relaciones. Su función es mostrar la estructura del sistema y la comunicación prevista entre sus partes [1]. La imagen de referencia utiliza cajas blancas divididas por nombre, atributos y operaciones, además de conectores y cantidades junto a sus extremos [2].

Una asociación corresponde a una referencia conservada en el estado de un objeto. Una dependencia expresa un uso, por ejemplo la creación de un objeto local o la recepción de un parámetro. La generalización representa la herencia y apunta con un triángulo vacío hacia el tipo general. La realización de una interfaz utiliza un triángulo vacío y una línea discontinua.

Los signos $+$, $-$, $\#$ y $\sim$ indican acceso público, privado, protegido y de paquete. Los elementos estáticos se subrayan; los tipos y métodos abstractos aparecen en cursiva. La anotación `{readOnly}` identifica atributos finales. En una colección final la referencia no puede reasignarse, aunque su contenido puede cambiar.

La multiplicidad expresa los valores admitidos en un extremo de una relación. `0..1` permite ausencia o una referencia; `0..*` permite una colección vacía y sin un máximo determinado. Una asociación no demuestra por sí misma composición: para usar el rombo sólido tendría que existir una dependencia de vida útil que las fuentes justificaran.

3. Desarrollo

Se revisaron las fuentes de las prácticas anteriores desde la 2 como antecedentes de nivel. El alcance de los diagramas comprende exclusivamente las prácticas 5, 6, 7 y 8, y `Proyecto2/src/hospital`. El directorio `src` de la segunda fase es la fuente de compilación que señala su documento de uso; las copias superiores no se cuentan como clases adicionales [3, 4].

Primero se extrajeron las declaraciones de Java con el analizador del JDK. Después se compilaron los cinco conjuntos en directorios temporales independientes y se contrastaron los miembros con `javap -private -s`. Las operaciones heredadas no se repiten. Los constructores implícitos, los componentes del record y los elementos de las enumeraciones se distinguen de las variables locales y de los miembros sintéticos del compilador.

Cada conjunto dispone de un maestro global con todos sus miembros y relaciones. Para su lectura impresa, el reporte desglosa ese maestro en vistas de detalle. La caja principal contiene los miembros del tipo que se explica; las cajas inferiores son referencias a los otros extremos, cuyos miembros se desarrollan en sus propias figuras. Las continuaciones reparten operaciones o relaciones, sin añadir clases nuevas.

Las asociaciones se etiquetan con el nombre del atributo y la multiplicidad del extremo referenciado. Cuando el mismo par conserva varios roles, se enumeran por separado en el conector. No se supone una multiplicidad inversa que el atributo no determina. Las dependencias de uso no reciben cantidades. Los tipos de la biblioteca se identifican como JDK cuando son un supertipo directo de una clase propia.

3.1. Práctica 5: encapsulamiento

MainEmpleado utiliza Empleado y MainFecha utiliza Fecha. Son dependencias porque los objetos se crean dentro de sus métodos principales, sin guardarse en atributos de esas clases principales. Empleado y Fecha son modelos independientes entre sí. [3].

La figura 3.1 muestra Empleado.

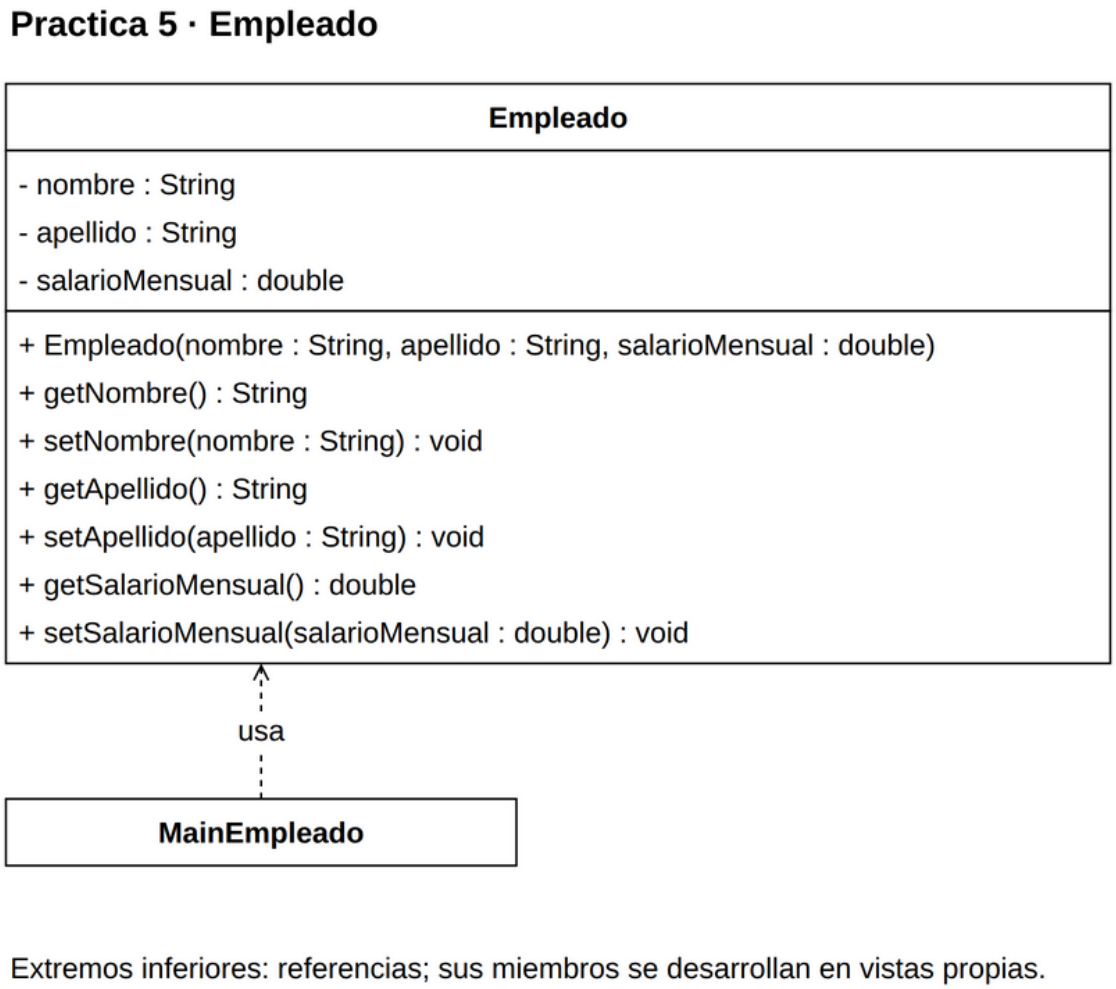


Figura 3.1: Práctica 5 — Empleado.

La figura 3.2 muestra Fecha.

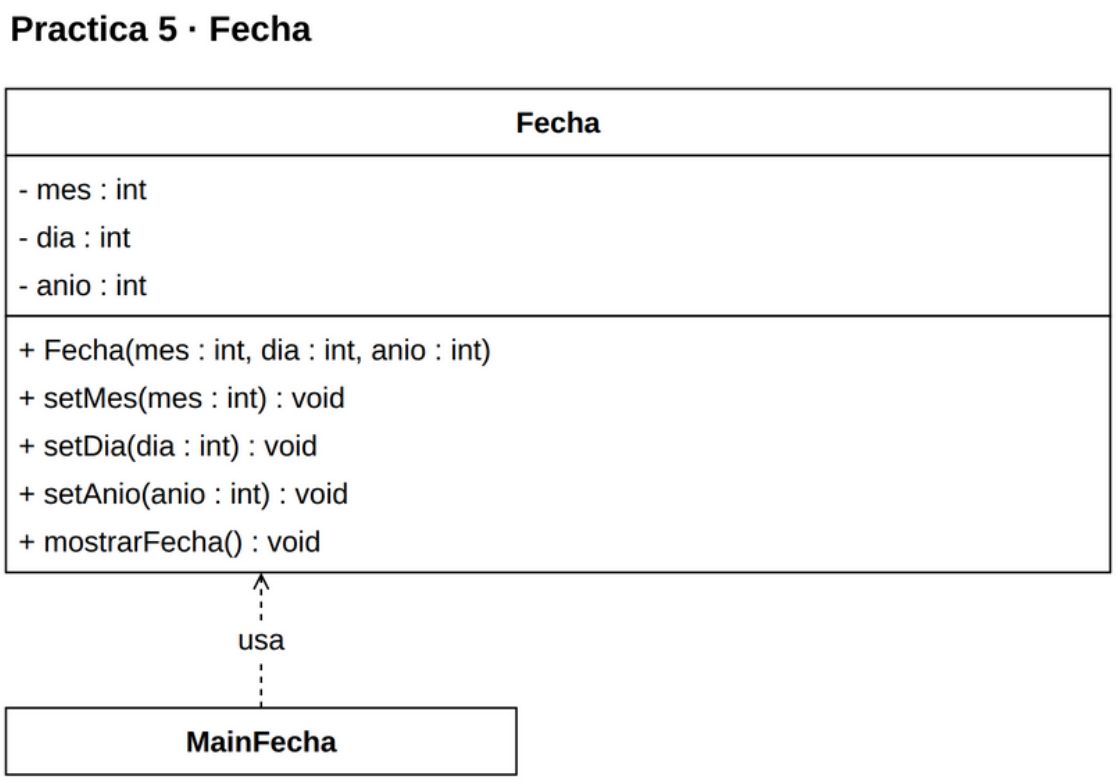


Figura 3.2: Práctica 5 — Fecha.

La figura 3.3 muestra MainEmpleado.

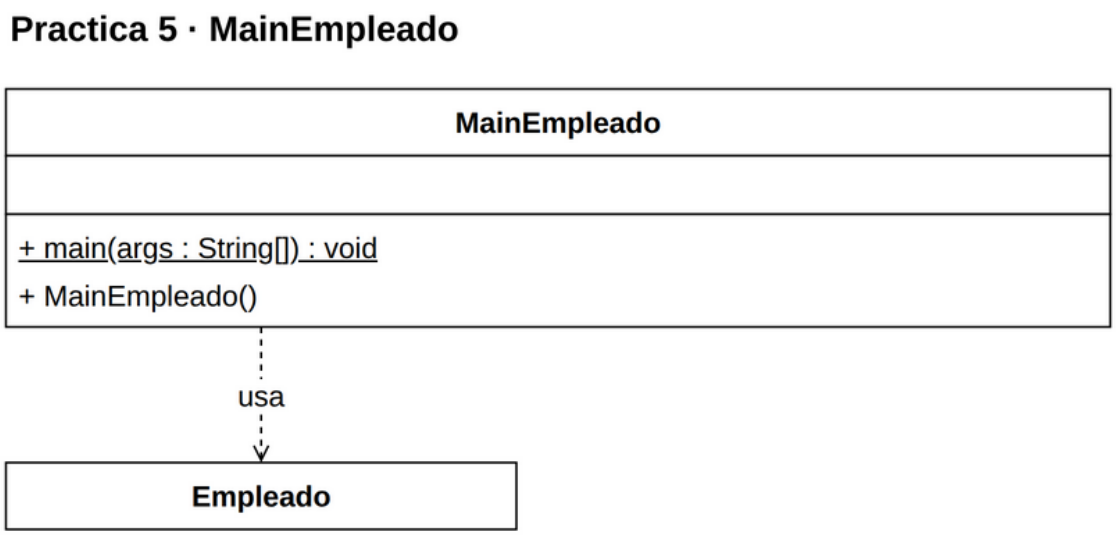


Figura 3.3: Práctica 5 — MainEmpleado.

La figura 3.4 muestra MainFecha.

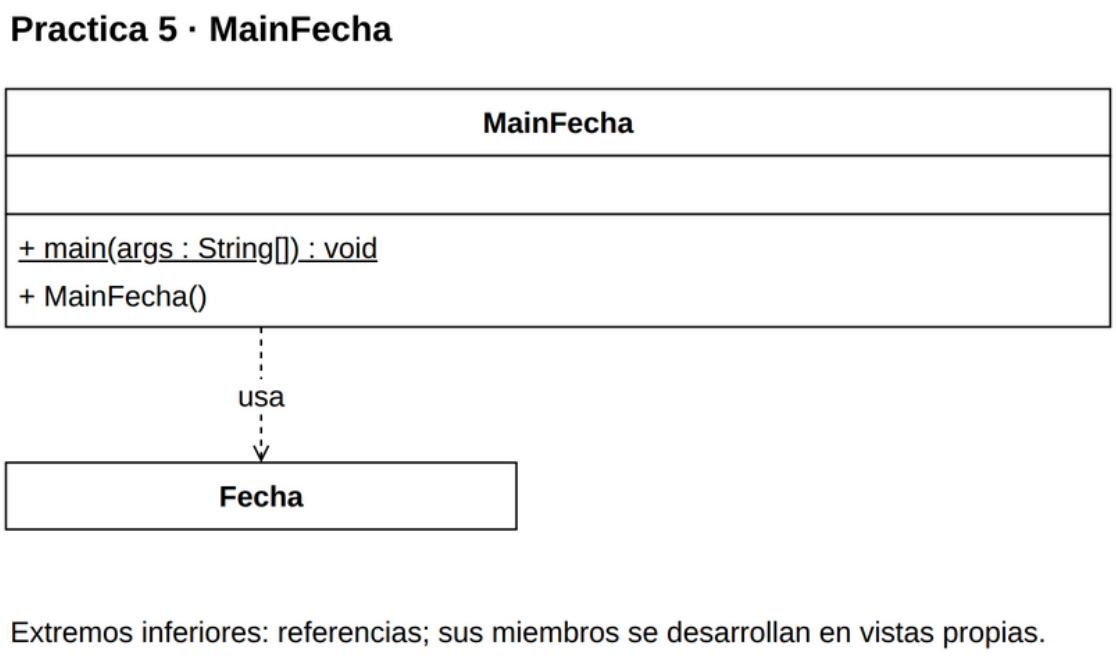


Figura 3.4: Práctica 5 — MainFecha.

3.2. Práctica 6: organización por paquetes

Esta práctica integra programas de aritmética, estructuras de control, palíndromos, análisis de palabras, empleados, fechas, un editor gráfico y el primer sistema hospitalario. El paquete identifica cada Main y evita confundirlos. Los programas numéricos independientes permanecen sin conexiones inventadas. EditorFrame conserva un CanvasPanel; el lienzo administra figuras e historial. Figura es un record ya existente y contiene la enumeración Tipo. El adaptador anónimo de ratón se muestra como un tipo local, con sus propios atributos y operaciones. En el hospital se conservan las asociaciones de Sistema, Medico, Enfermero y Paciente. Sus colecciones expuestas no permiten convertir los límites del recorrido habitual en restricciones globales garantizadas. [3].

La figura 3.5 muestra p1.Calculadora.

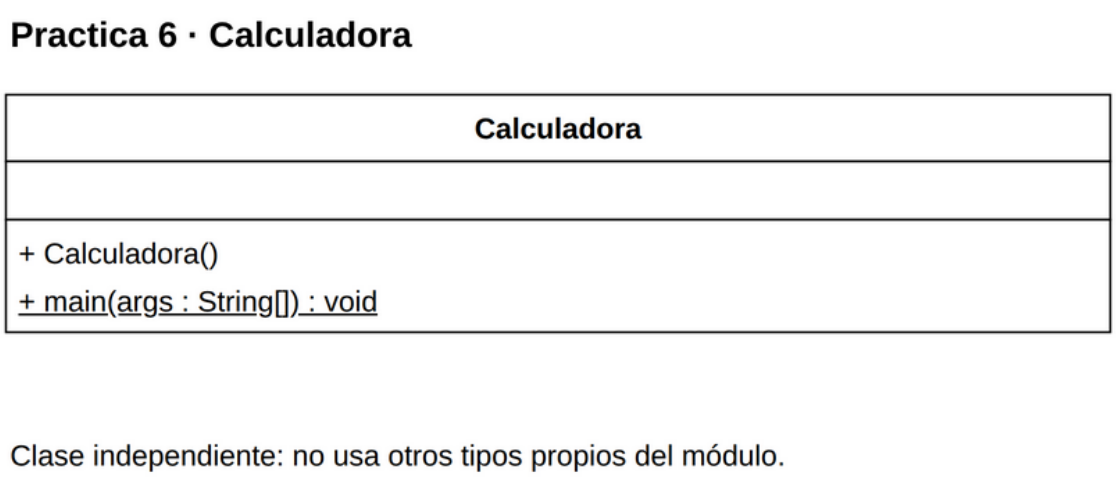


Figura 3.5: Práctica 6 — p1.Calculadora.

La figura 3.6 muestra p1.Division.

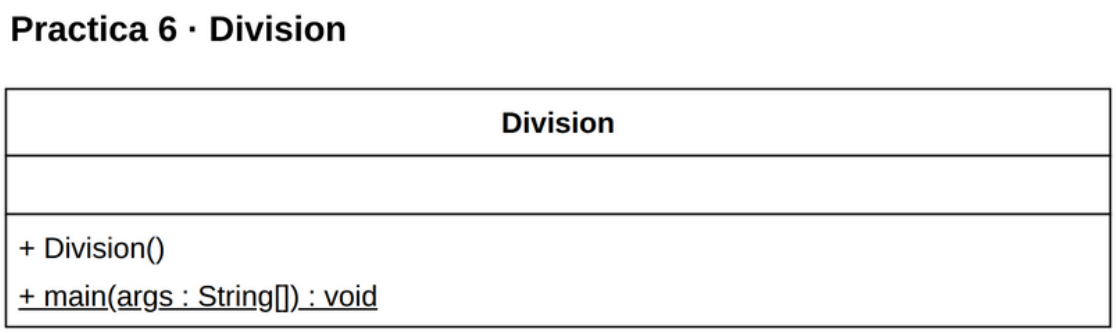


Figura 3.6: Práctica 6 — p1.Division.

La figura 3.7 muestra p1.Exponente.

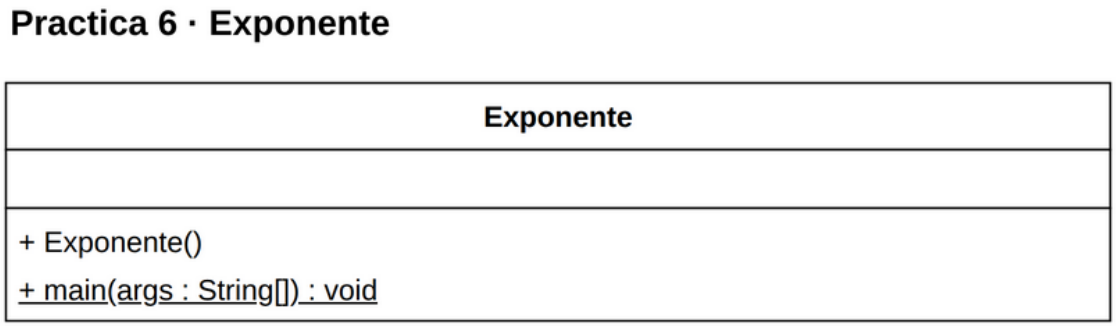


Figura 3.7: Práctica 6 — p1.Exponente.

La figura 3.8 muestra p1.Multiplicacion.

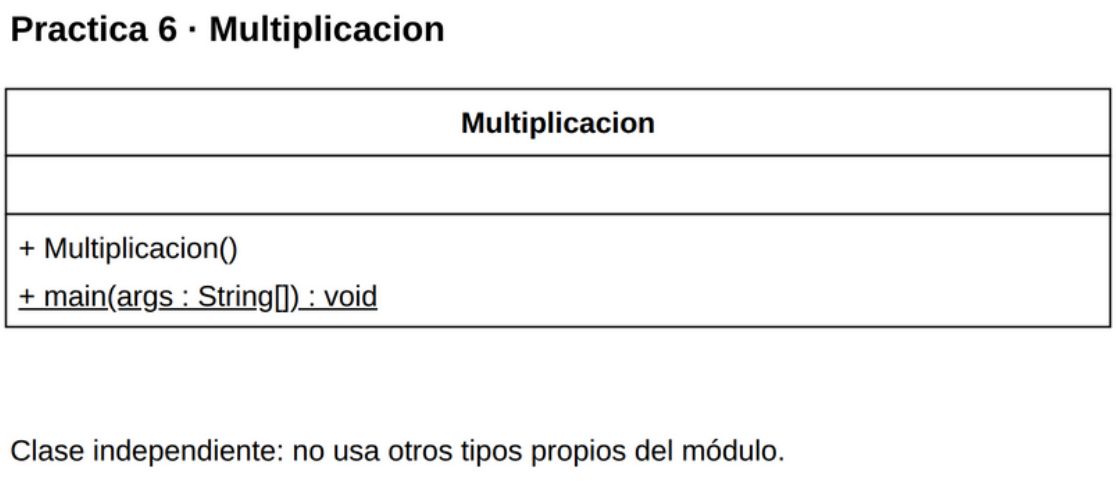


Figura 3.8: Práctica 6 — p1.Multiplicacion.

La figura 3.9 muestra p1.Raiz.

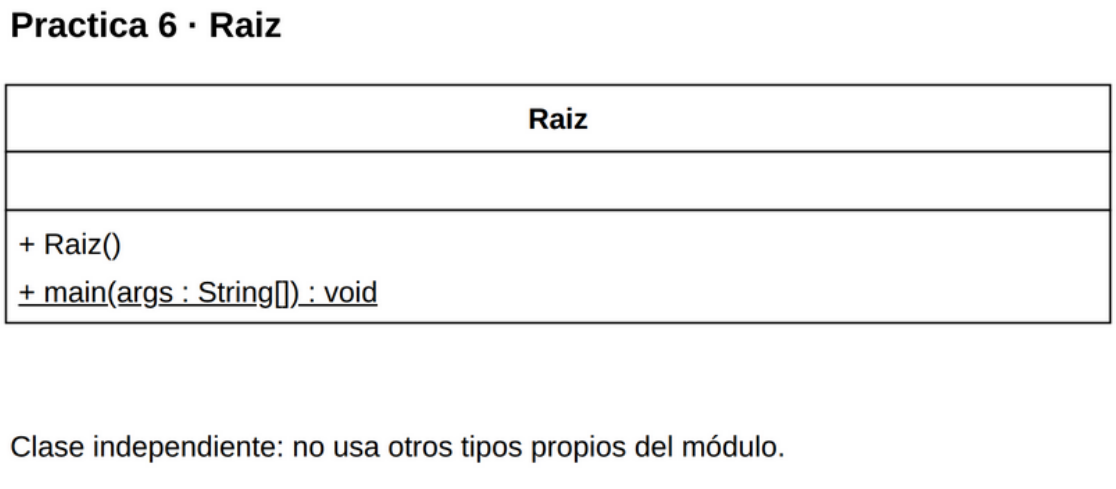


Figura 3.9: Práctica 6 — p1.Raiz.

La figura 3.10 muestra p1.Resta.



Figura 3.10: Práctica 6 — p1.Resta.

La figura 3.11 muestra p1.Suma.

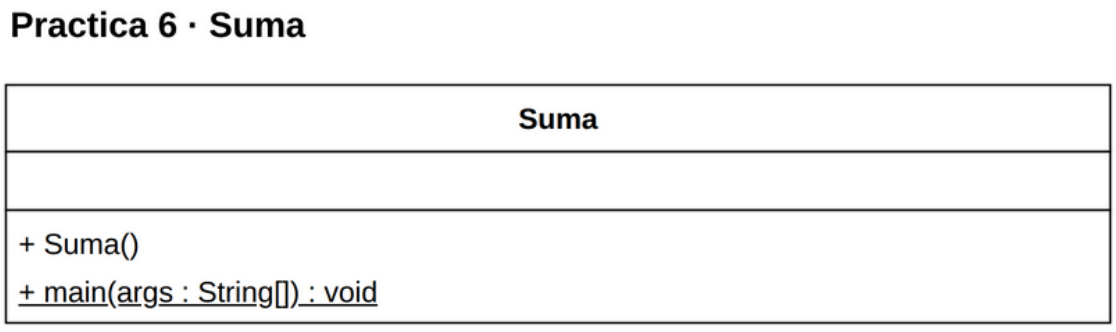


Figura 3.11: Práctica 6 — p1.Suma.

La figura 3.12 muestra editor.CanvasPanel, parte 1 de 2.

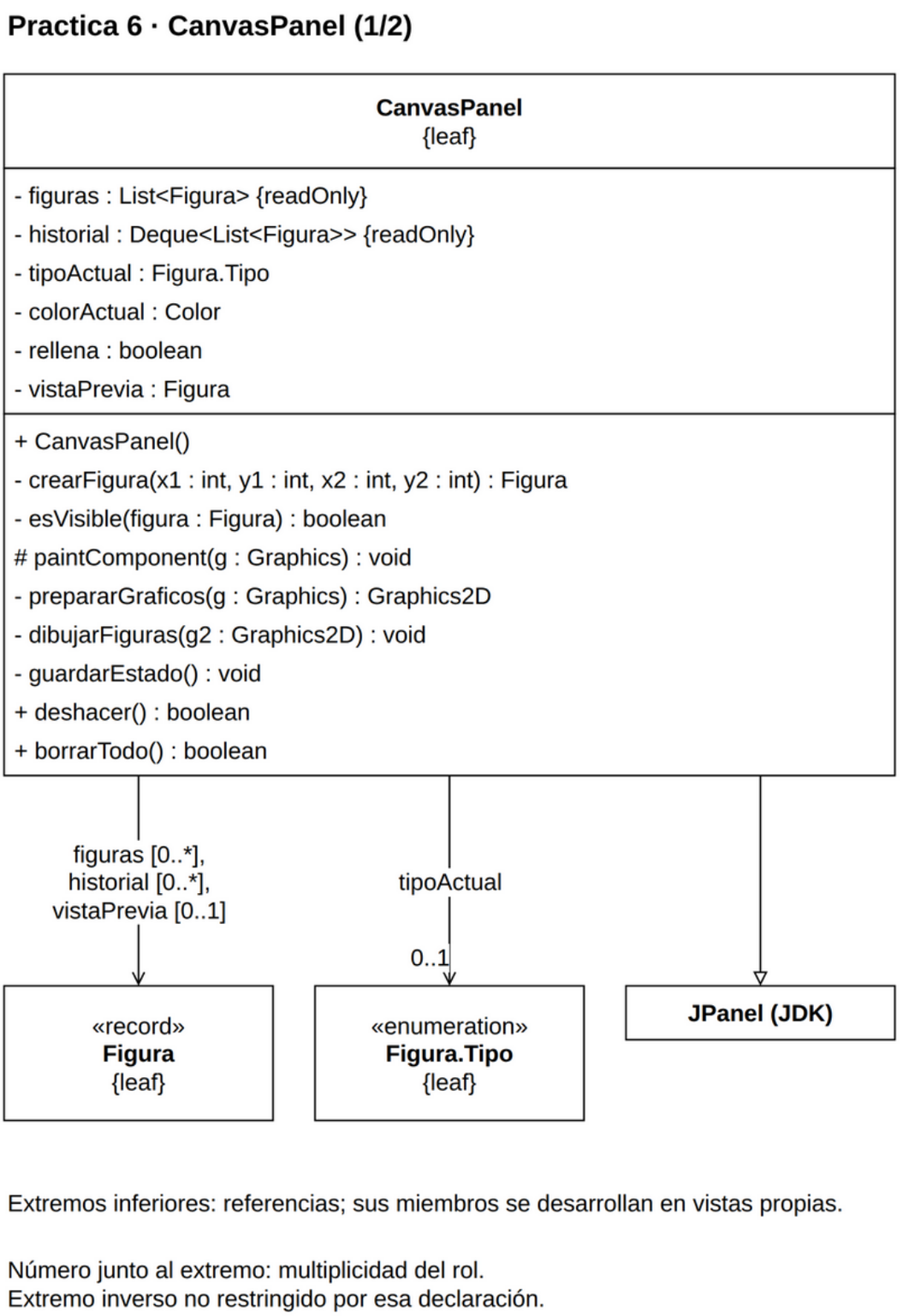


Figura 3.12: Práctica 6 — editor.CanvasPanel. Parte 1 de 2.

La figura 3.13 muestra editor.CanvasPanel, parte 2 de 2.

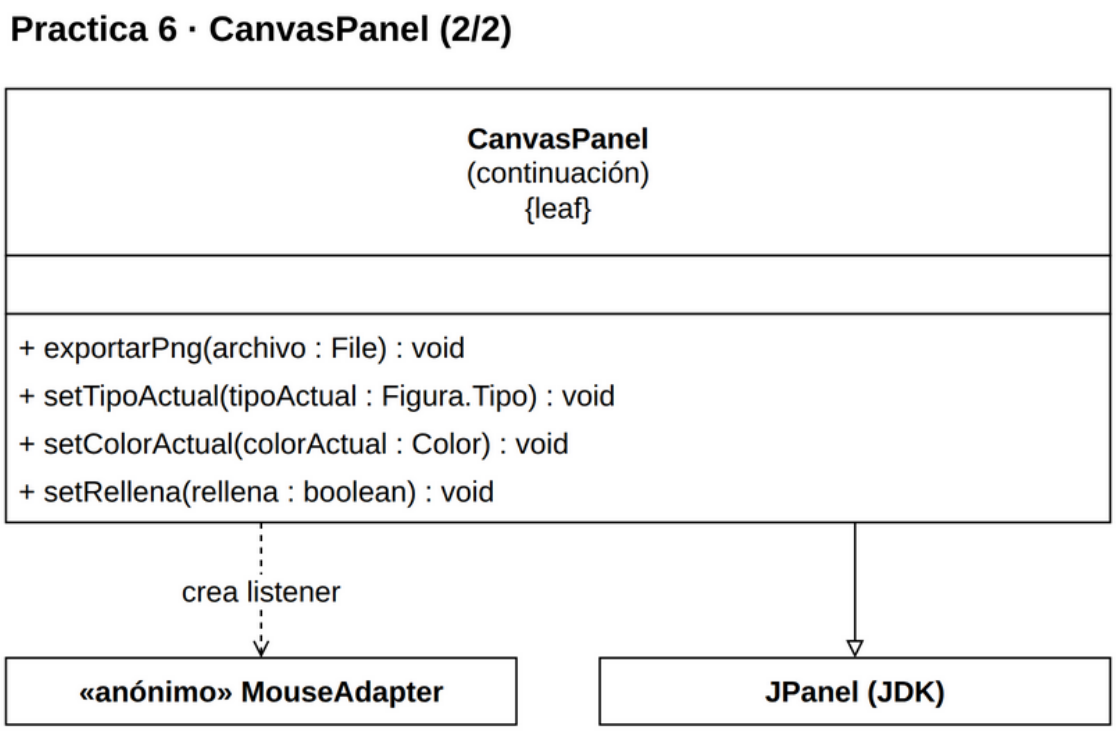


Figura 3.13: Práctica 6 — editor.CanvasPanel. Parte 2 de 2.

La figura 3.14 muestra editor.CanvasPanel.anónimo MouseAdapter.

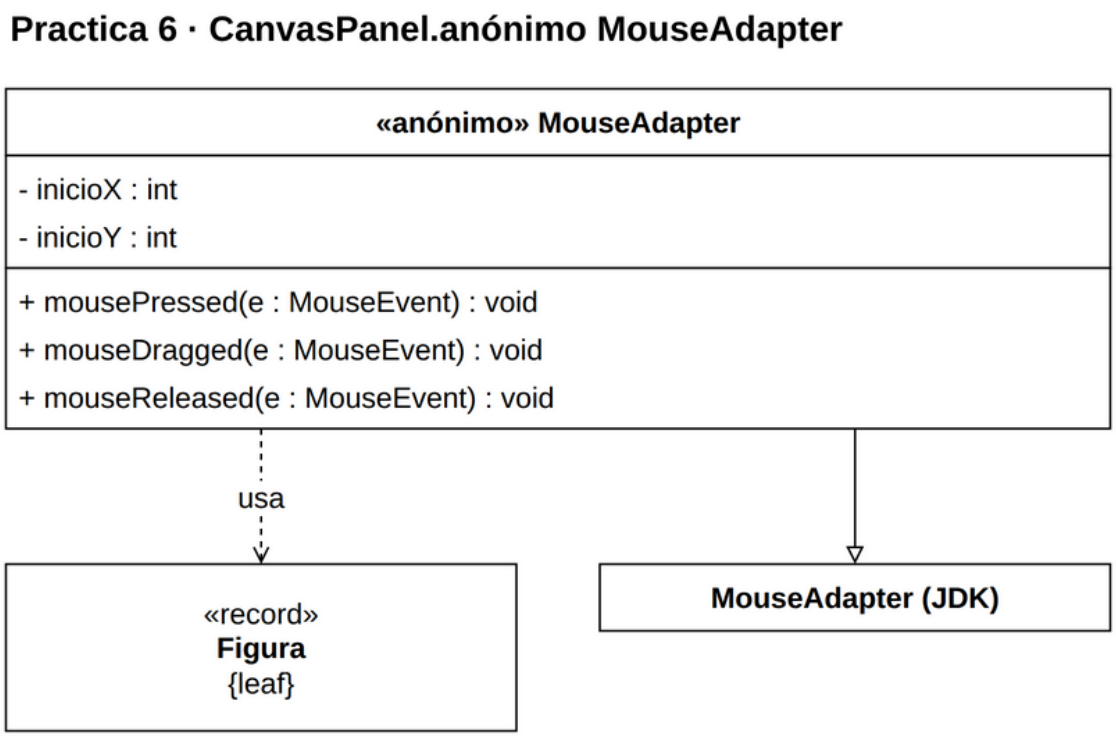


Figura 3.14: Práctica 6 — editor.CanvasPanel.anónimo MouseAdapter.

La figura 3.15 muestra editor.EditorFrame, parte 1 de 2.

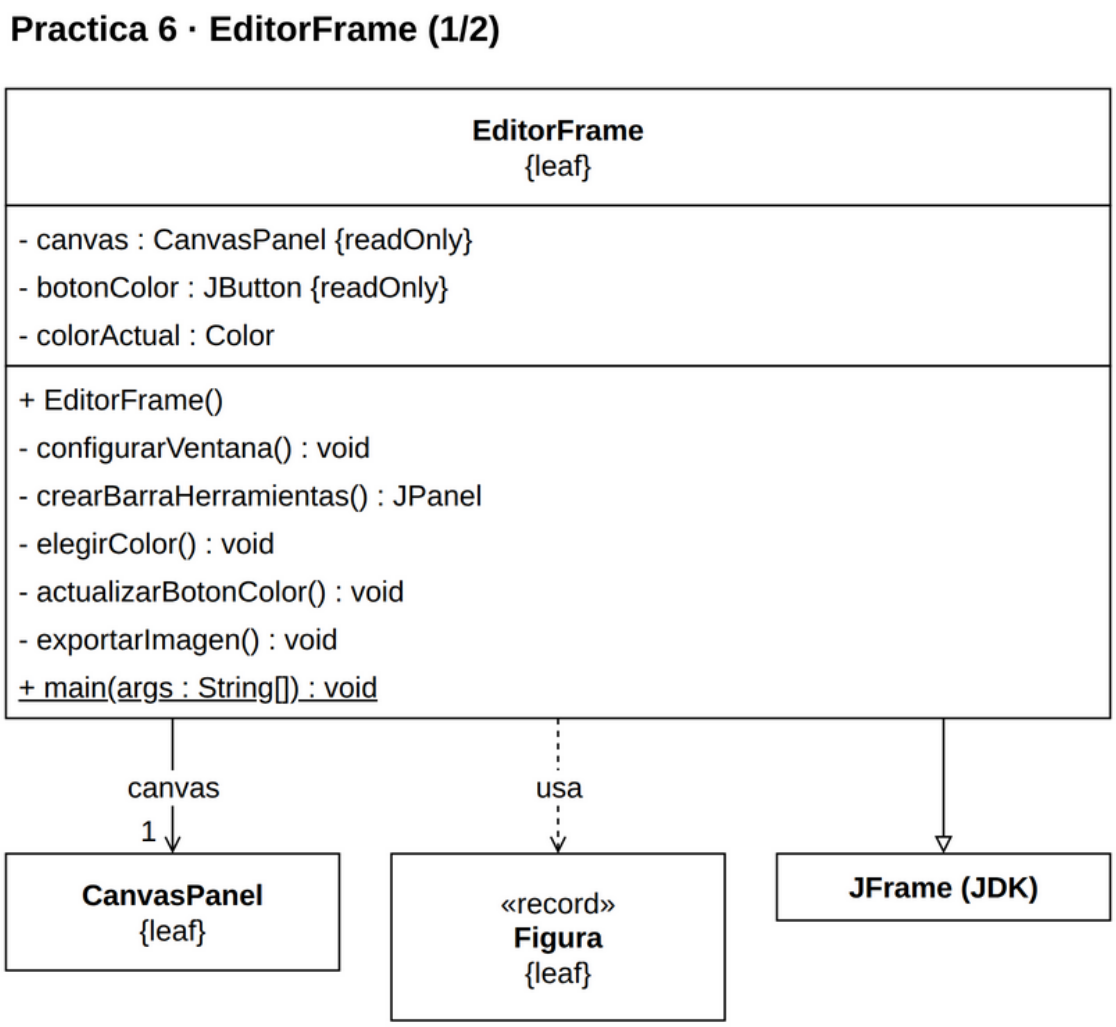


Figura 3.15: Práctica 6 — editor.EditorFrame. Parte 1 de 2.

La figura 3.16 muestra editor.EditorFrame, parte 2 de 2.

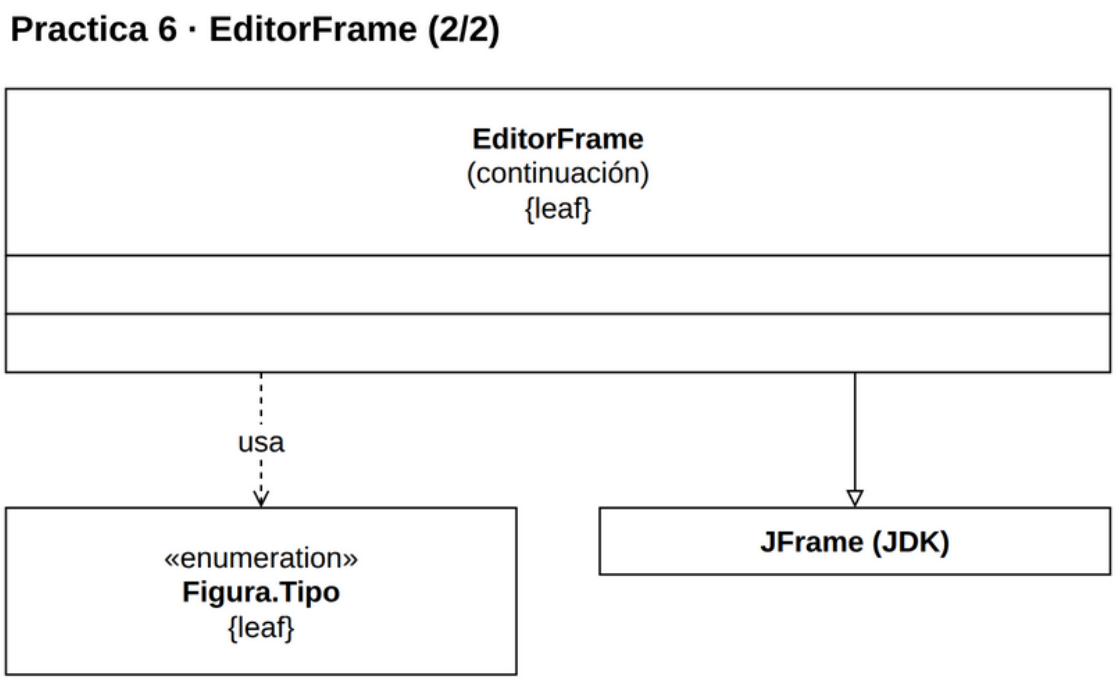


Figura 3.16: Práctica 6 — editor.EditorFrame. Parte 2 de 2.

La figura 3.17 muestra editor.Figura, parte 1 de 2.

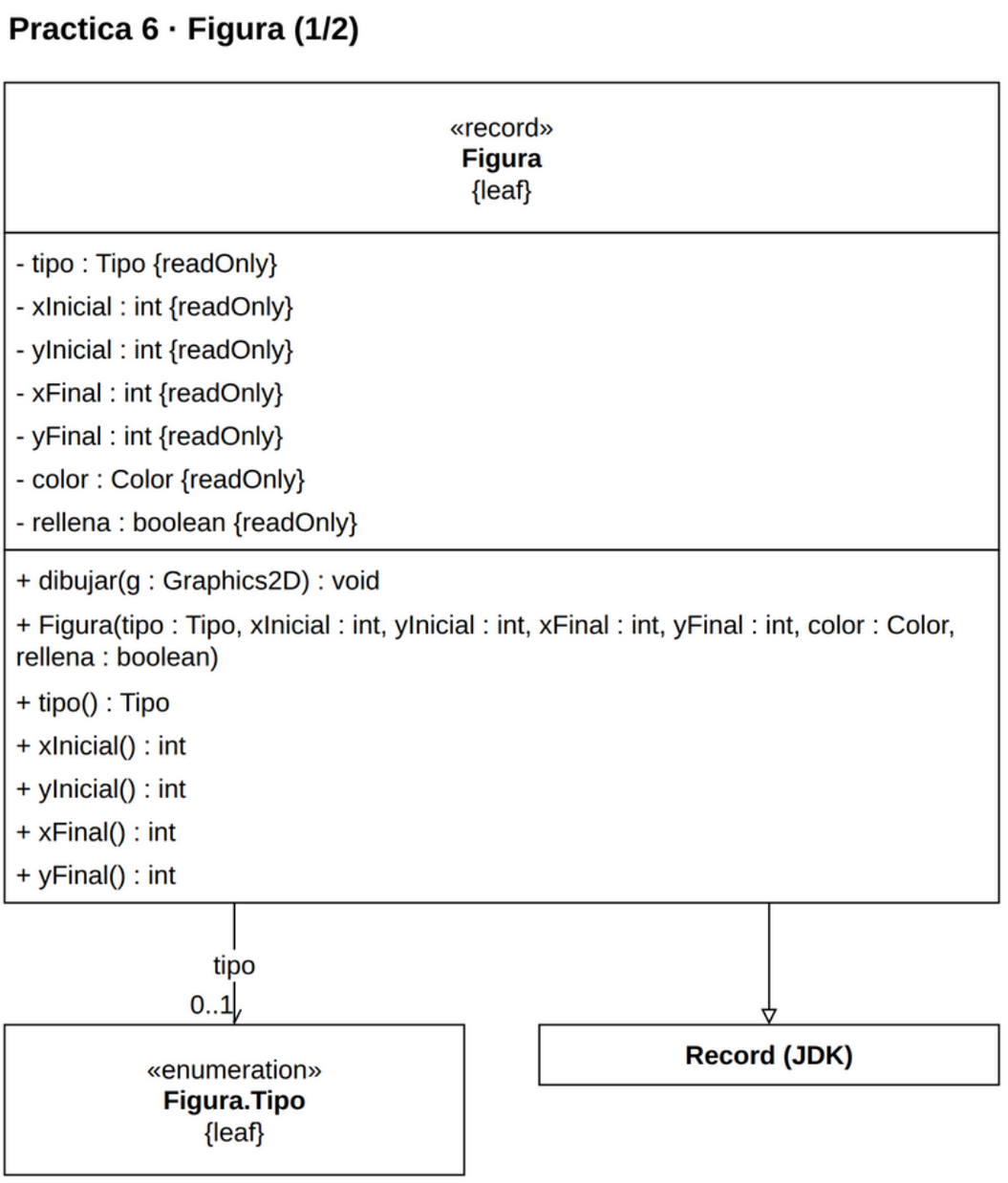


Figura 3.17: Práctica 6 — editor.Figura. Parte 1 de 2.

La figura 3.18 muestra editor.Figura, parte 2 de 2.

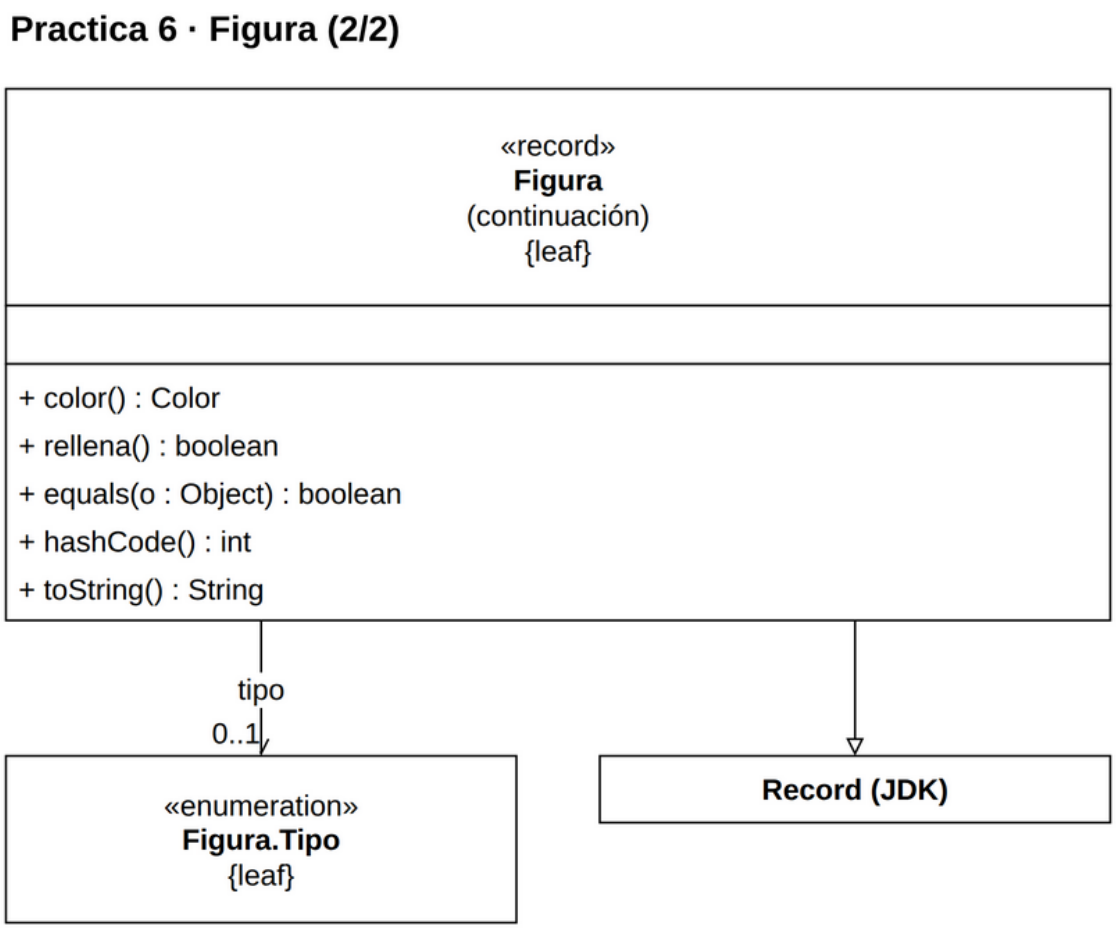


Figura 3.18: Práctica 6 — editor.Figura. Parte 2 de 2.

La figura 3.19 muestra editor.Figura.Tipo.

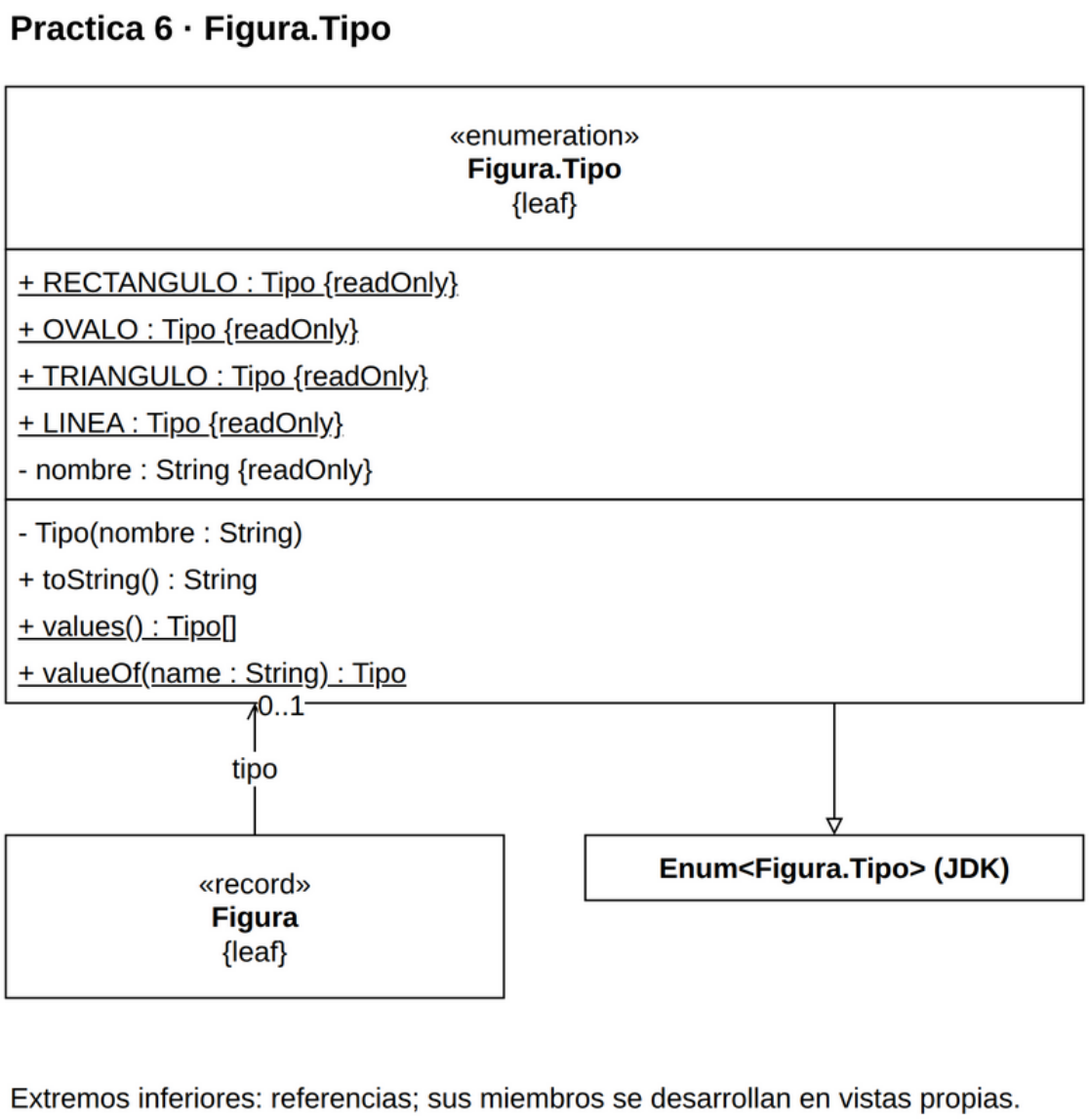


Figura 3.19: Práctica 6 — editor.Figura.Tipo.

La figura 3.20 muestra p2.Ejercicio1_P2.

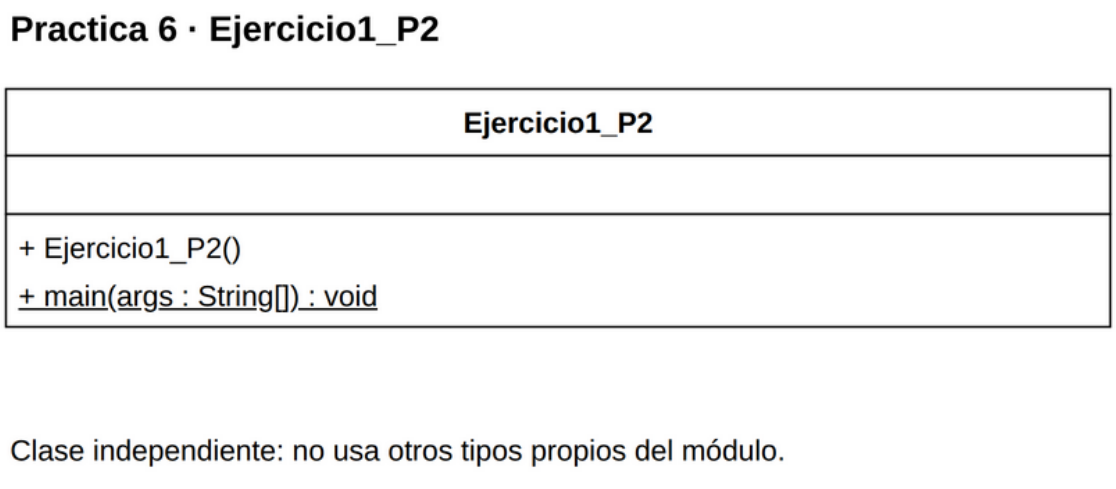


Figura 3.20: Práctica 6 — p2.Ejercicio1_P2.

La figura 3.21 muestra p2.Ejercicio2_P2.

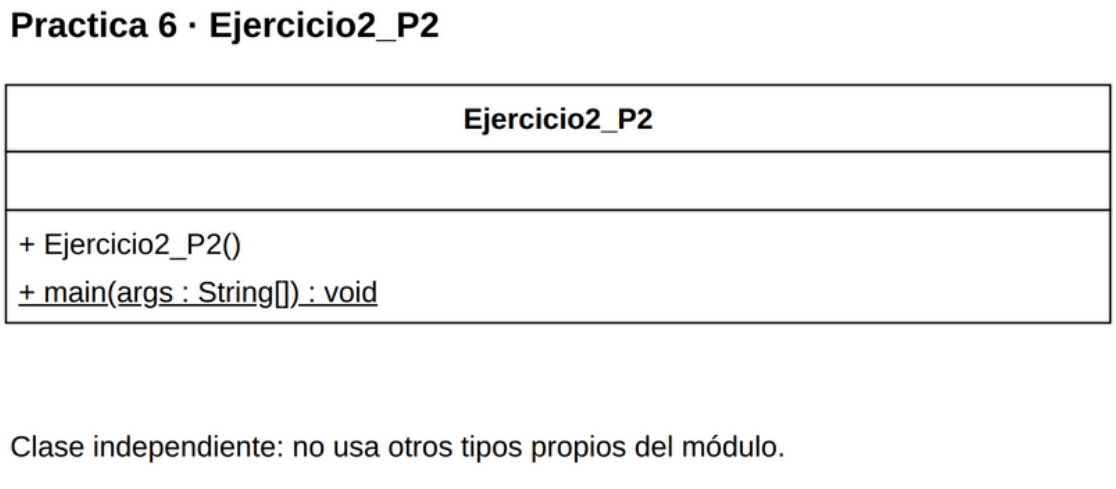


Figura 3.21: Práctica 6 — p2.Ejercicio2_P2.

La figura 3.22 muestra p2.Ejercicio3_P2.

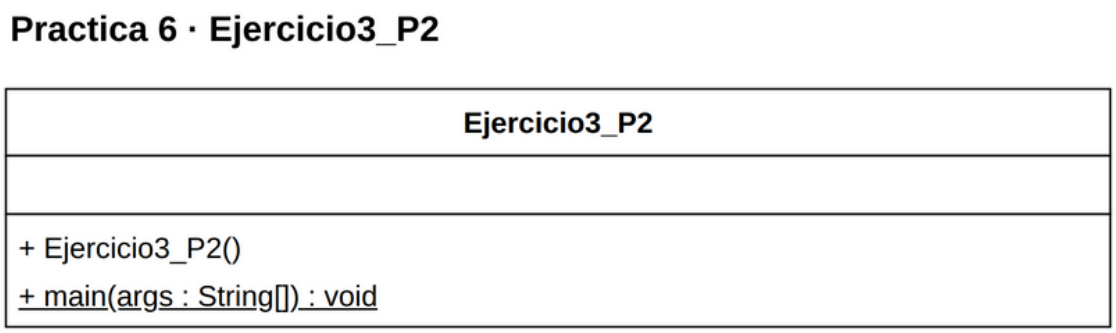


Figura 3.22: Práctica 6 — p2.Ejercicio3_P2.

La figura 3.23 muestra p3.Palindromo.

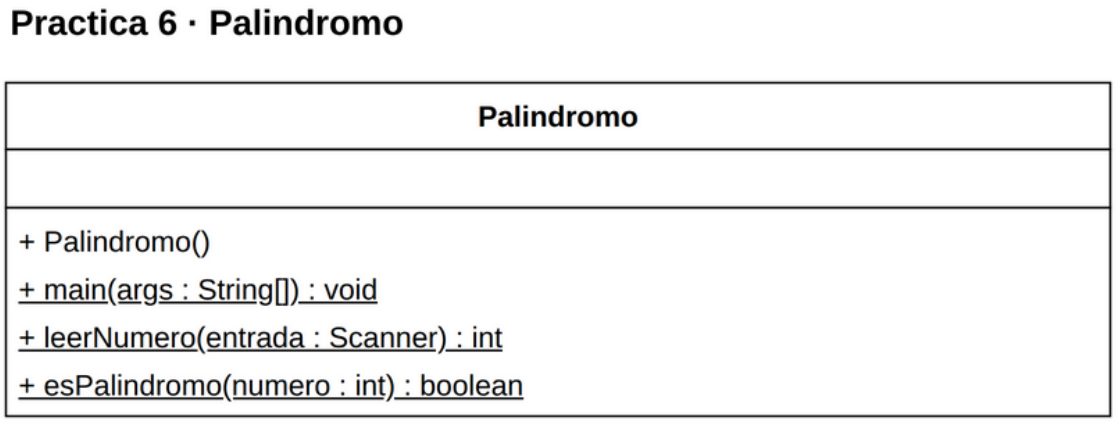


Figura 3.23: Práctica 6 — p3.Palindromo.

La figura 3.24 muestra p4.AnalizadorPalabras.

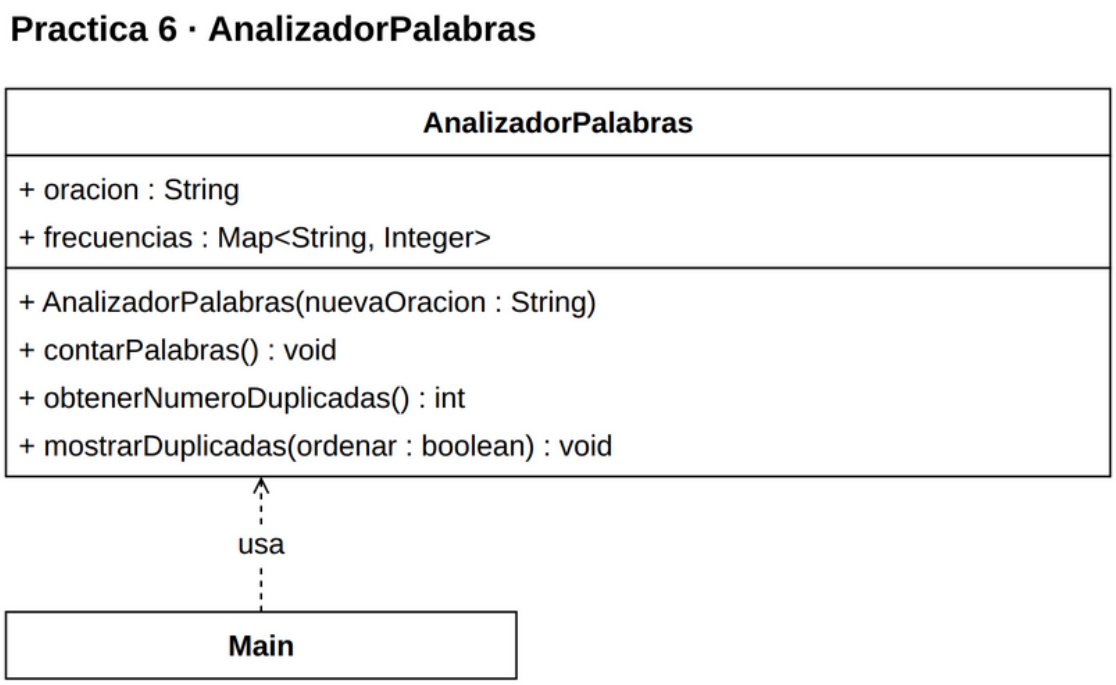


Figura 3.24: Práctica 6 — p4.AnalizadorPalabras.

La figura 3.25 muestra p4.Main.

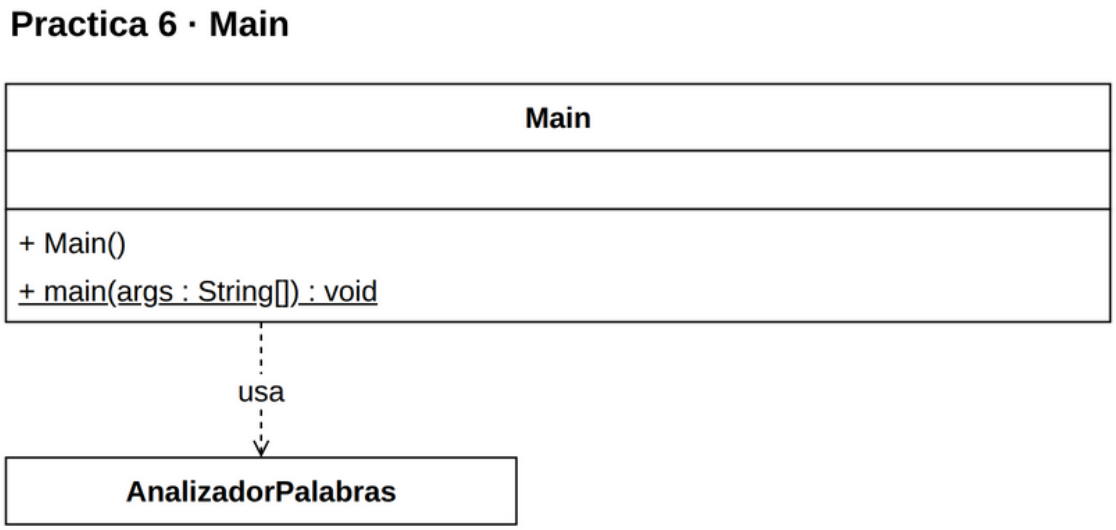


Figura 3.25: Práctica 6 — p4.Main.

La figura 3.26 muestra p5.Empleado.

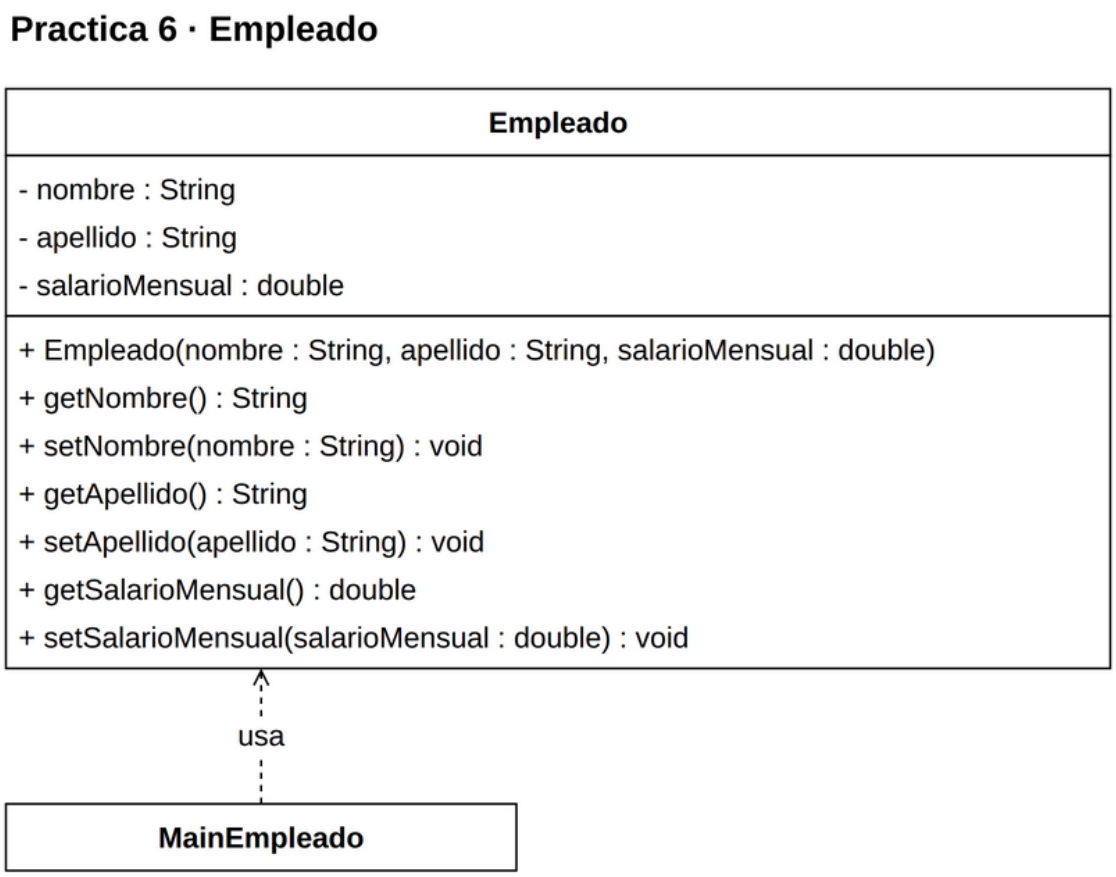


Figura 3.26: Práctica 6 — p5.Empleado.

La figura 3.27 muestra p5.Fecha.

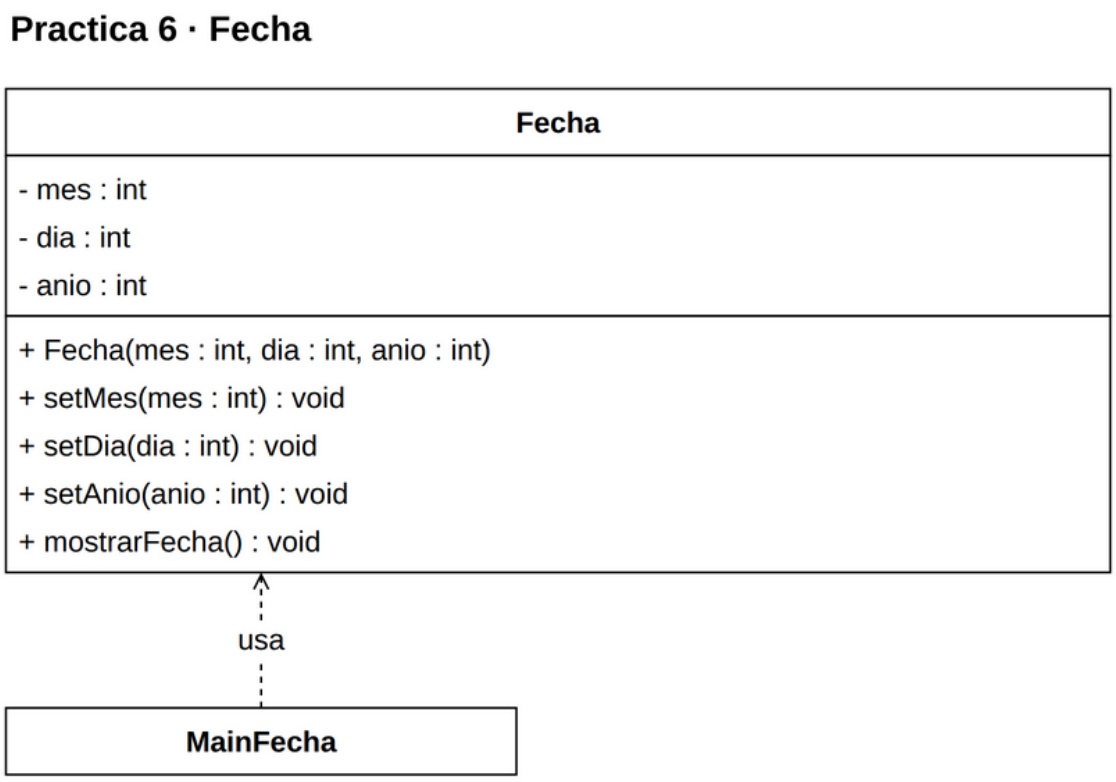


Figura 3.27: Práctica 6 — p5.Fecha.

La figura 3.28 muestra p5.MainEmpleado.

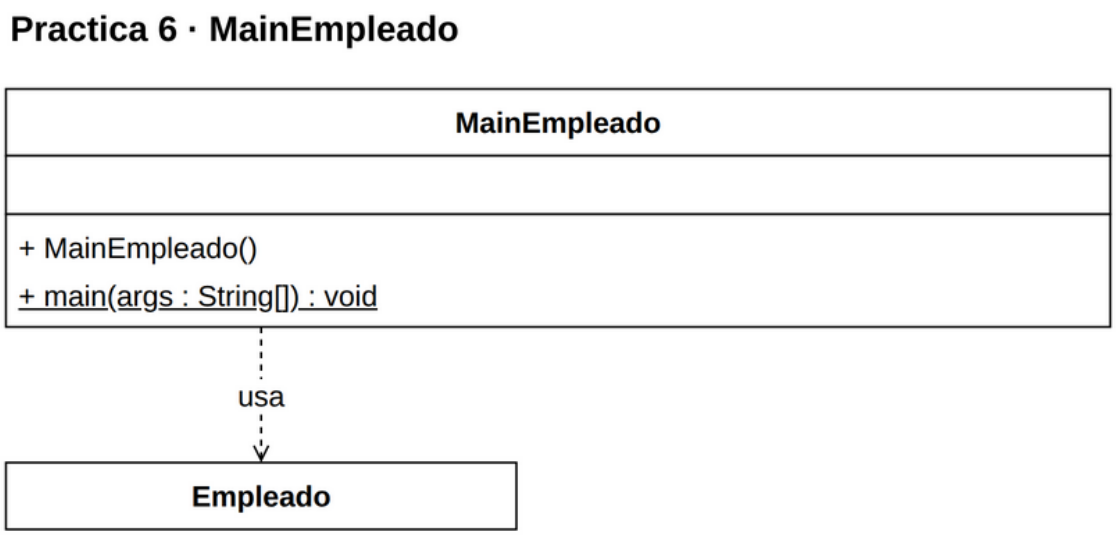


Figura 3.28: Práctica 6 — p5.MainEmpleado.

La figura 3.29 muestra p5.MainFecha.

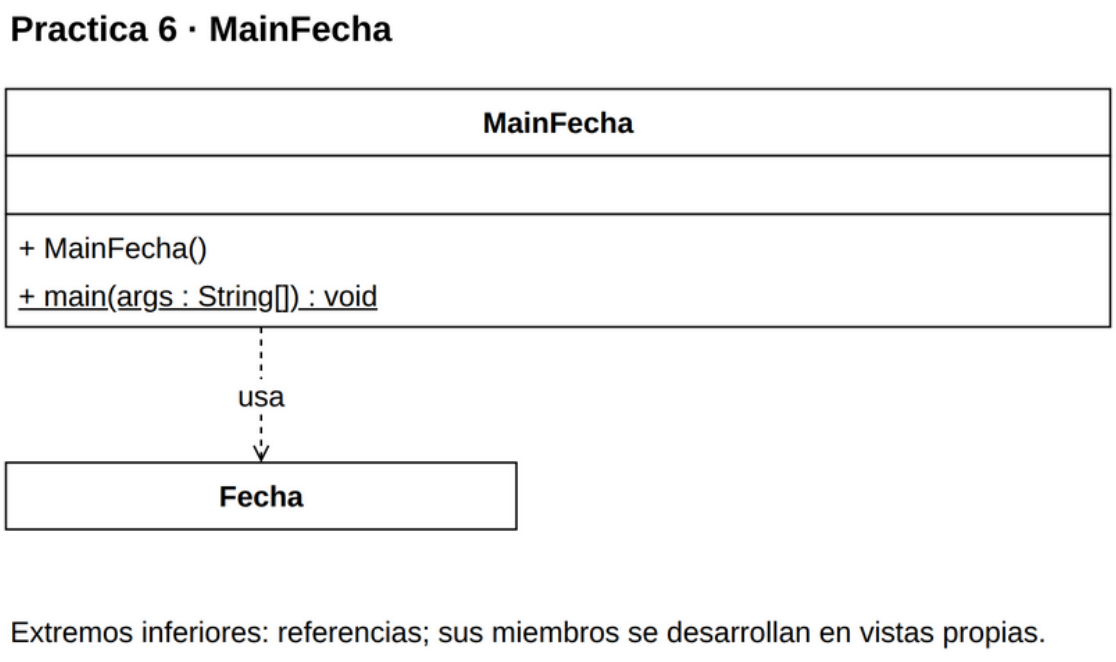


Figura 3.29: Práctica 6 — p5.MainFecha.

La figura 3.30 muestra proyecto.Enfermero, parte 1 de 2.

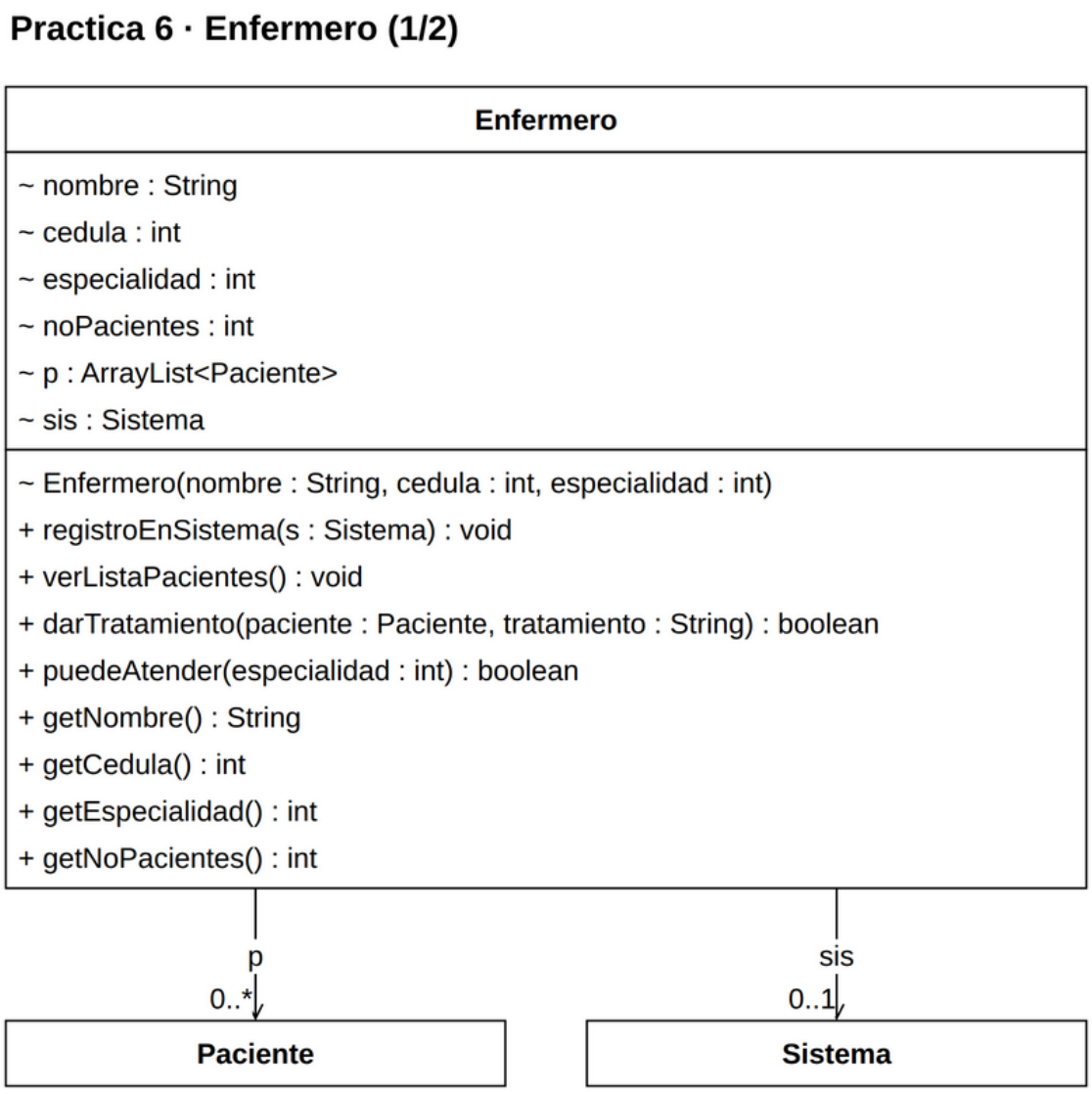


Figura 3.30: Práctica 6 — proyecto.Enfermero. Parte 1 de 2.

La figura 3.31 muestra proyecto.Enfermero, parte 2 de 2.

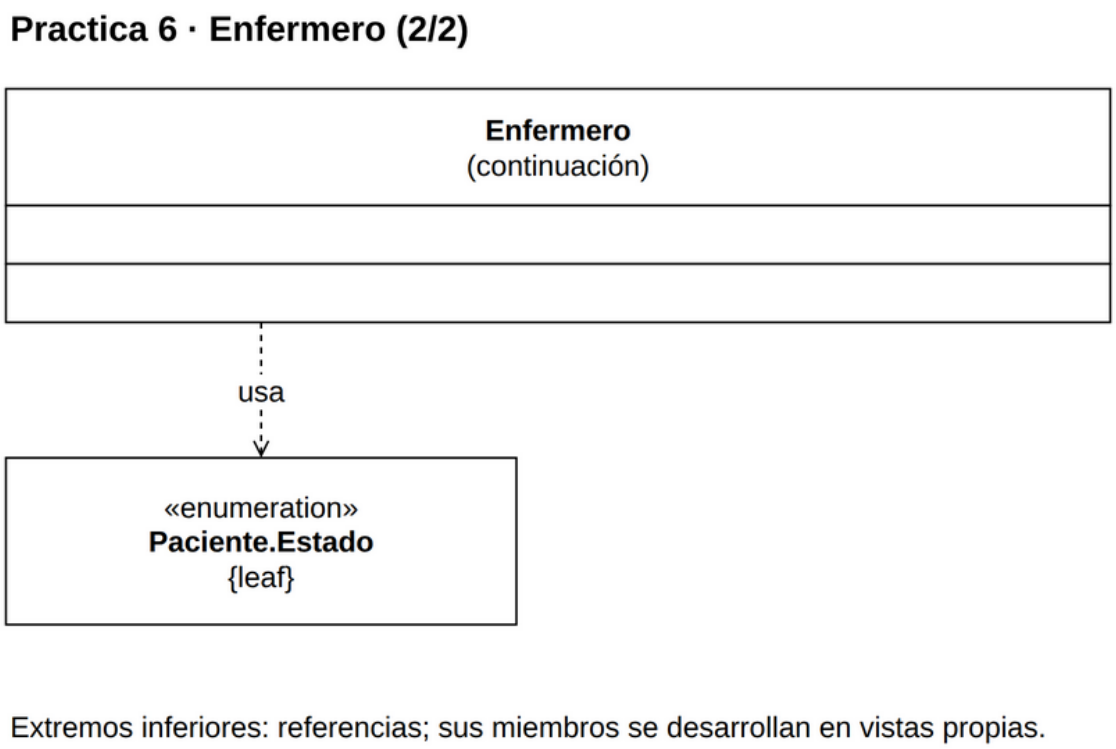


Figura 3.31: Práctica 6 — proyecto.Enfermero. Parte 2 de 2.

La figura 3.32 muestra proyecto.Main, parte 1 de 2.

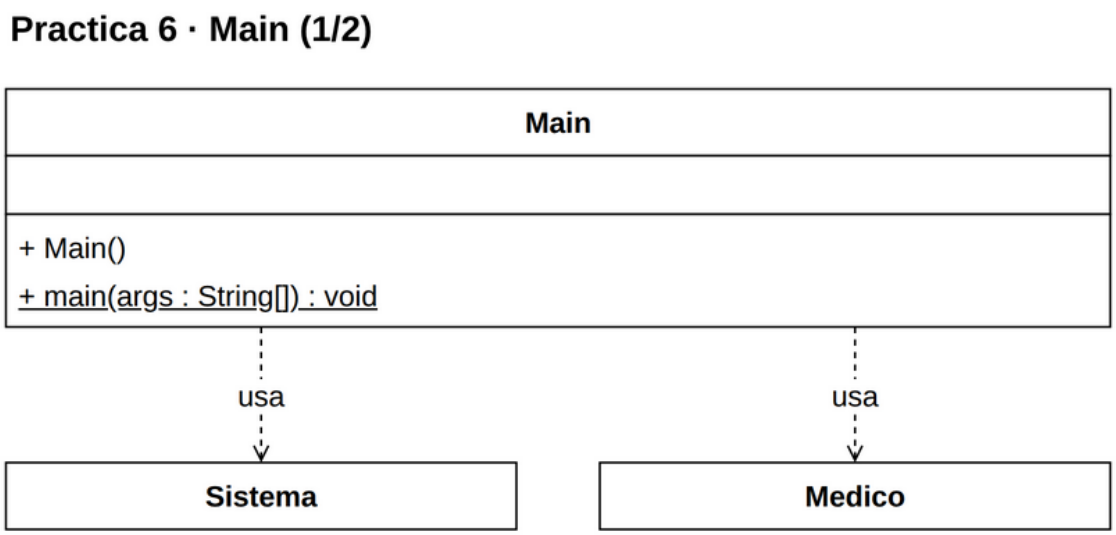


Figura 3.32: Práctica 6 — proyecto.Main. Parte 1 de 2.

La figura 3.33 muestra proyecto.Main, parte 2 de 2.

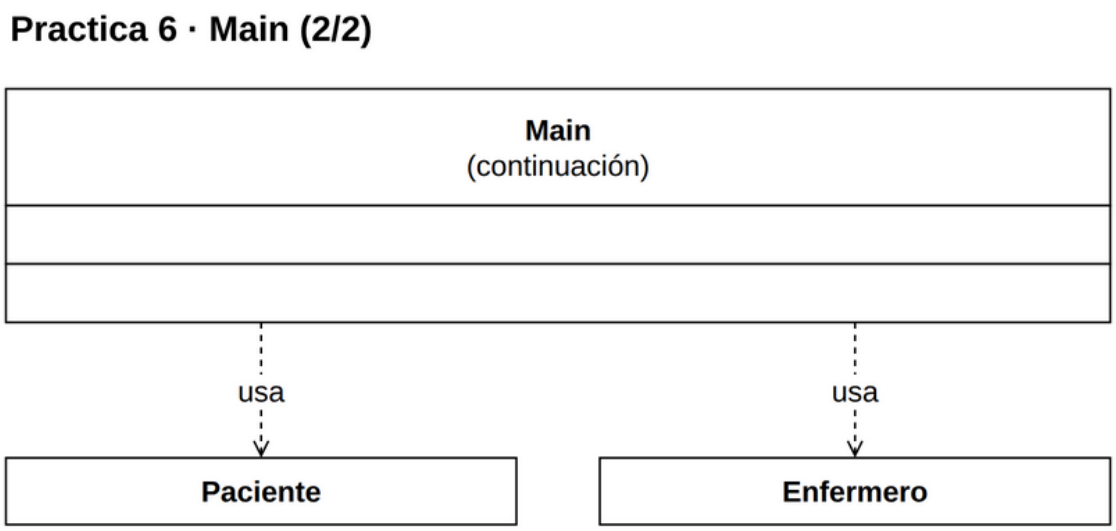


Figura 3.33: Práctica 6 — proyecto.Main. Parte 2 de 2.

La figura 3.34 muestra proyecto.Medico, parte 1 de 2.

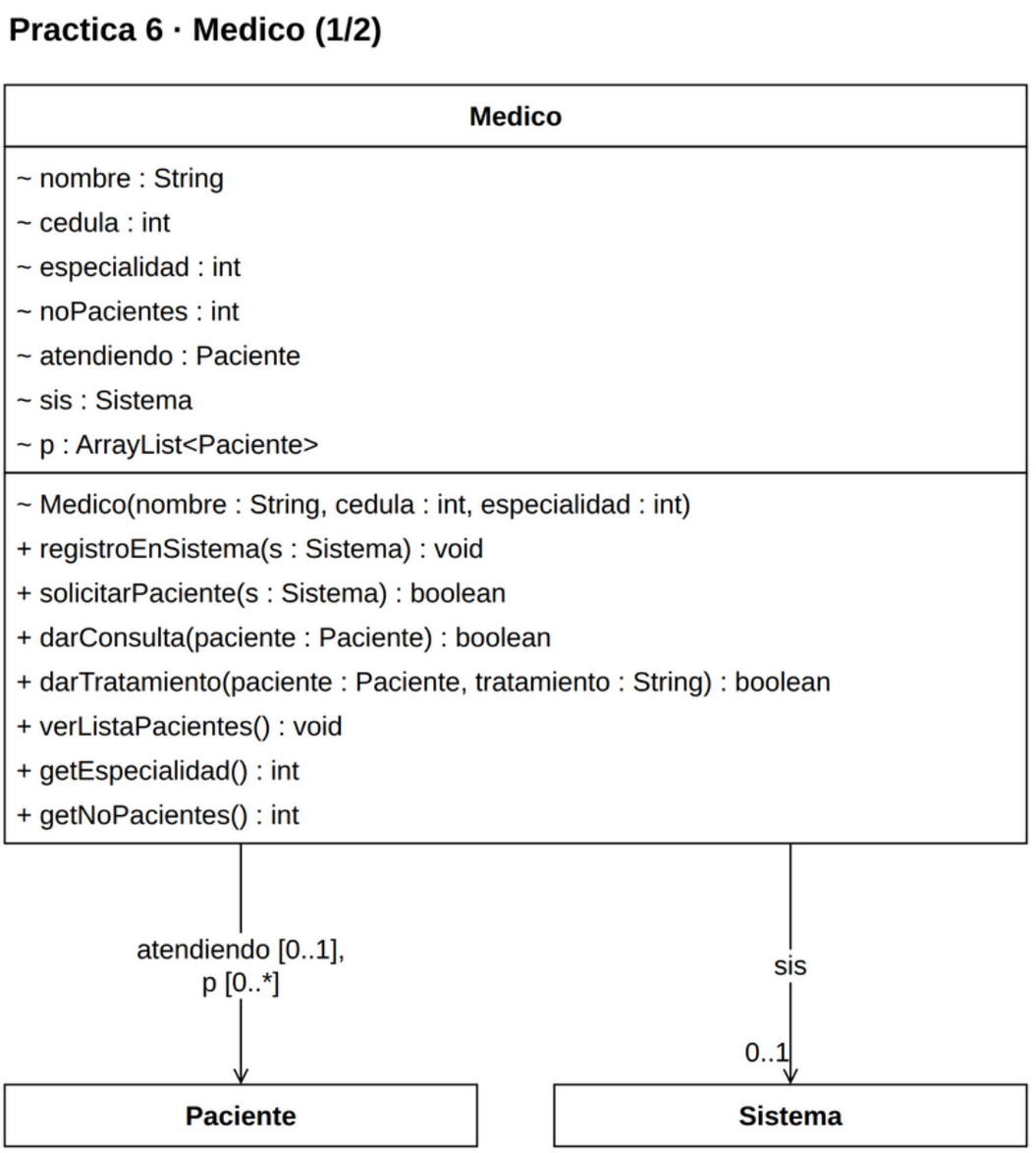


Figura 3.34: Práctica 6 — proyecto.Medico. Parte 1 de 2.

La figura 3.35 muestra proyecto.Medico, parte 2 de 2.

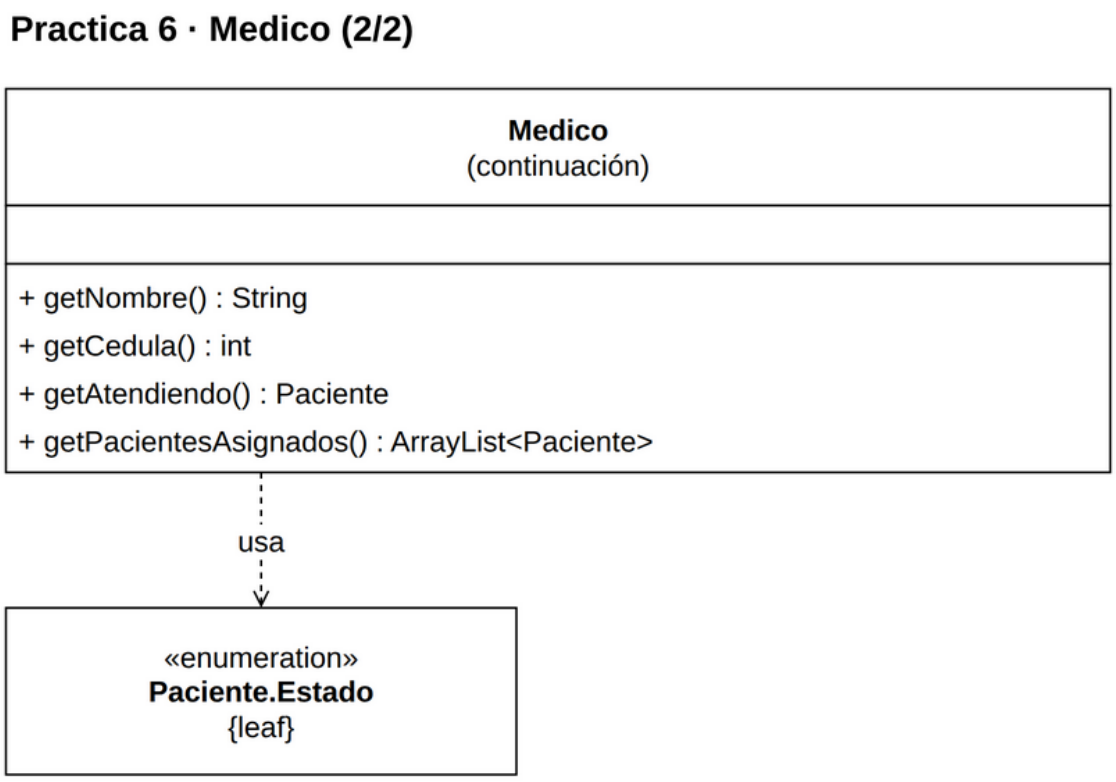


Figura 3.35: Práctica 6 — proyecto.Medico. Parte 2 de 2.

La figura 3.36 muestra proyecto.Paciente, parte 1 de 2.

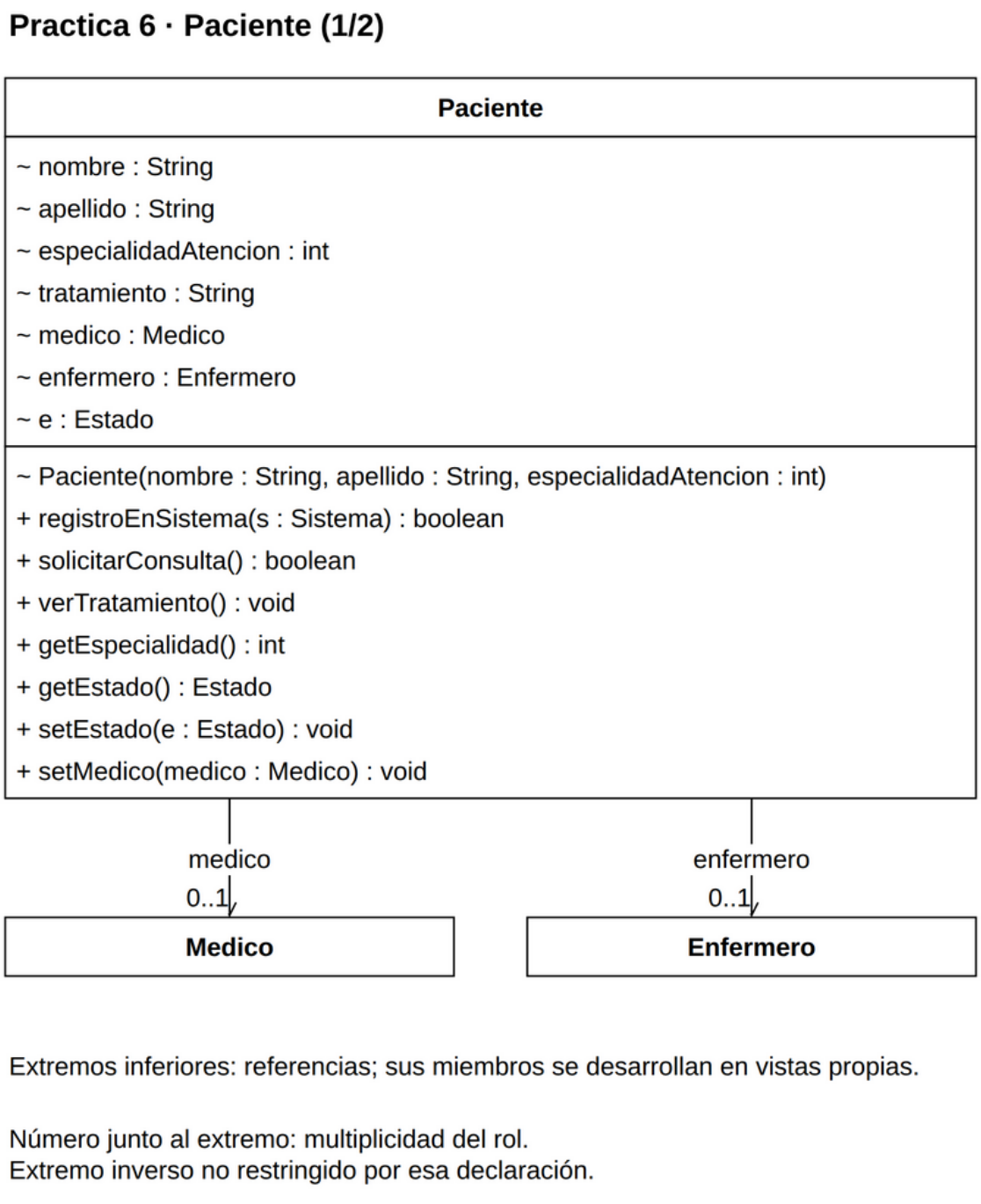


Figura 3.36: Práctica 6 — proyecto.Paciente. Parte 1 de 2.

La figura 3.37 muestra proyecto.Paciente, parte 2 de 2.

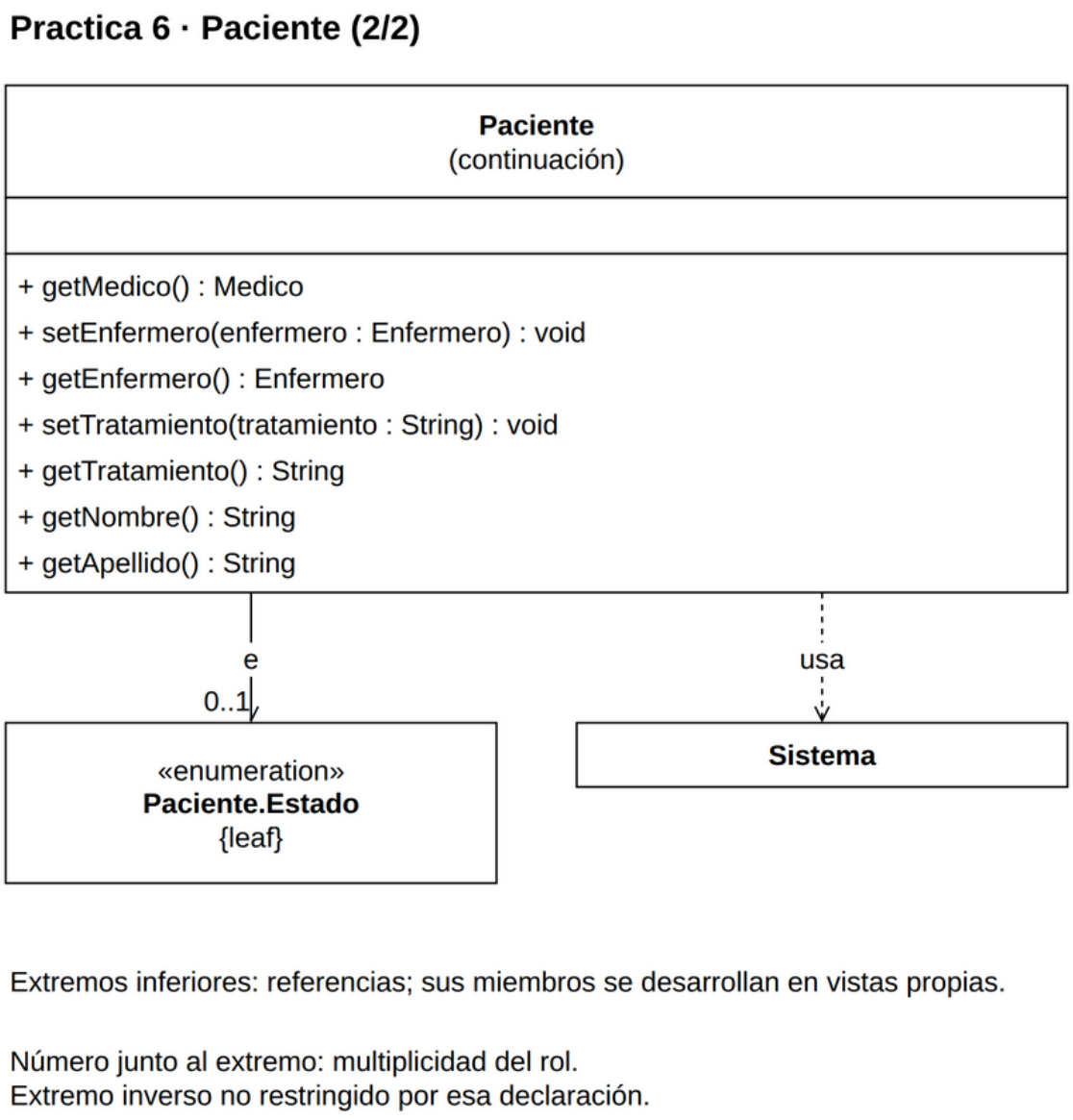


Figura 3.37: Práctica 6 — proyecto.Paciente. Parte 2 de 2.

La figura 3.38 muestra proyecto.Paciente.Estado.

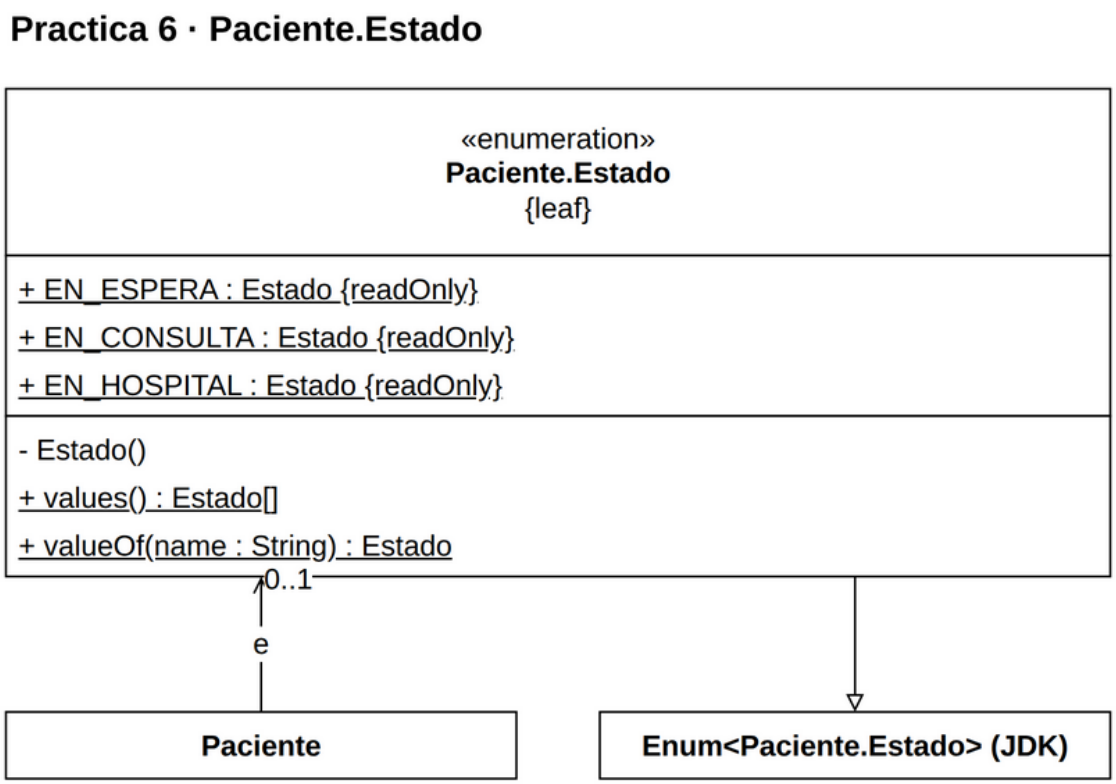


Figura 3.38: Práctica 6 — proyecto.Paciente.Estado.

La figura 3.39 muestra proyecto.Sistema, parte 1 de 2.

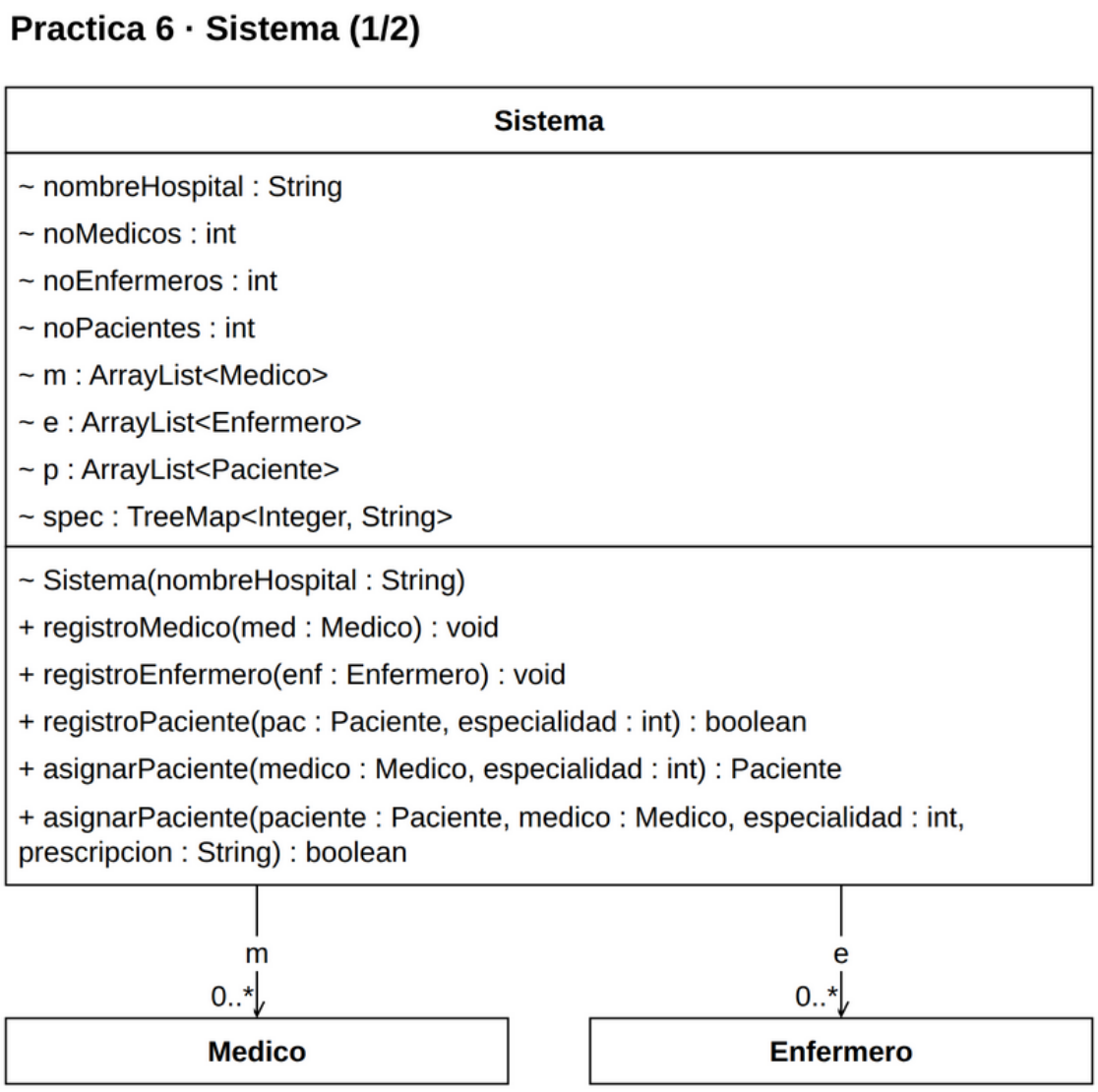


Figura 3.39: Práctica 6 — proyecto.Sistema. Parte 1 de 2.

La figura 3.40 muestra proyecto.Sistema, parte 2 de 2.

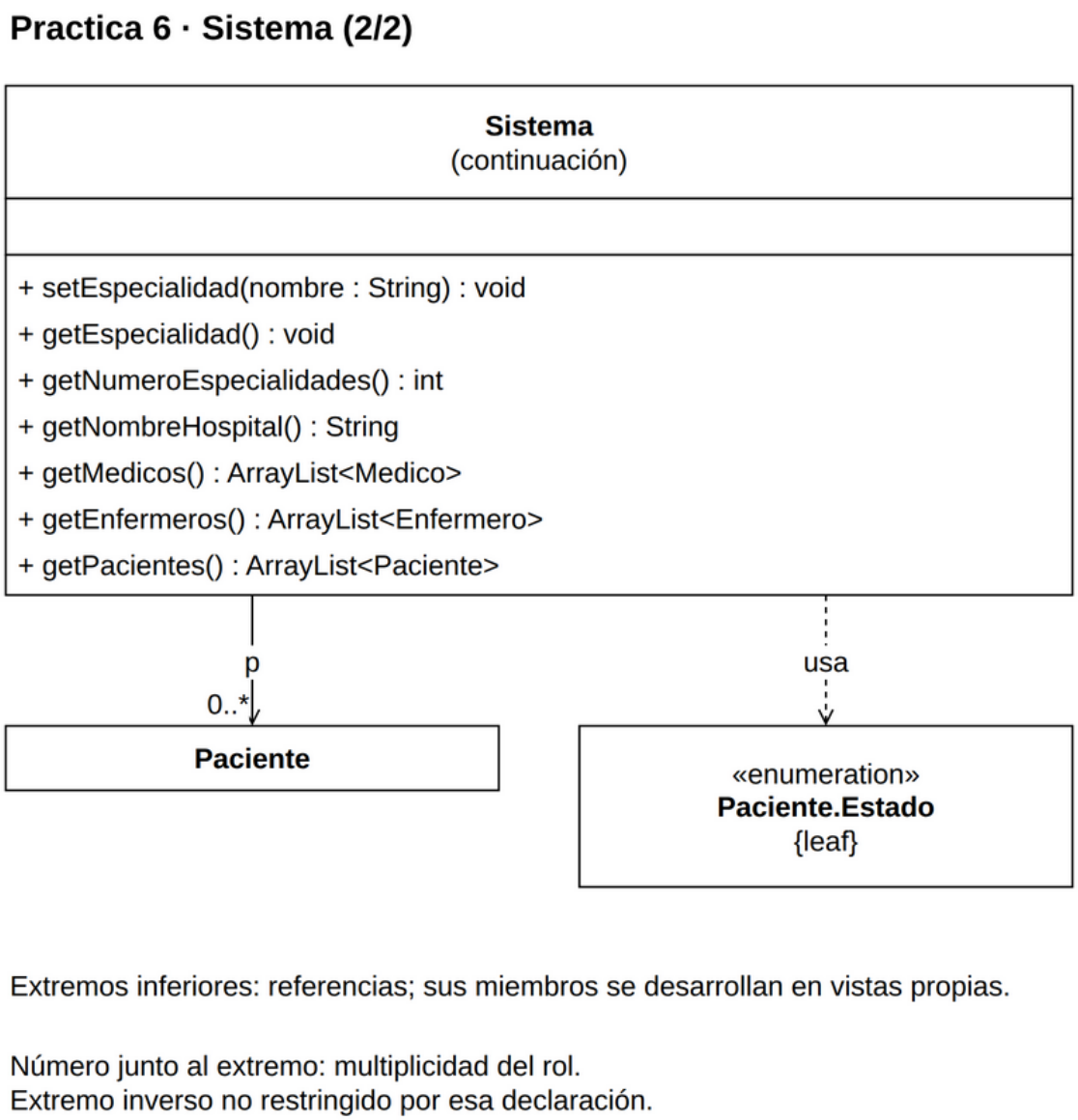


Figura 3.40: Práctica 6 — proyecto.Sistema. Parte 2 de 2.

3.3. Práctica 7: jerarquía de cuadriláteros

La cadena real es Cuadrado, Rectangulo, Paralelogramo, Trapezoide y Cuadrilatero, en ese orden hacia el tipo general. Cuadrilatero conserva p1, p2, p3 y p4 como referencias a Punto. Se representa cada rol como 0..1, porque sus constructores y modificadores aceptan null. El programa no garantiza cuatro puntos distintos ni propiedad exclusiva; por eso se evita una composición o una multiplicidad de cuatro objetos obligatoriamente diferentes. [3].

La figura 3.41 muestra Cuadrado.

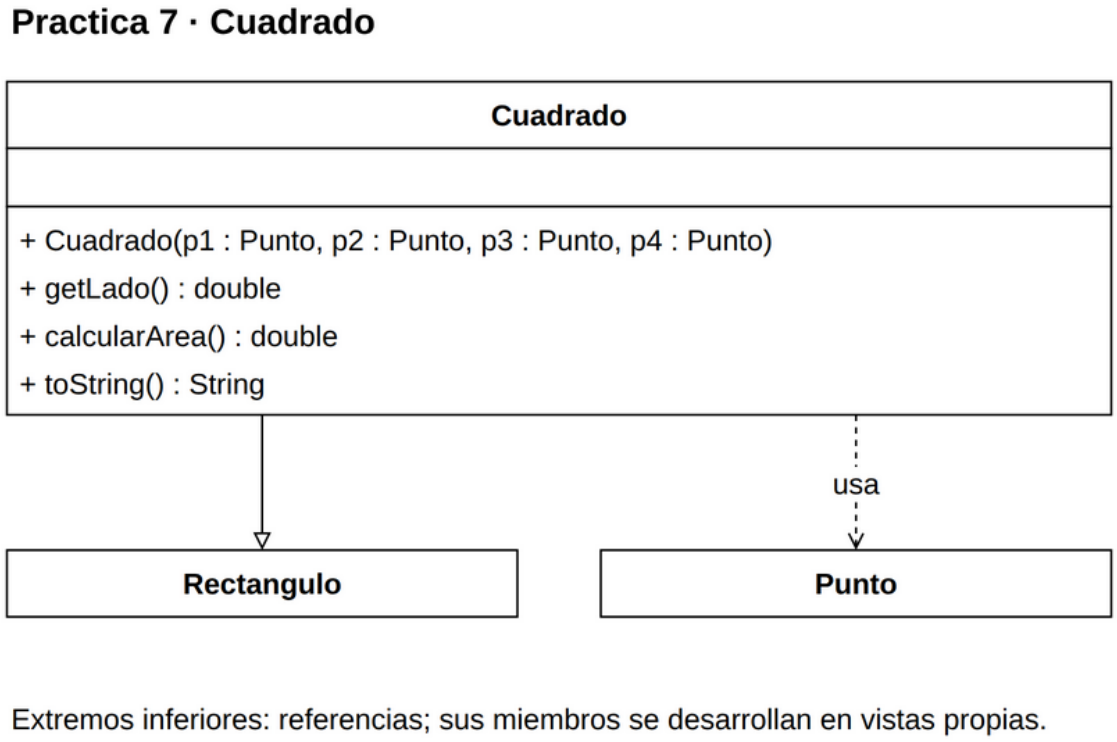


Figura 3.41: Práctica 7 — Cuadrado.

La figura 3.42 muestra Cuadrilatero.

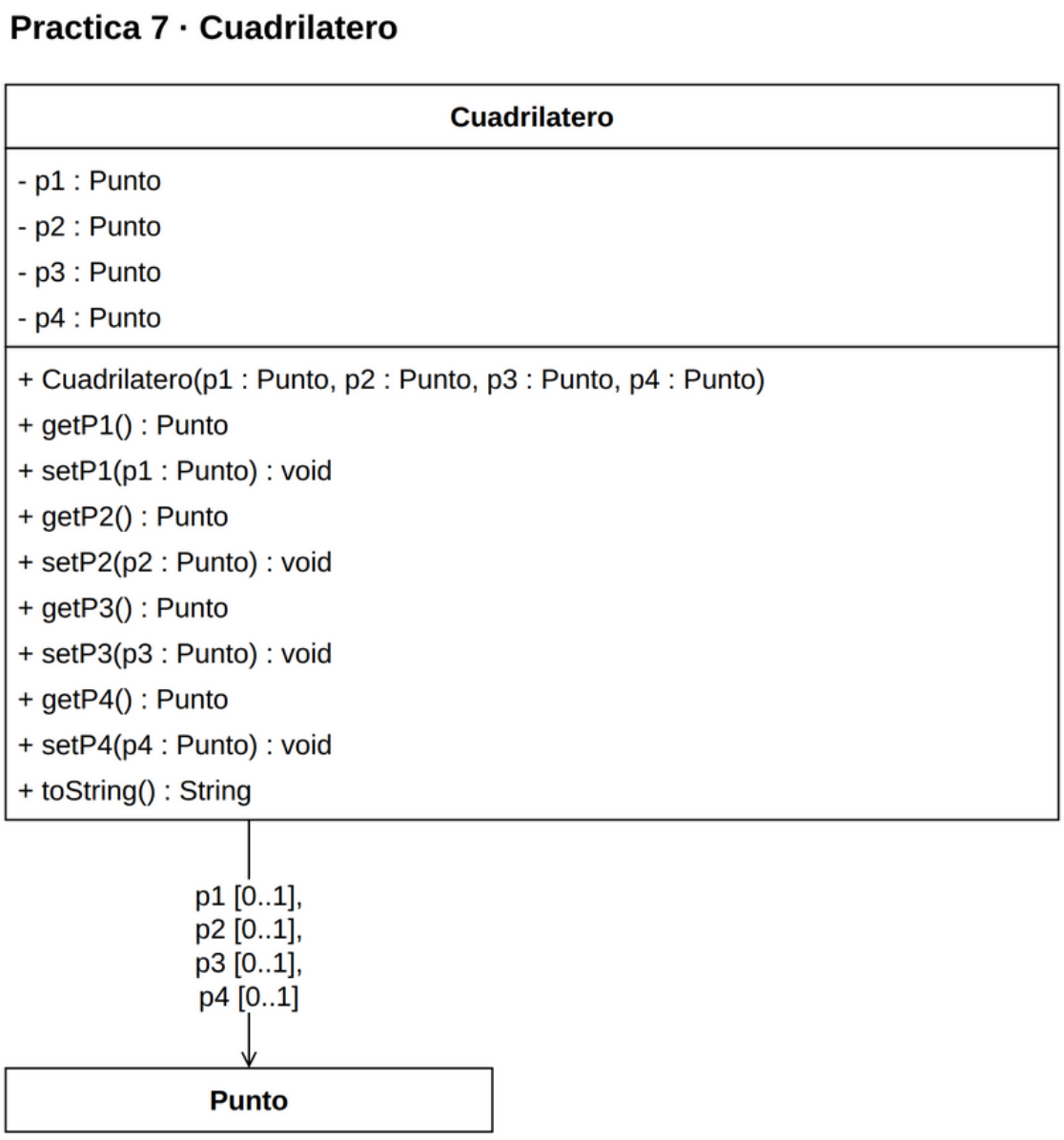


Figura 3.42: Práctica 7 — Cuadrilatero.

La figura 3.43 muestra Main, parte 1 de 3.

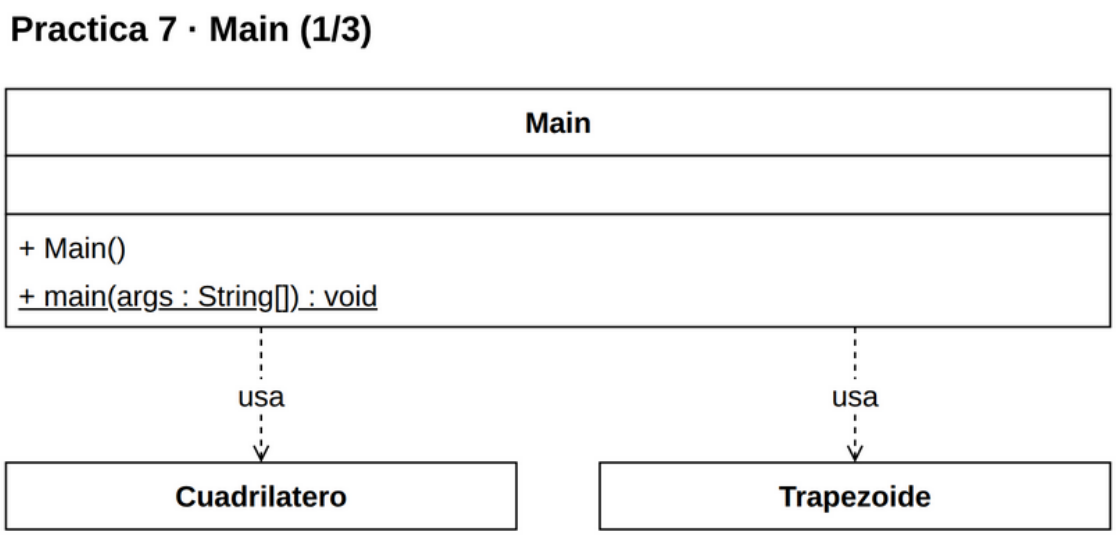


Figura 3.43: Práctica 7 — Main. Parte 1 de 3.

La figura 3.44 muestra Main, parte 2 de 3.

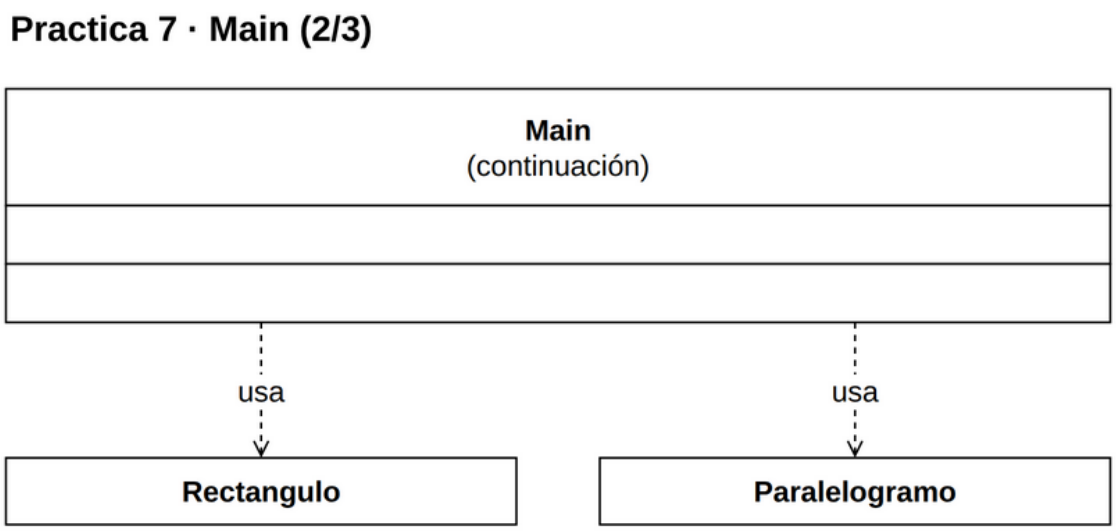


Figura 3.44: Práctica 7 — Main. Parte 2 de 3.

La figura 3.45 muestra Main, parte 3 de 3.

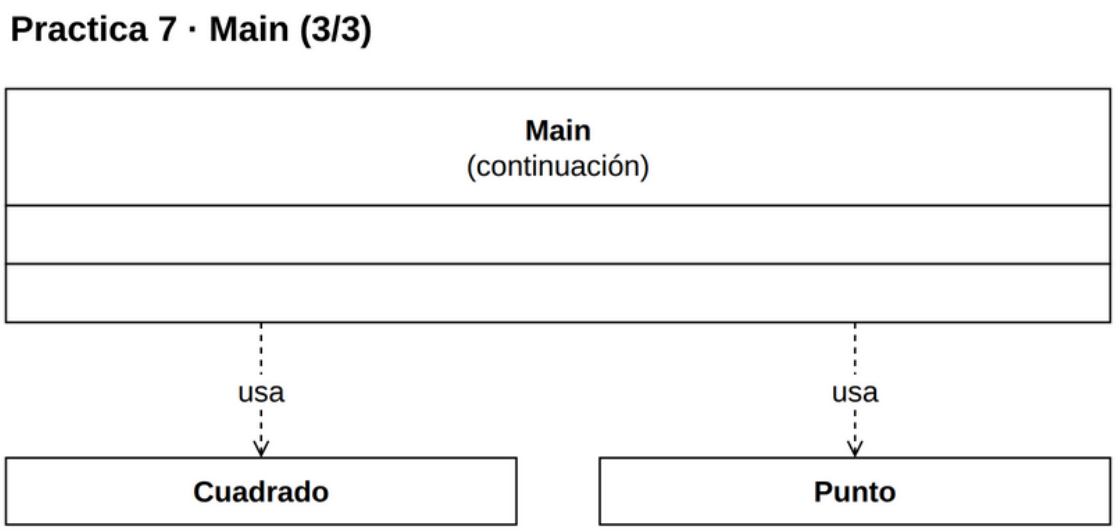


Figura 3.45: Práctica 7 — Main. Parte 3 de 3.

La figura 3.46 muestra Paralelogramo.

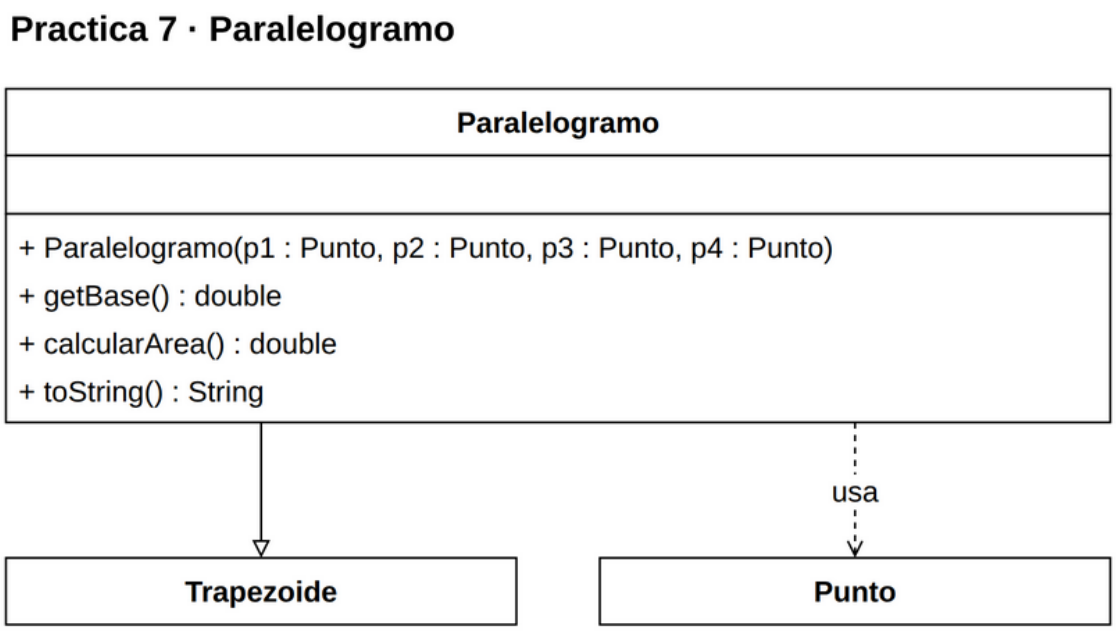


Figura 3.46: Práctica 7 — Paralelogramo.

La figura 3.47 muestra Punto.

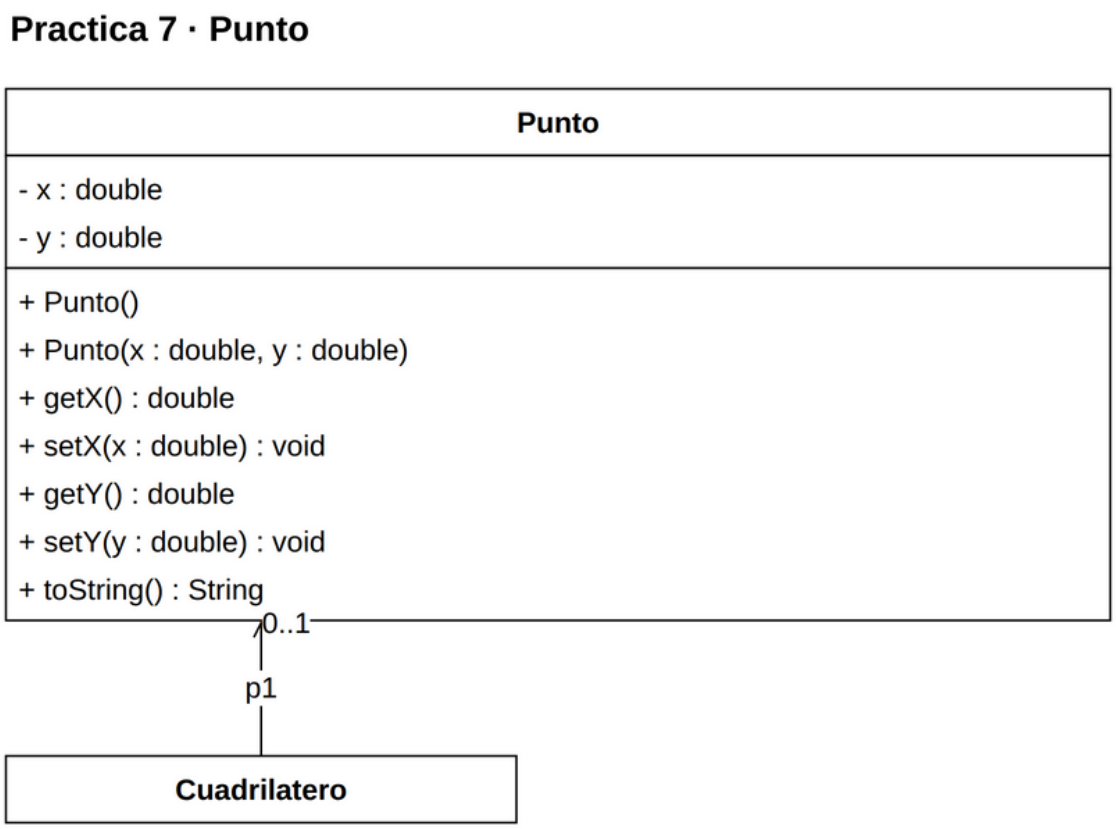


Figura 3.47: Práctica 7 — Punto.

La figura 3.48 muestra Rectangulo.

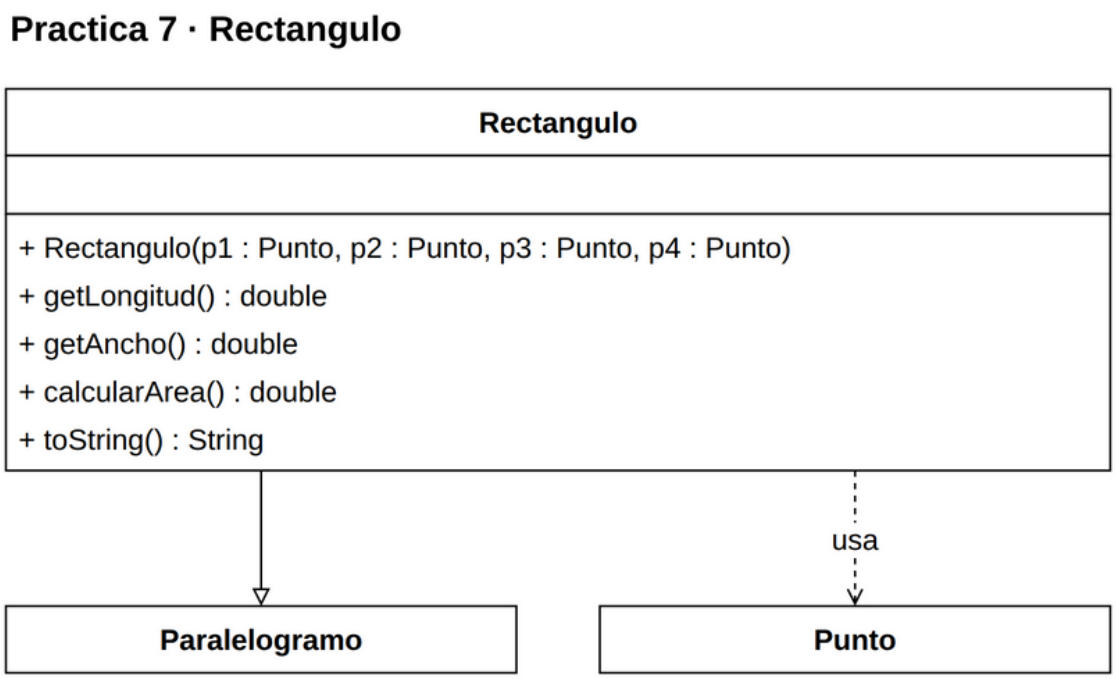


Figura 3.48: Práctica 7 — Rectangulo.

La figura 3.49 muestra Trapezoide.

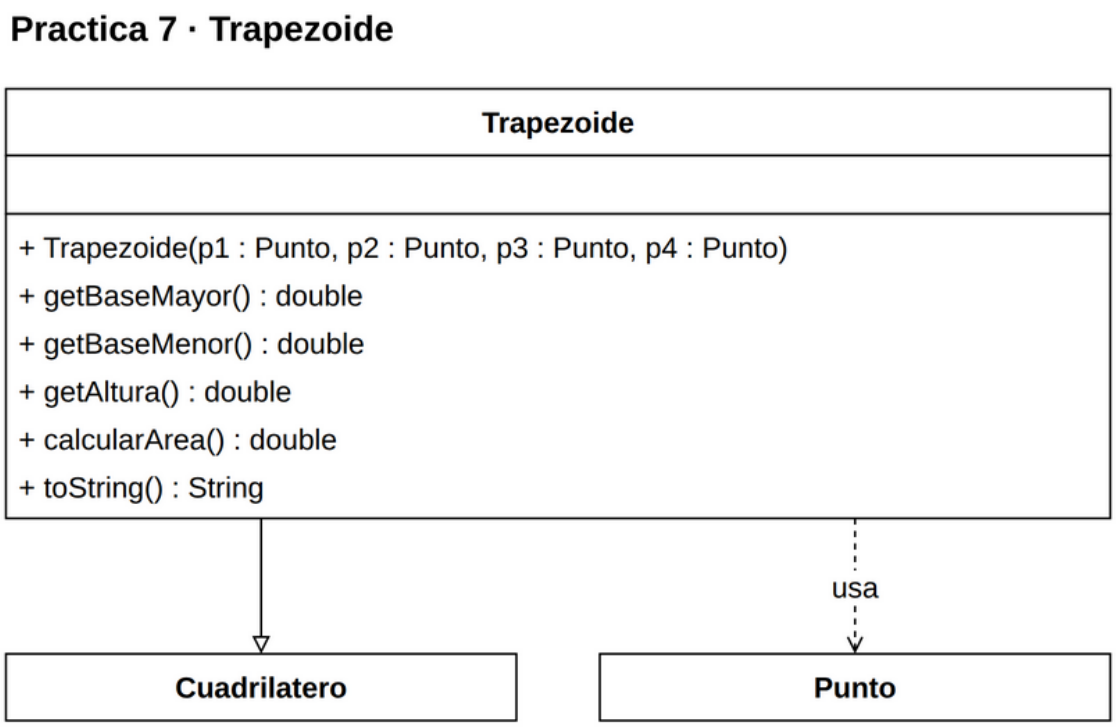


Figura 3.49: Práctica 7 — Trapezoide.

3.4. Práctica 8: contratos e interfaz gráfica

Auto, Bicicleta y Edificio realizan HuellaCarbono. Main depende de las implementaciones que crea y de la interfaz utilizada en el recorrido polimórfico. LoginGUI es un programa separado: hereda de JFrame y realiza ActionListener, ambos del JDK. No existe una llamada entre LoginGUI y las clases del cálculo de huellas. [3].

La figura 3.50 muestra Auto.

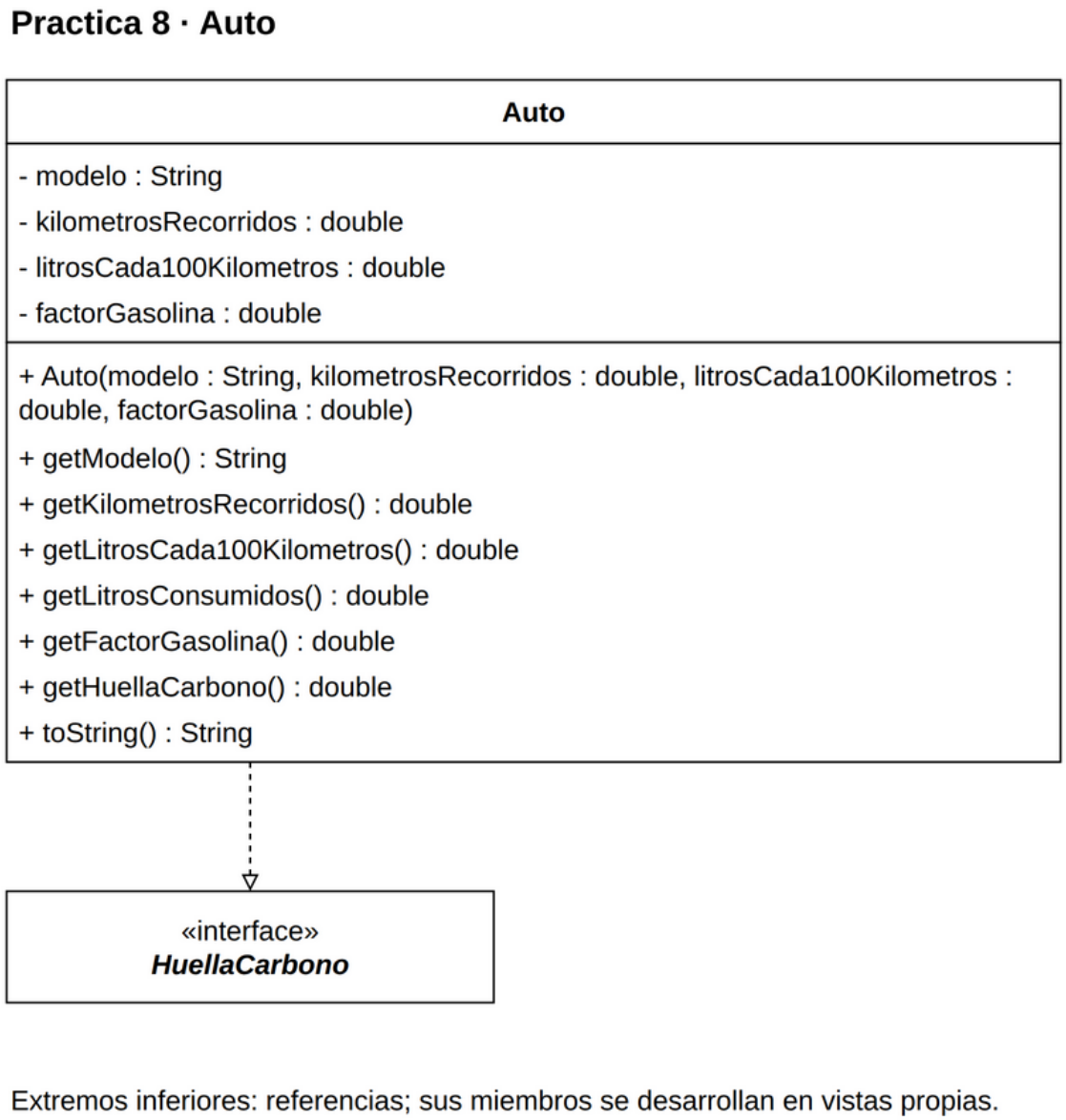


Figura 3.50: Práctica 8 — Auto.

La figura 3.51 muestra Bicicleta.

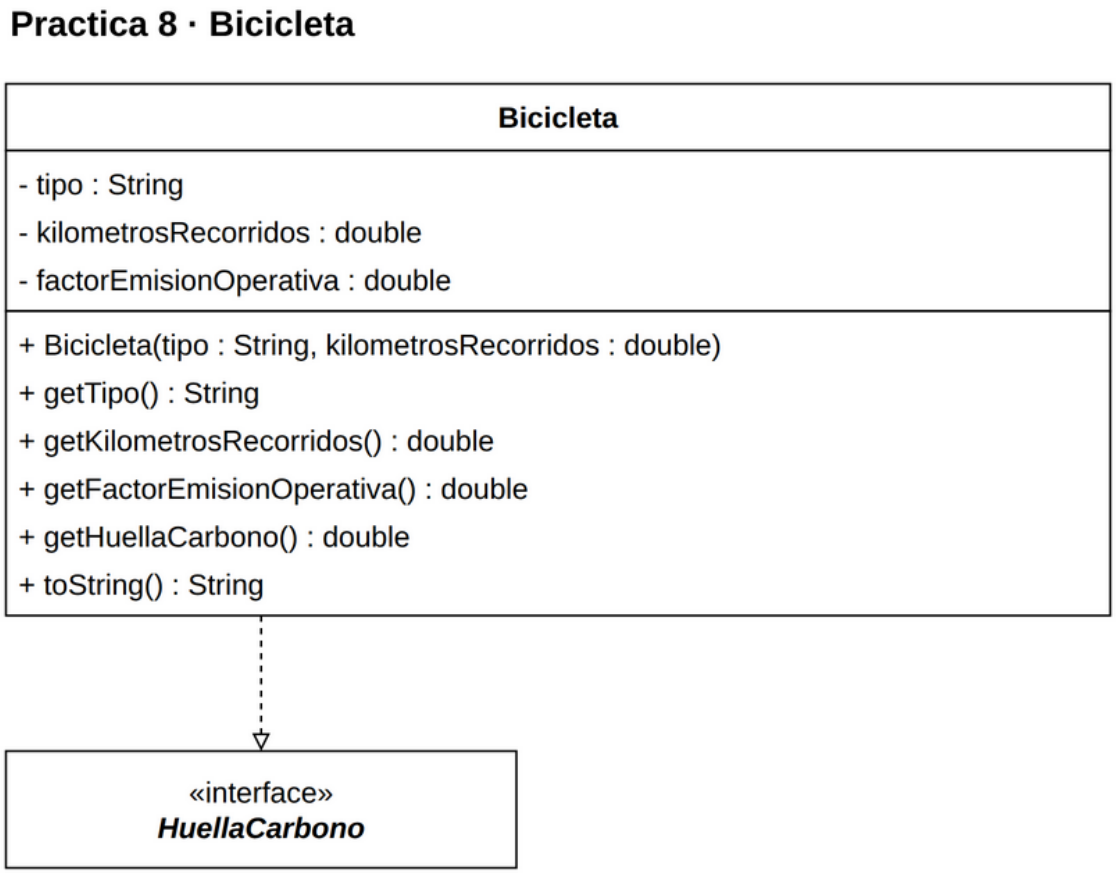


Figura 3.51: Práctica 8 — Bicicleta.

La figura 3.52 muestra Edificio.

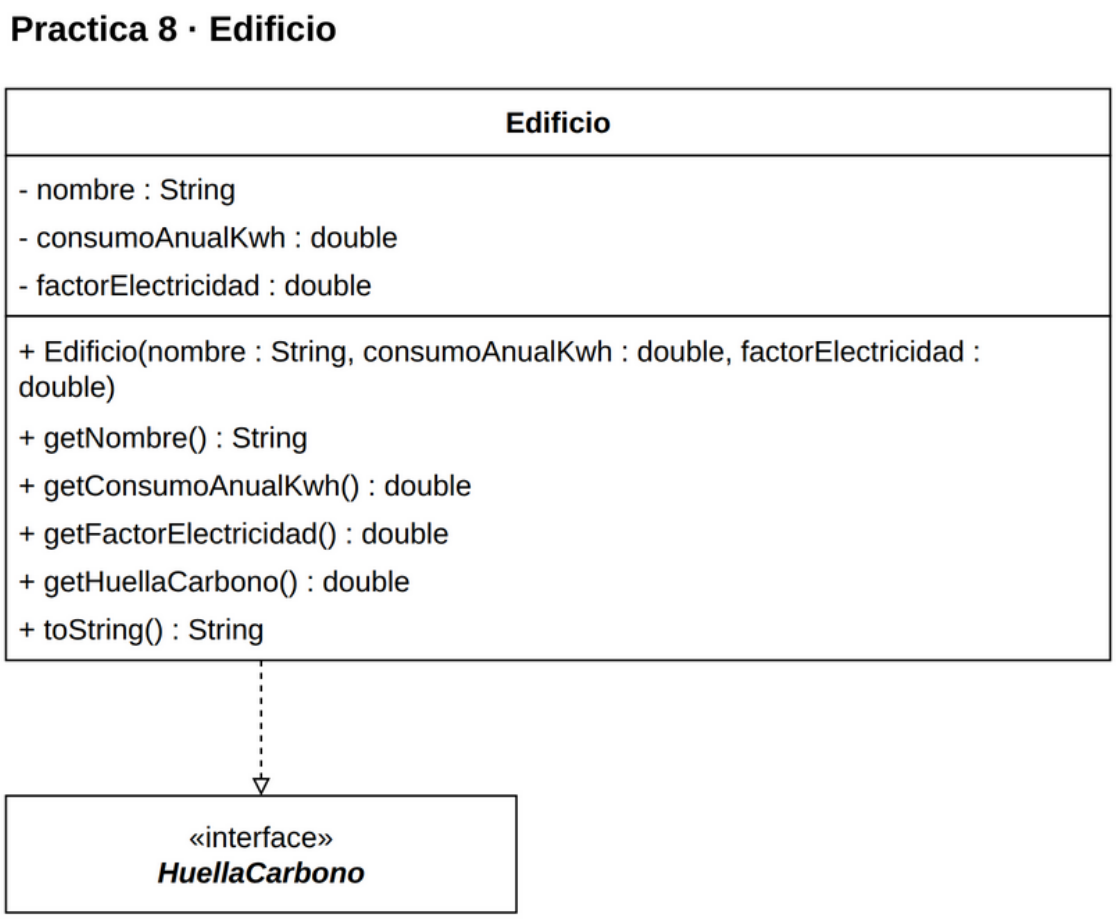


Figura 3.52: Práctica 8 — Edificio.

La figura 3.53 muestra HuellaCarbono.

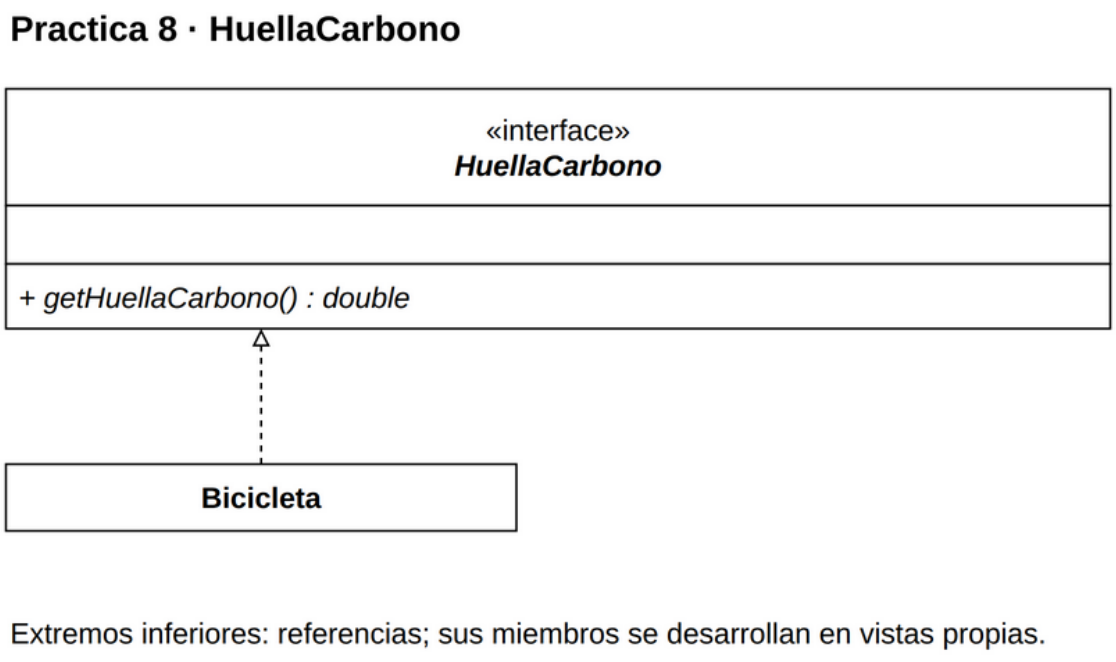


Figura 3.53: Práctica 8 — HuellaCarbono.

La figura 3.54 muestra LoginGUI.

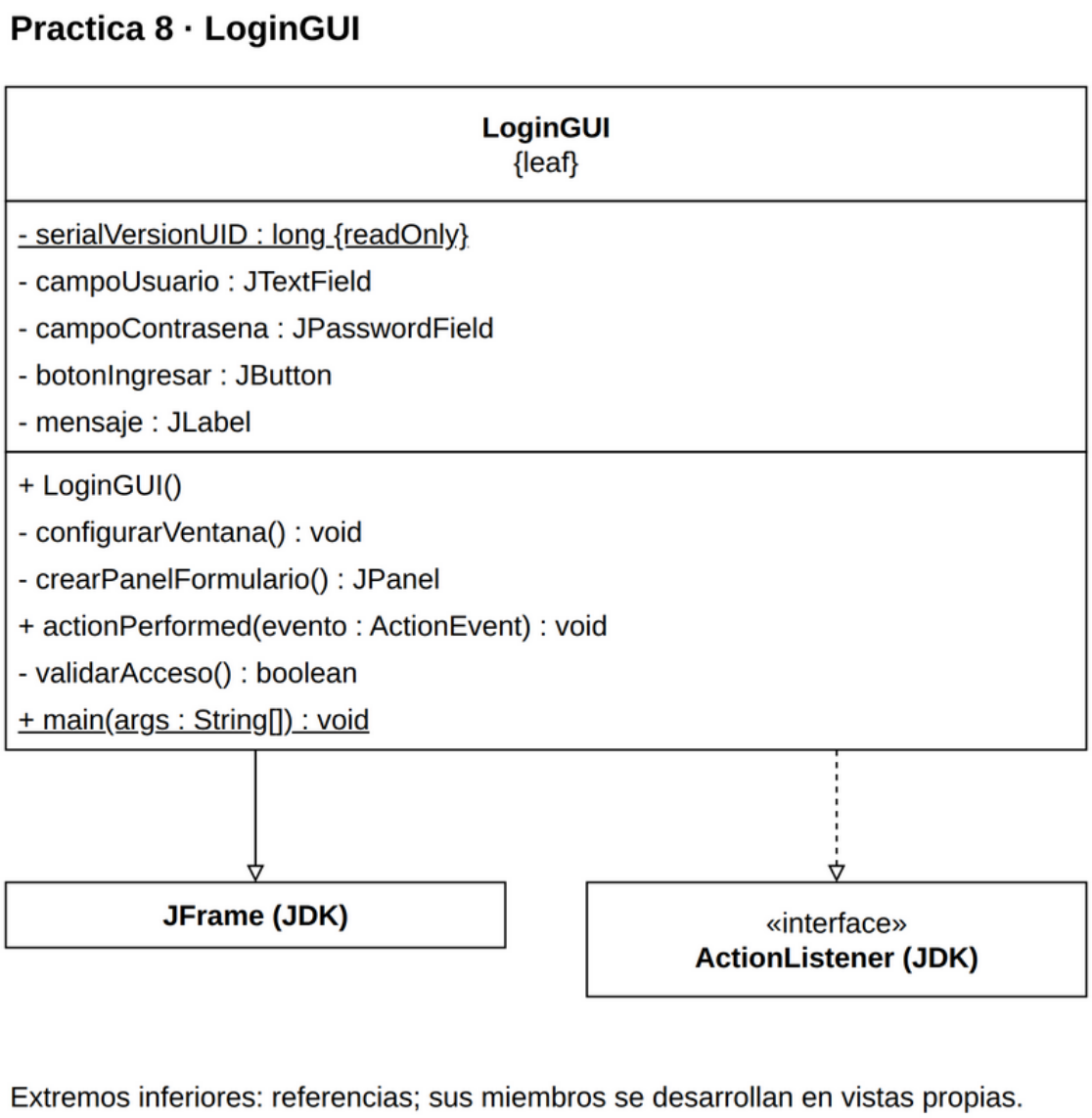


Figura 3.54: Práctica 8 — LoginGUI.

La figura 3.55 muestra Main, parte 1 de 2.

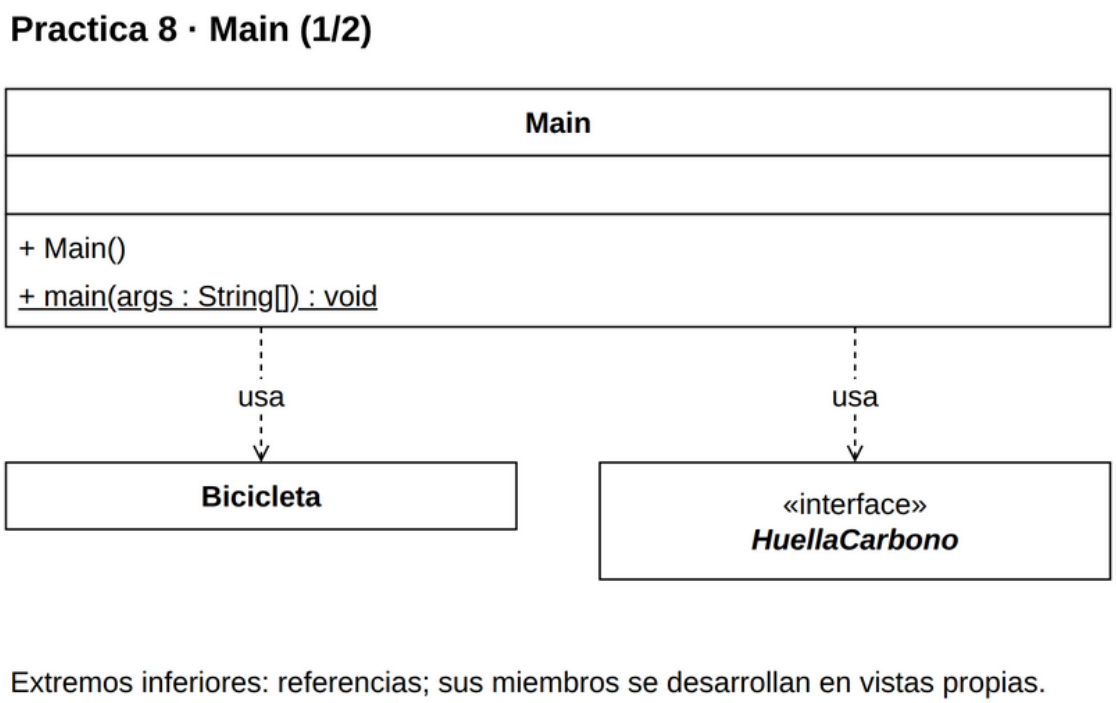


Figura 3.55: Práctica 8 — Main. Parte 1 de 2.

La figura 3.56 muestra Main, parte 2 de 2.

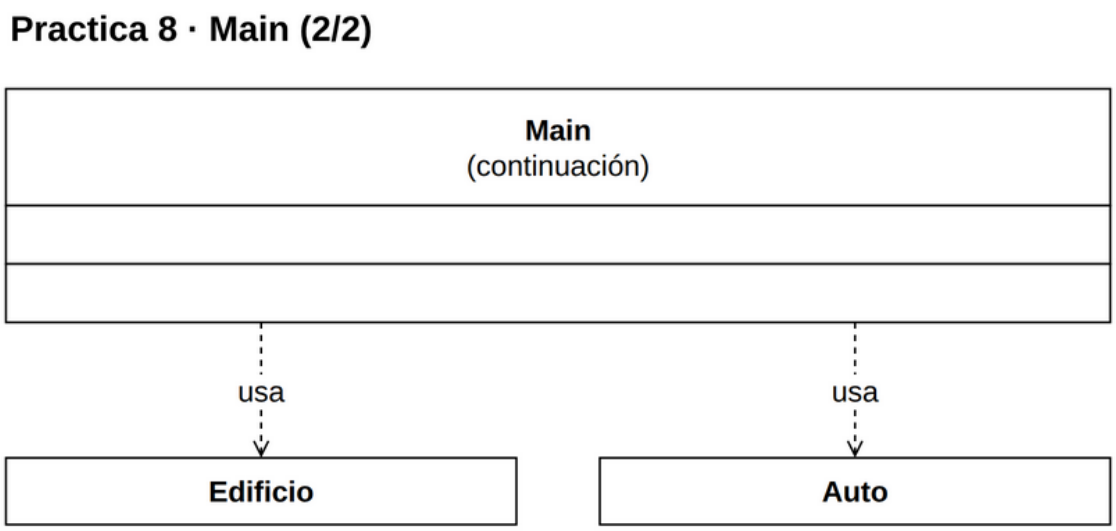


Figura 3.56: Práctica 8 — Main. Parte 2 de 2.

3.5. Proyecto fase 2: sistema hospitalario

El sistema separa pacientes, personal, contrato de atención y coordinación. Medico, Enfermero y Paciente son abstractos; sus especializaciones heredan comportamientos y agregan sus datos particulares. Medico y Enfermero realizan AtencionMedica. Sistema mantiene registros de las tres familias y las entidades conservan referencias al sistema. El atributo de médico en consulta admite 0..1 paciente. Las listas privadas y las validaciones de asignación permiten indicar 0..10 pacientes por médico y 0..3 por enfermero. JefeEnfermeria hereda de Enfermero, pero rechaza el tratamiento directo; su supervisión se consulta a través del sistema, sin inventar una colección permanente de equipo. Los estados se representan mediante la enumeración anidada Paciente.Estado. [4].

La figura 3.57 muestra AtencionMedica.

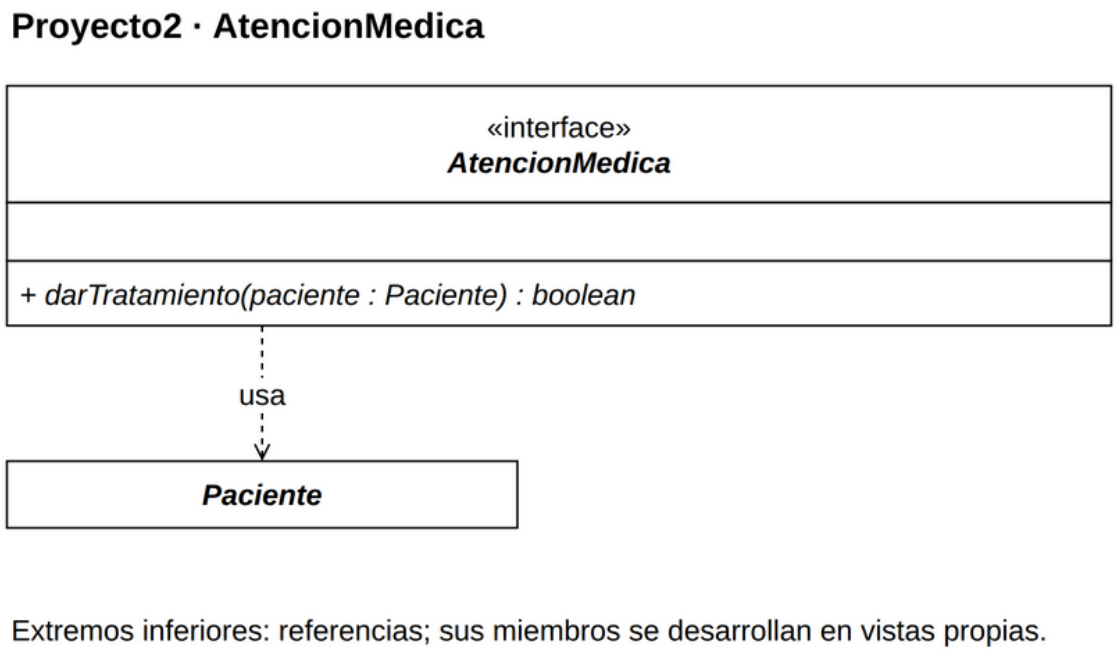


Figura 3.57: Proyecto fase 2 — AtencionMedica.

La figura 3.58 muestra Main, parte 1 de 6.

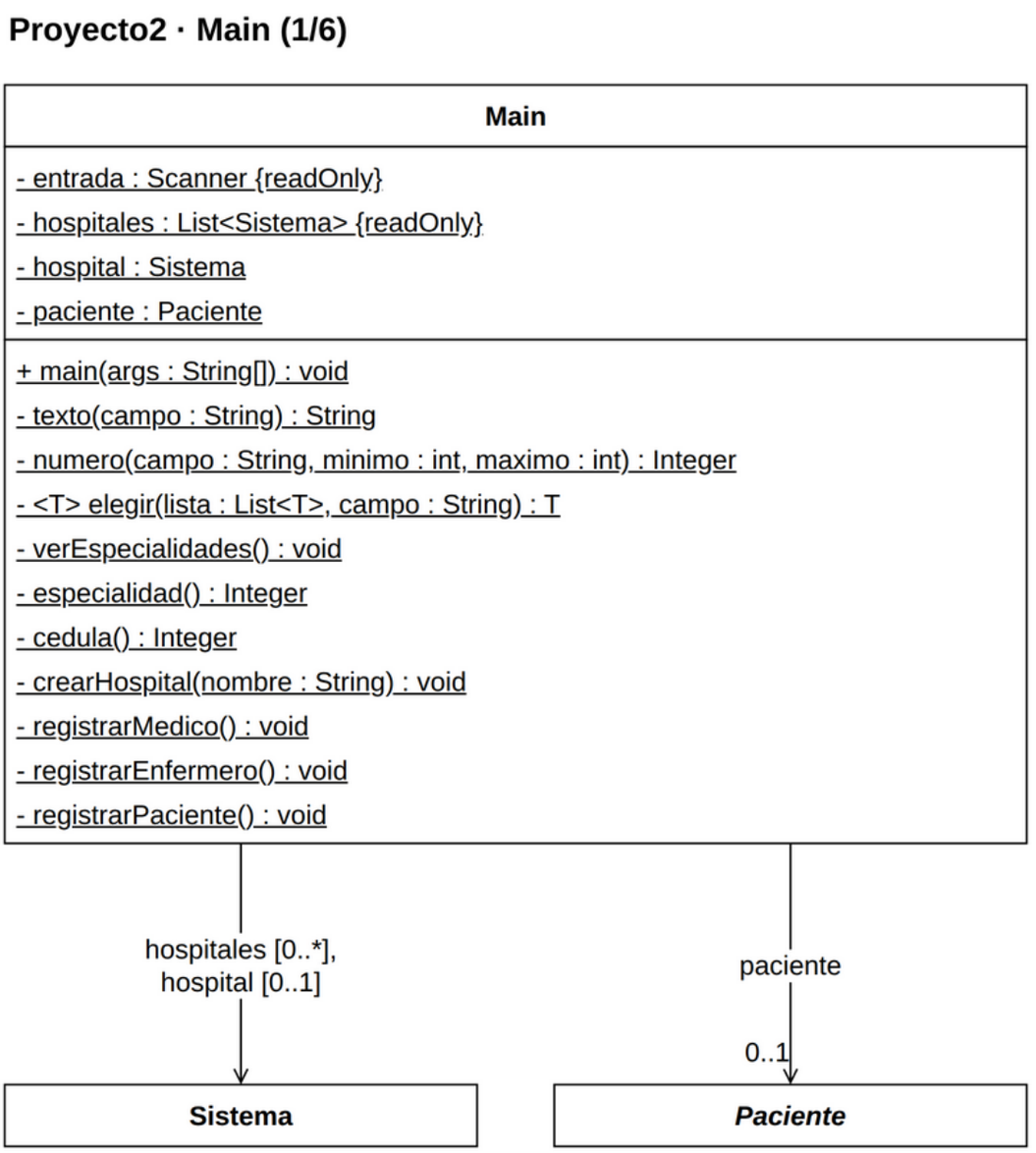


Figura 3.58: Proyecto fase 2 — Main. Parte 1 de 6.

La figura 3.59 muestra Main, parte 2 de 6.

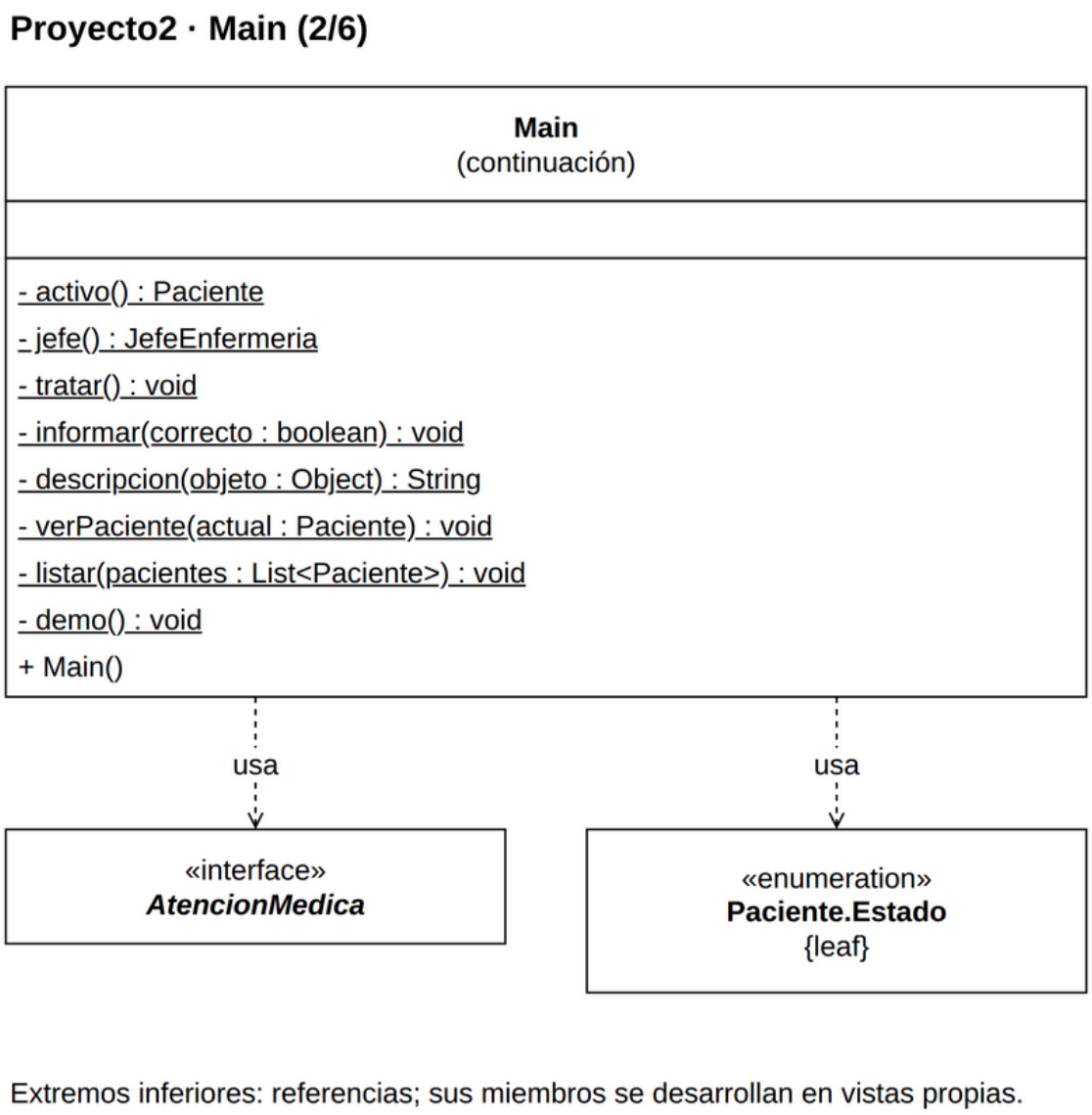


Figura 3.59: Proyecto fase 2 — Main. Parte 2 de 6.

La figura 3.60 muestra Main, parte 3 de 6.

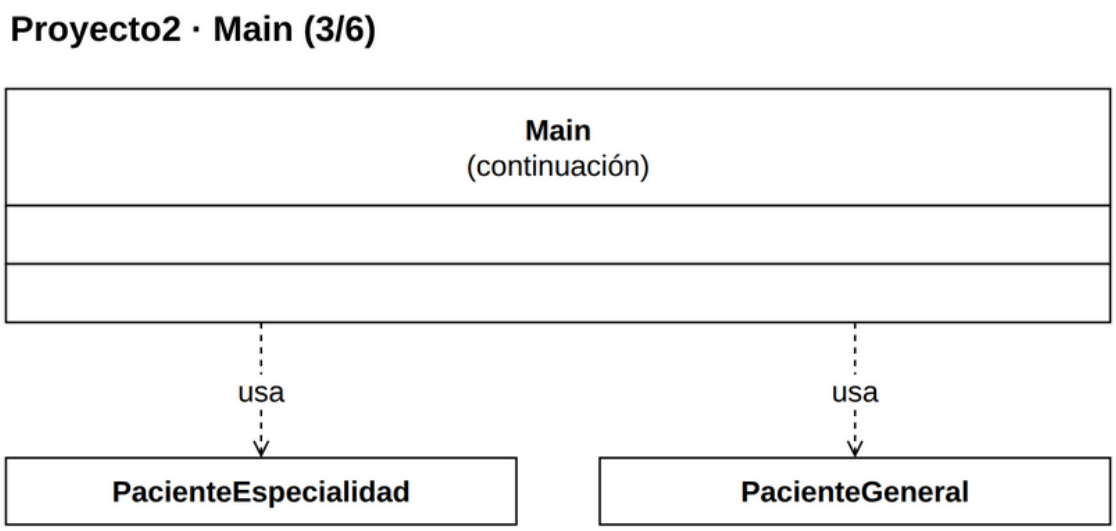


Figura 3.60: Proyecto fase 2 — Main. Parte 3 de 6.

La figura 3.61 muestra Main, parte 4 de 6.

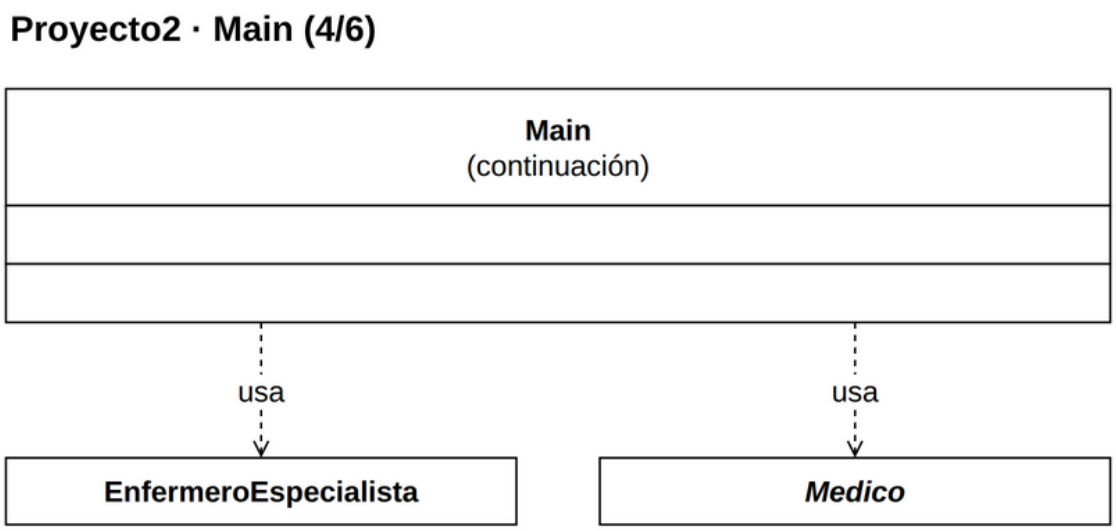


Figura 3.61: Proyecto fase 2 — Main. Parte 4 de 6.

La figura 3.62 muestra Main, parte 5 de 6.

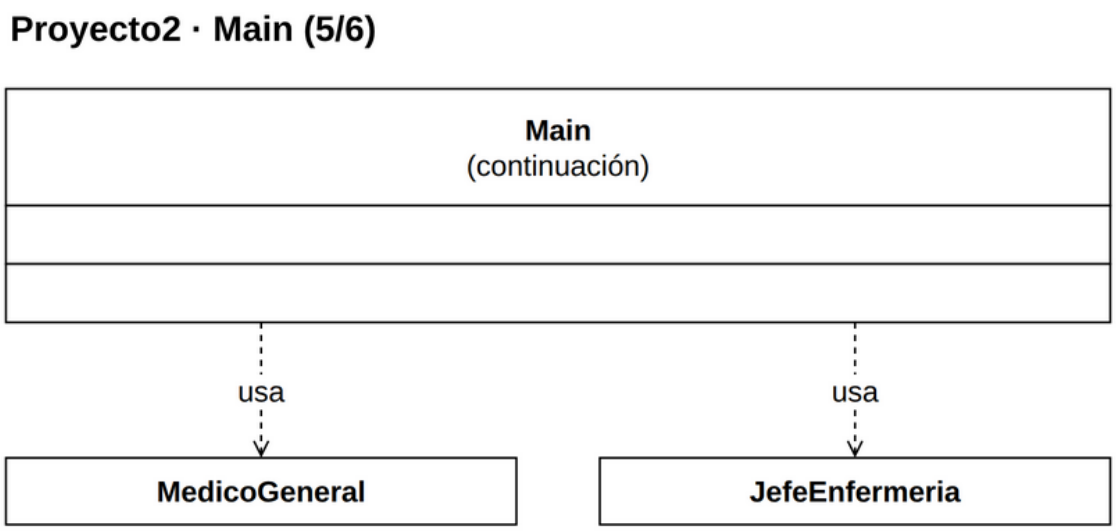


Figura 3.62: Proyecto fase 2 — Main. Parte 5 de 6.

La figura 3.63 muestra Main, parte 6 de 6.

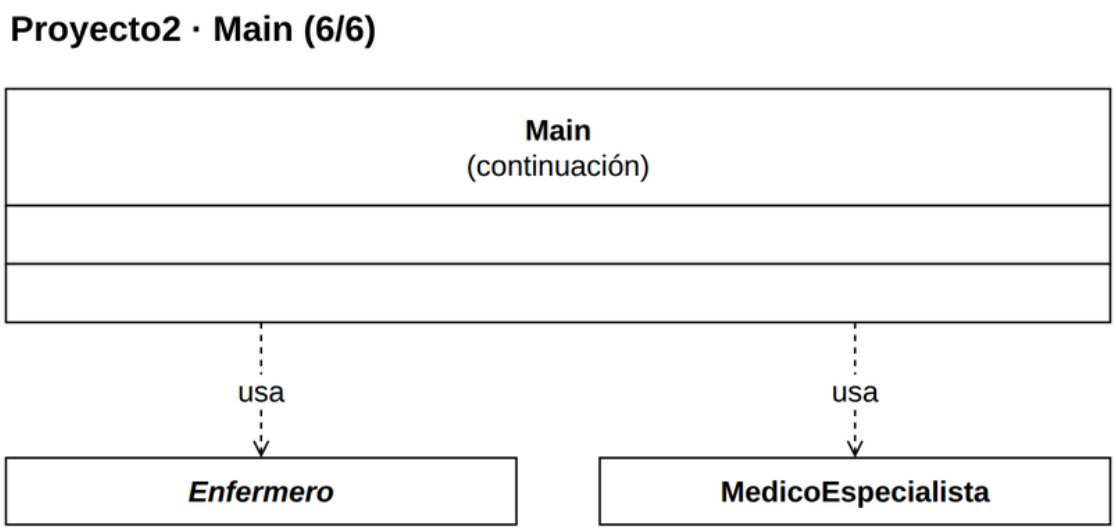


Figura 3.63: Proyecto fase 2 — Main. Parte 6 de 6.

La figura 3.64 muestra Paciente, parte 1 de 3.

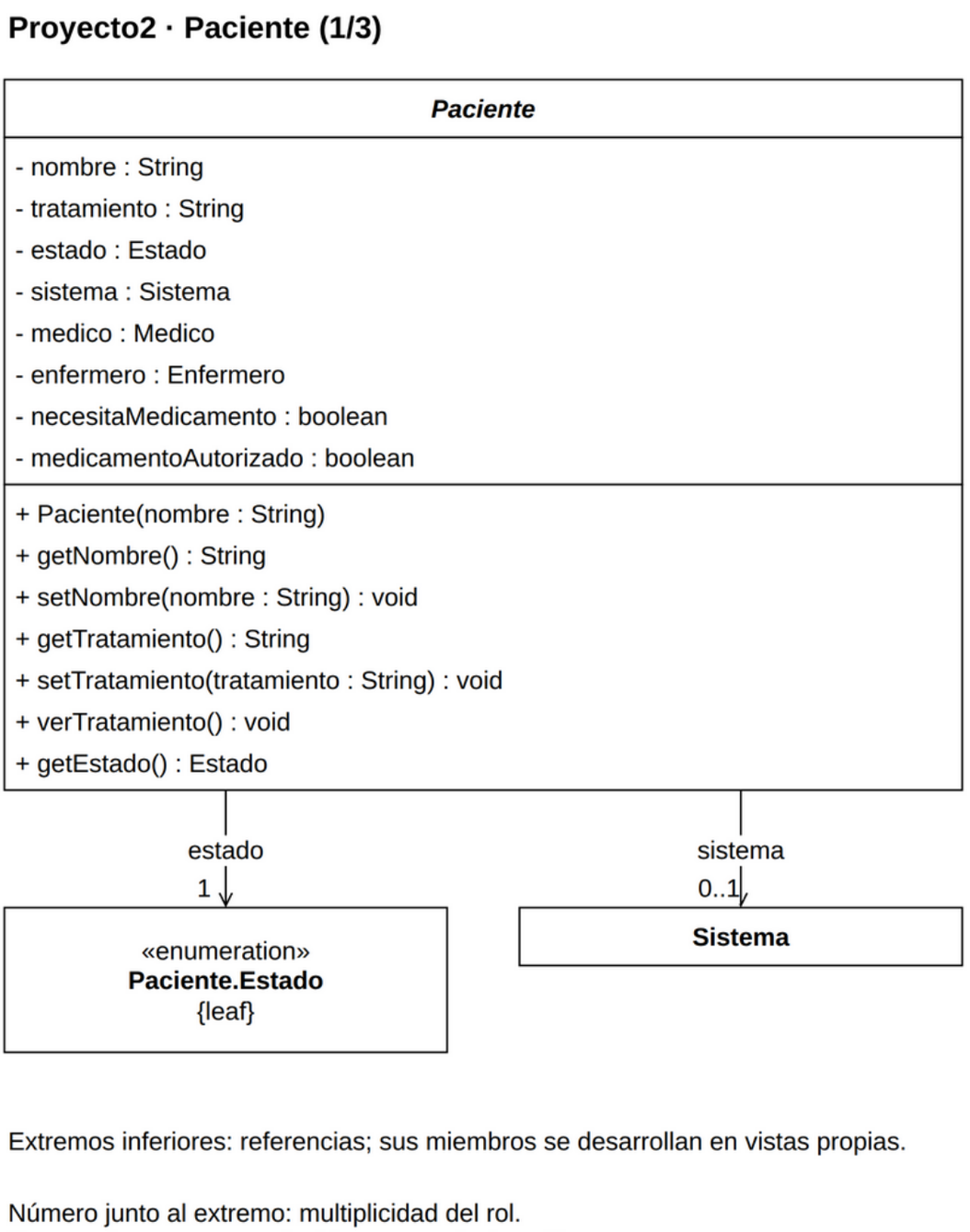


Figura 3.64: Proyecto fase 2 — Paciente. Parte 1 de 3.

La figura 3.65 muestra Paciente, parte 2 de 3.

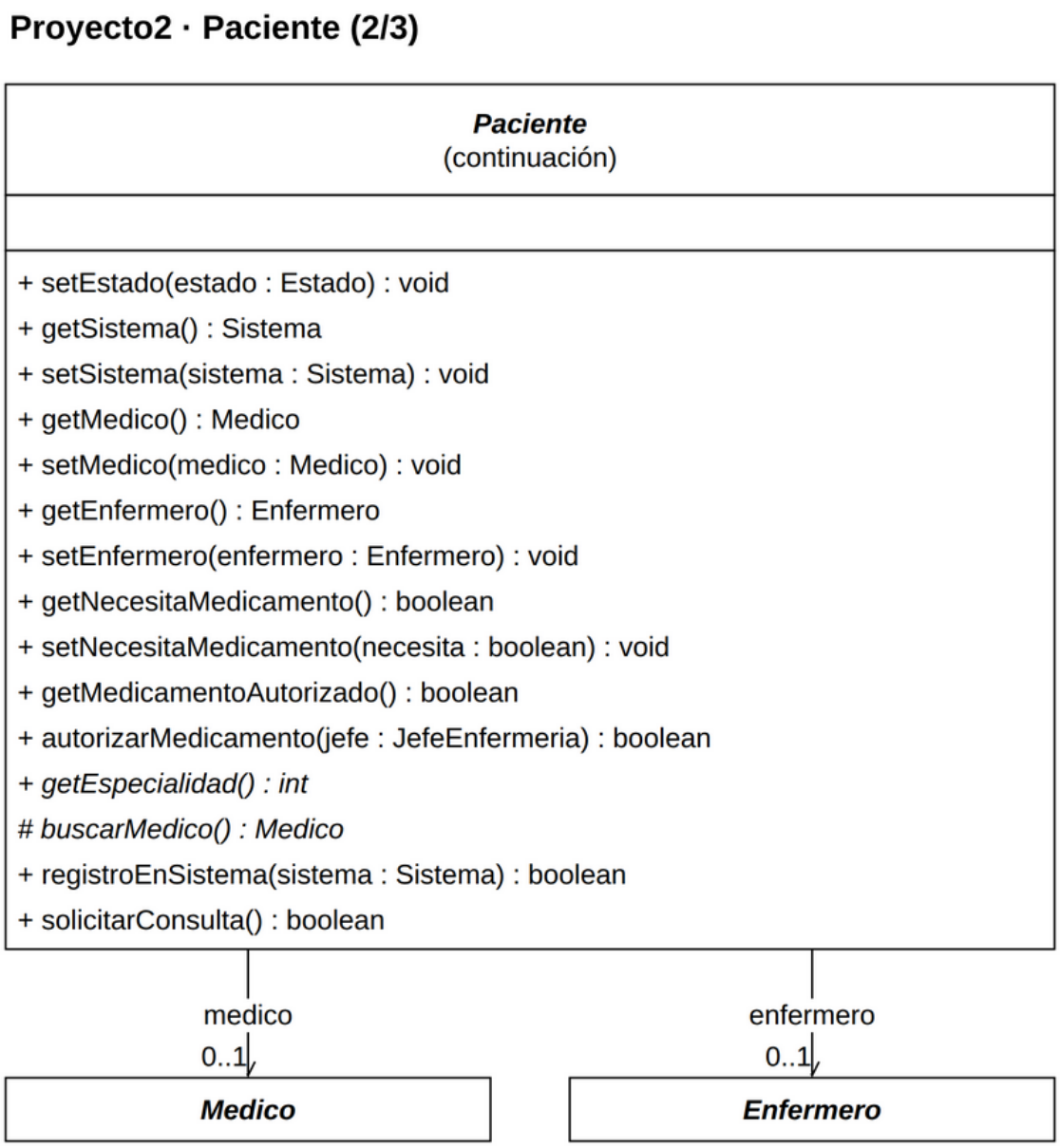


Figura 3.65: Proyecto fase 2 — Paciente. Parte 2 de 3.

La figura 3.66 muestra Paciente, parte 3 de 3.

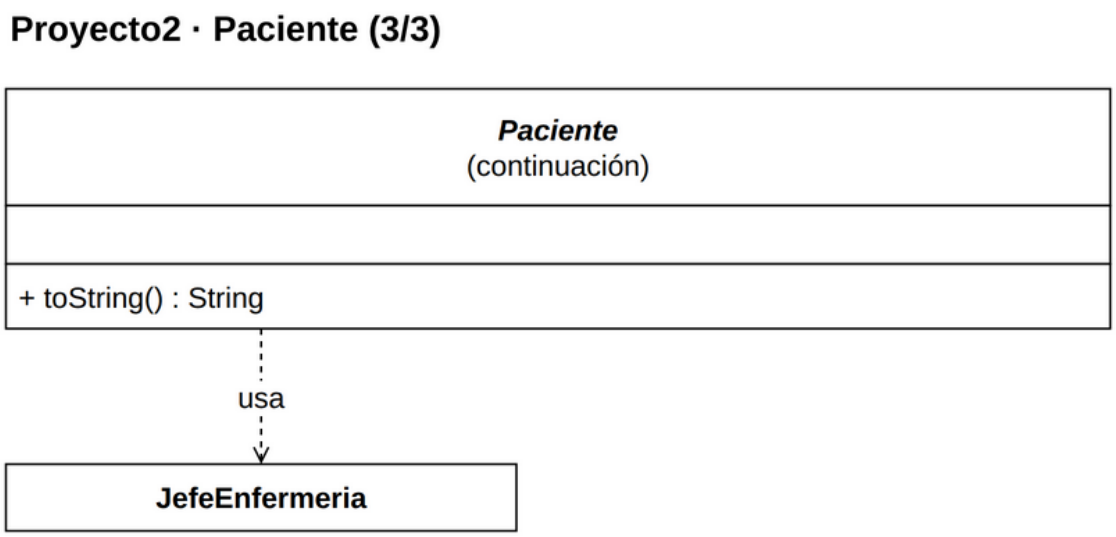


Figura 3.66: Proyecto fase 2 — Paciente. Parte 3 de 3.

La figura 3.67 muestra Paciente.Estado.

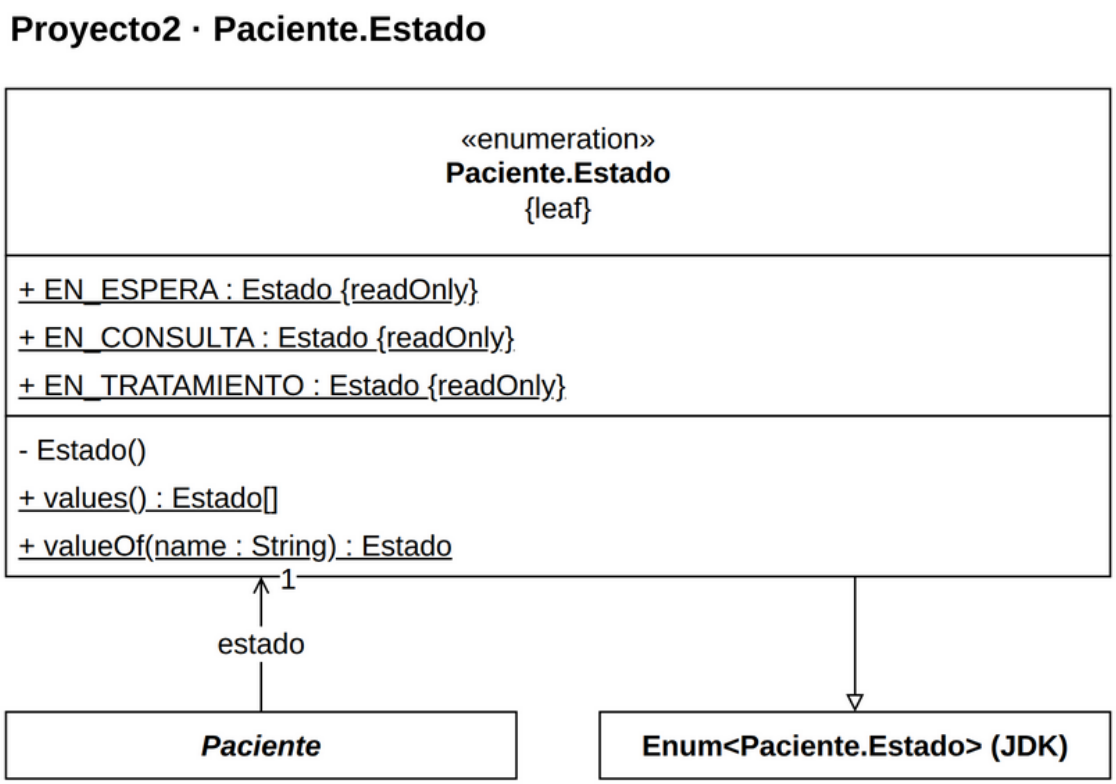


Figura 3.67: Proyecto fase 2 — Paciente.Estado.

La figura 3.68 muestra PacienteEspecialidad.

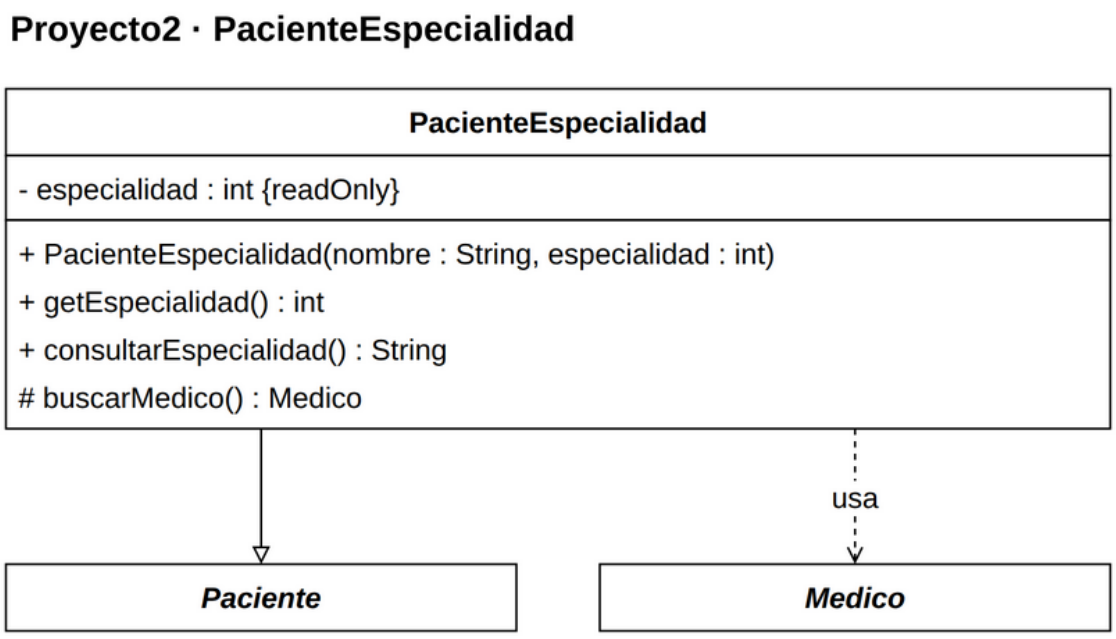


Figura 3.68: Proyecto fase 2 — PacienteEspecialidad.

La figura 3.69 muestra PacienteGeneral.

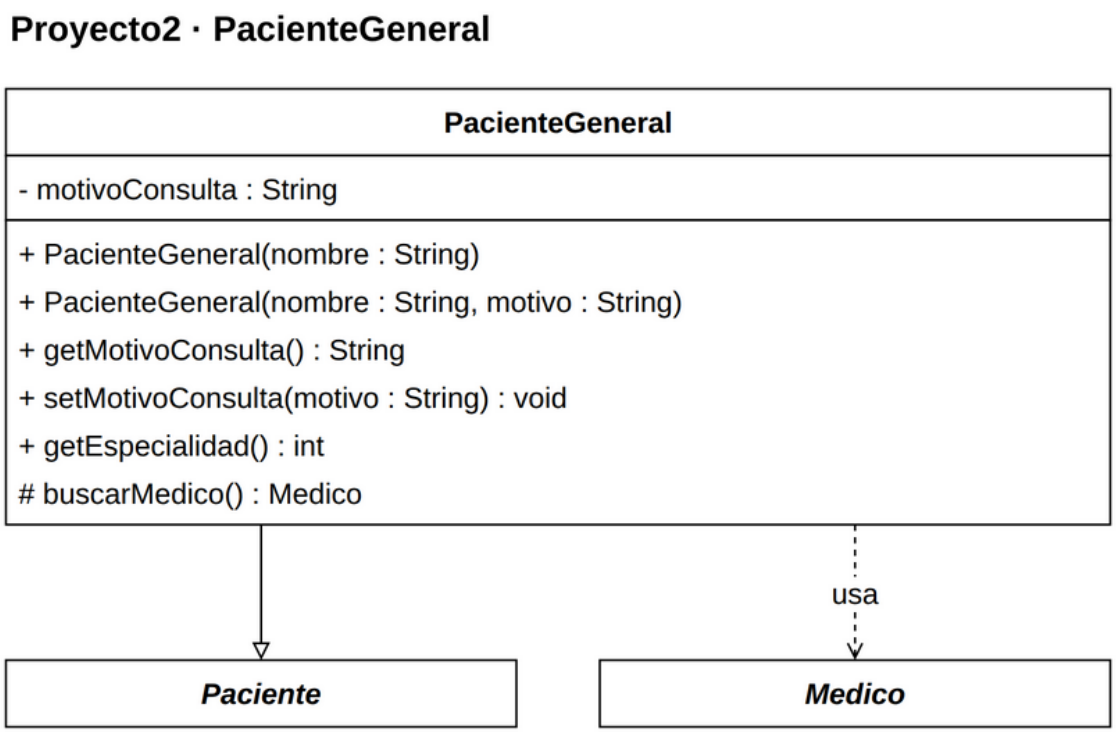


Figura 3.69: Proyecto fase 2 — PacienteGeneral.

La figura 3.70 muestra Enfermero, parte 1 de 3.

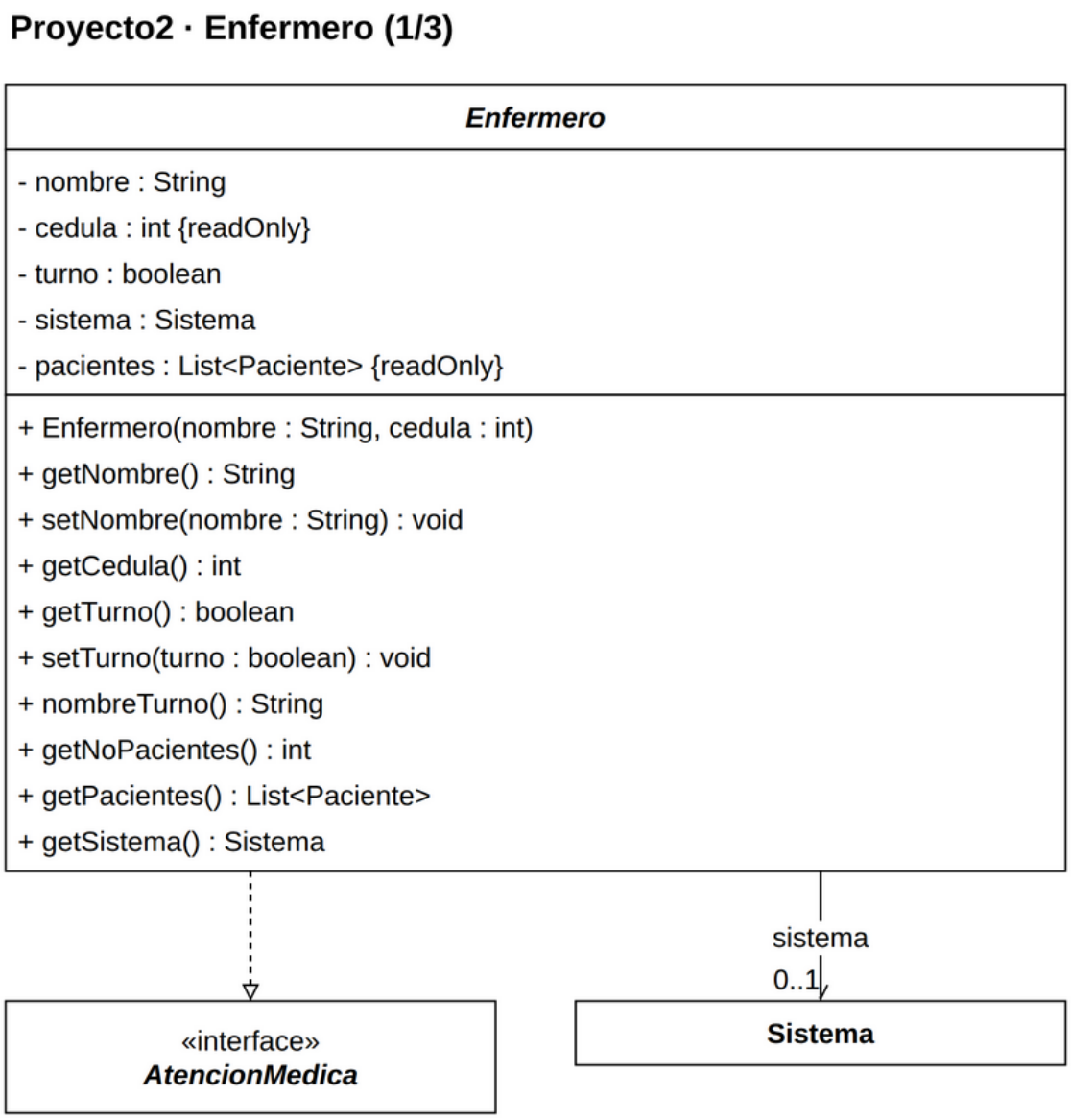


Figura 3.70: Proyecto fase 2 — Enfermero. Parte 1 de 3.

La figura 3.71 muestra Enfermero, parte 2 de 3.

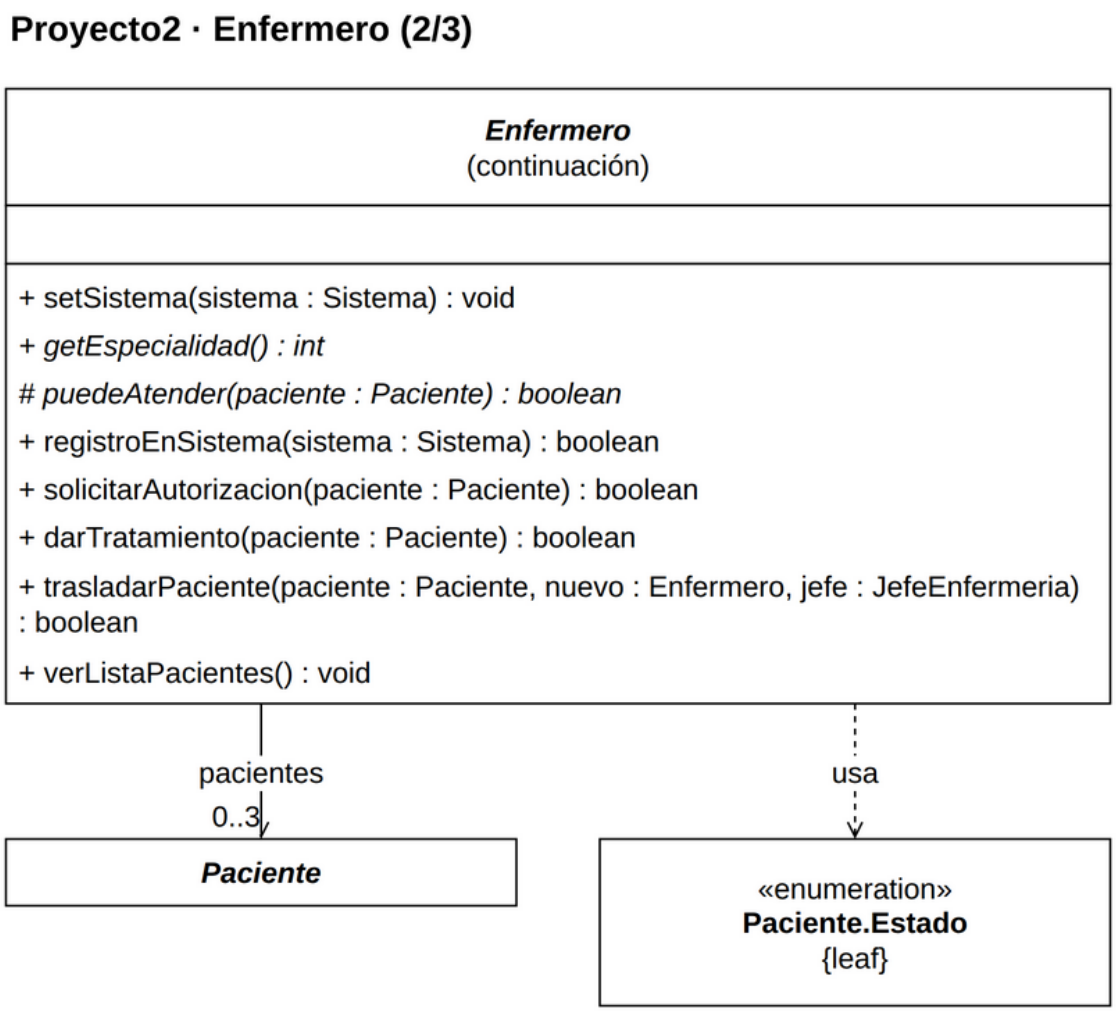


Figura 3.71: Proyecto fase 2 — Enfermero. Parte 2 de 3.

La figura 3.72 muestra Enfermero, parte 3 de 3.

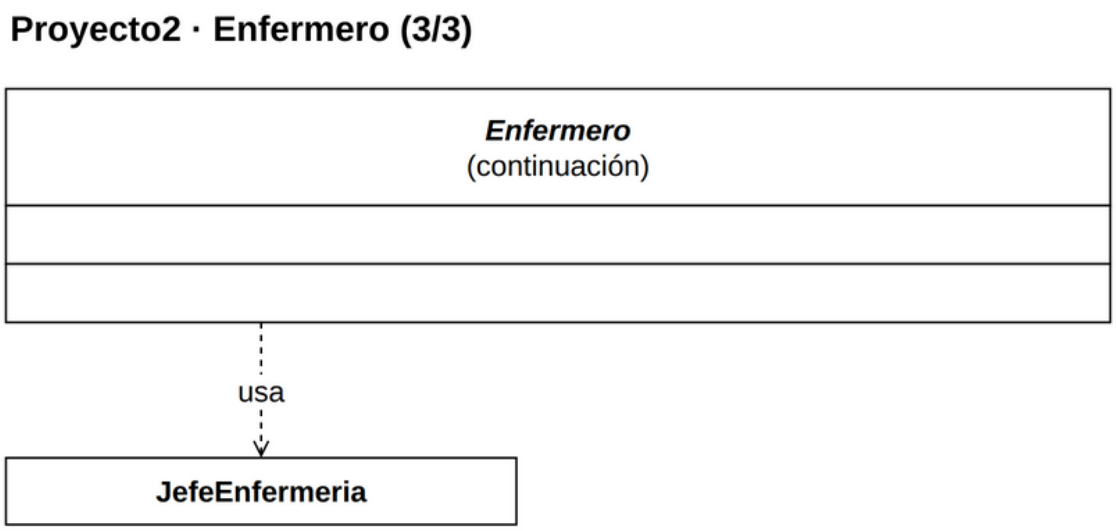


Figura 3.72: Proyecto fase 2 — Enfermero. Parte 3 de 3.

La figura 3.73 muestra EnfermeroEspecialista.

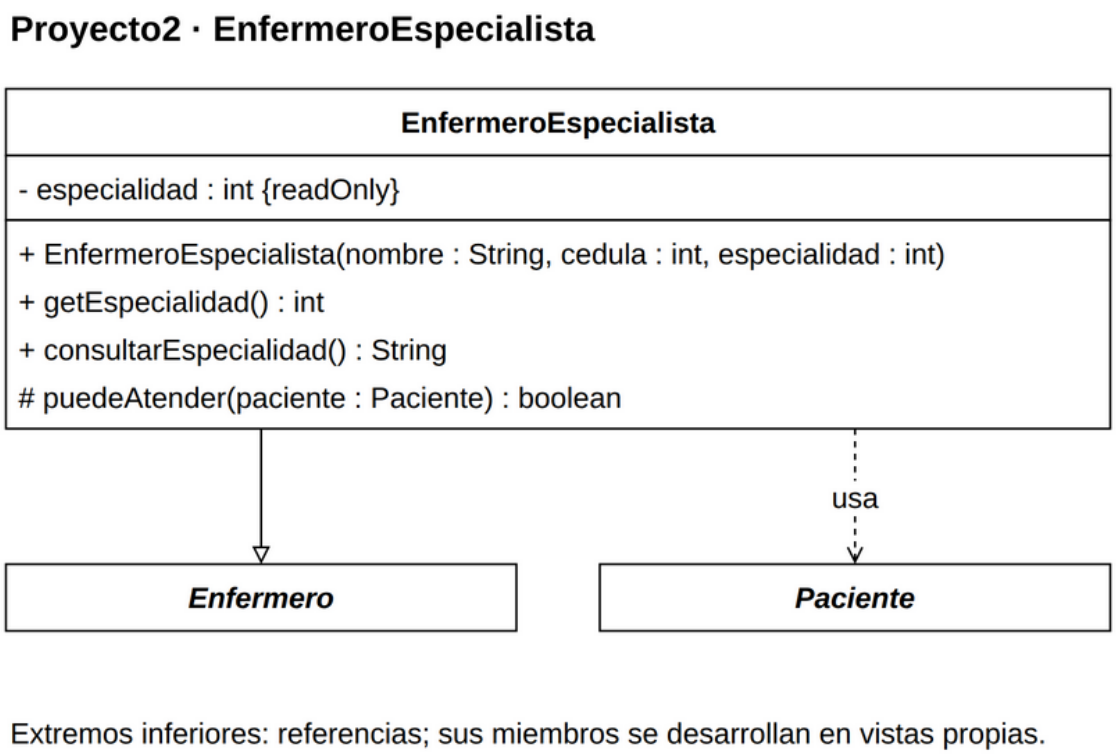


Figura 3.73: Proyecto fase 2 — EnfermeroEspecialista.

La figura 3.74 muestra JefeEnfermeria, parte 1 de 2.

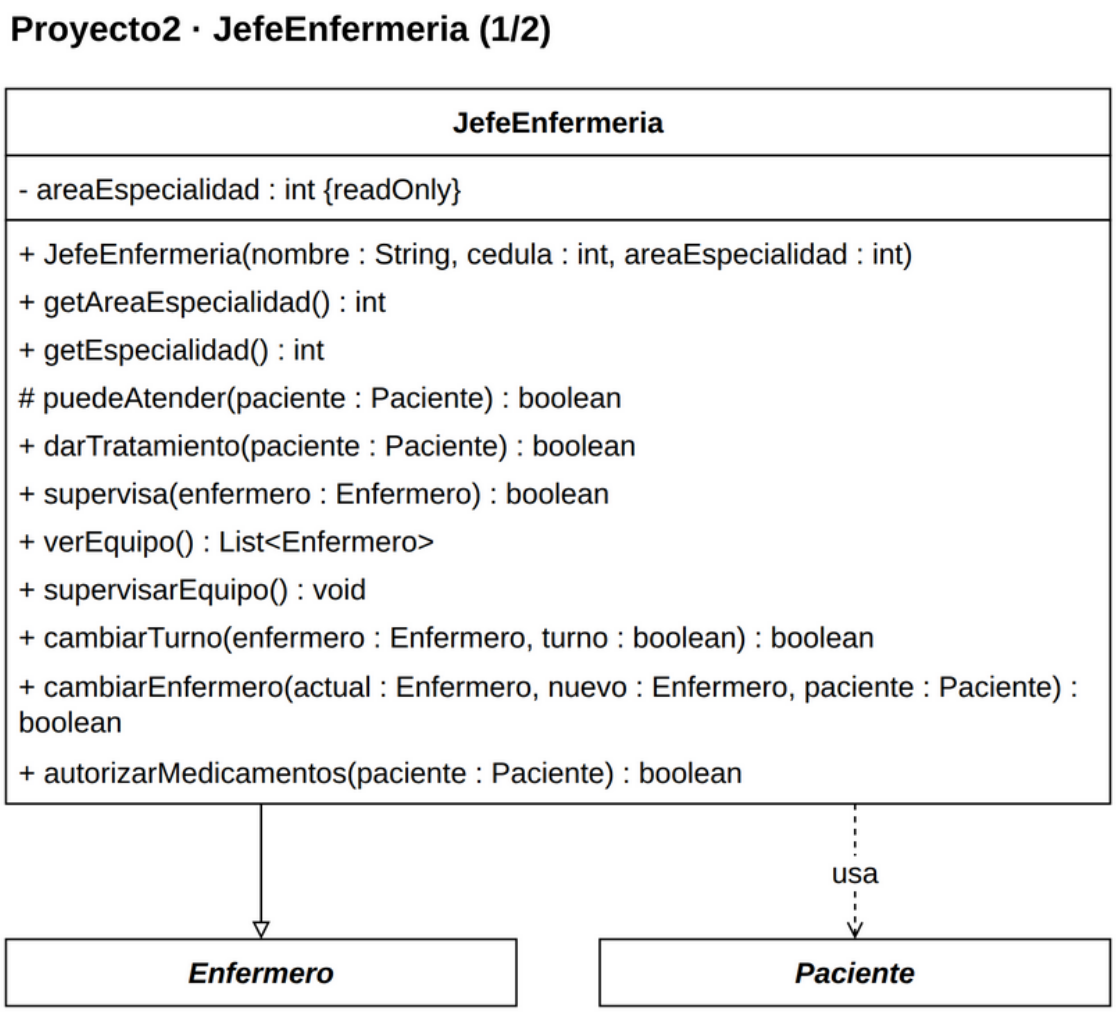


Figura 3.74: Proyecto fase 2 — JefeEnfermeria. Parte 1 de 2.

La figura 3.75 muestra JefeEnfermeria, parte 2 de 2.

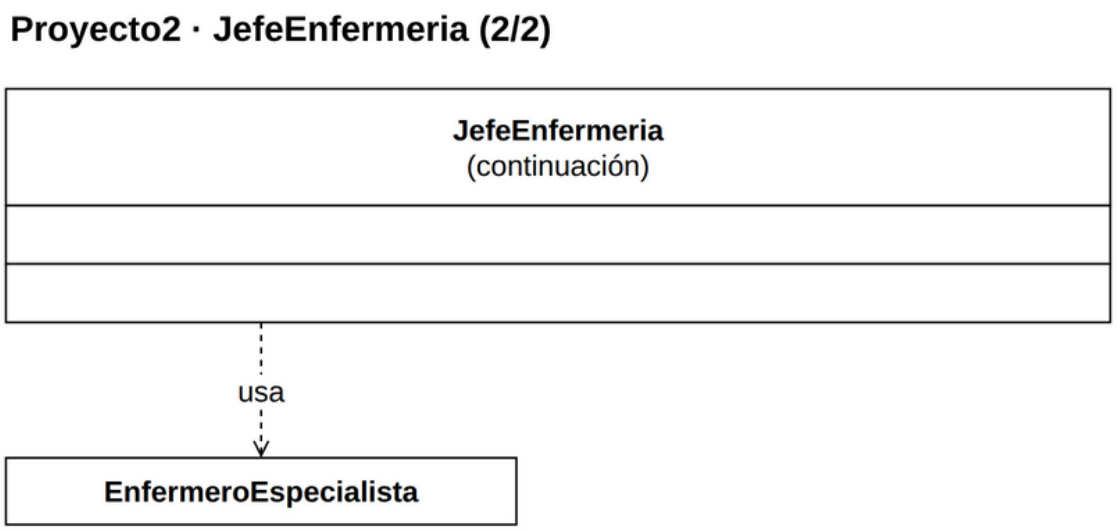


Figura 3.75: Proyecto fase 2 — JefeEnfermeria. Parte 2 de 2.

La figura 3.76 muestra Medico, parte 1 de 2.

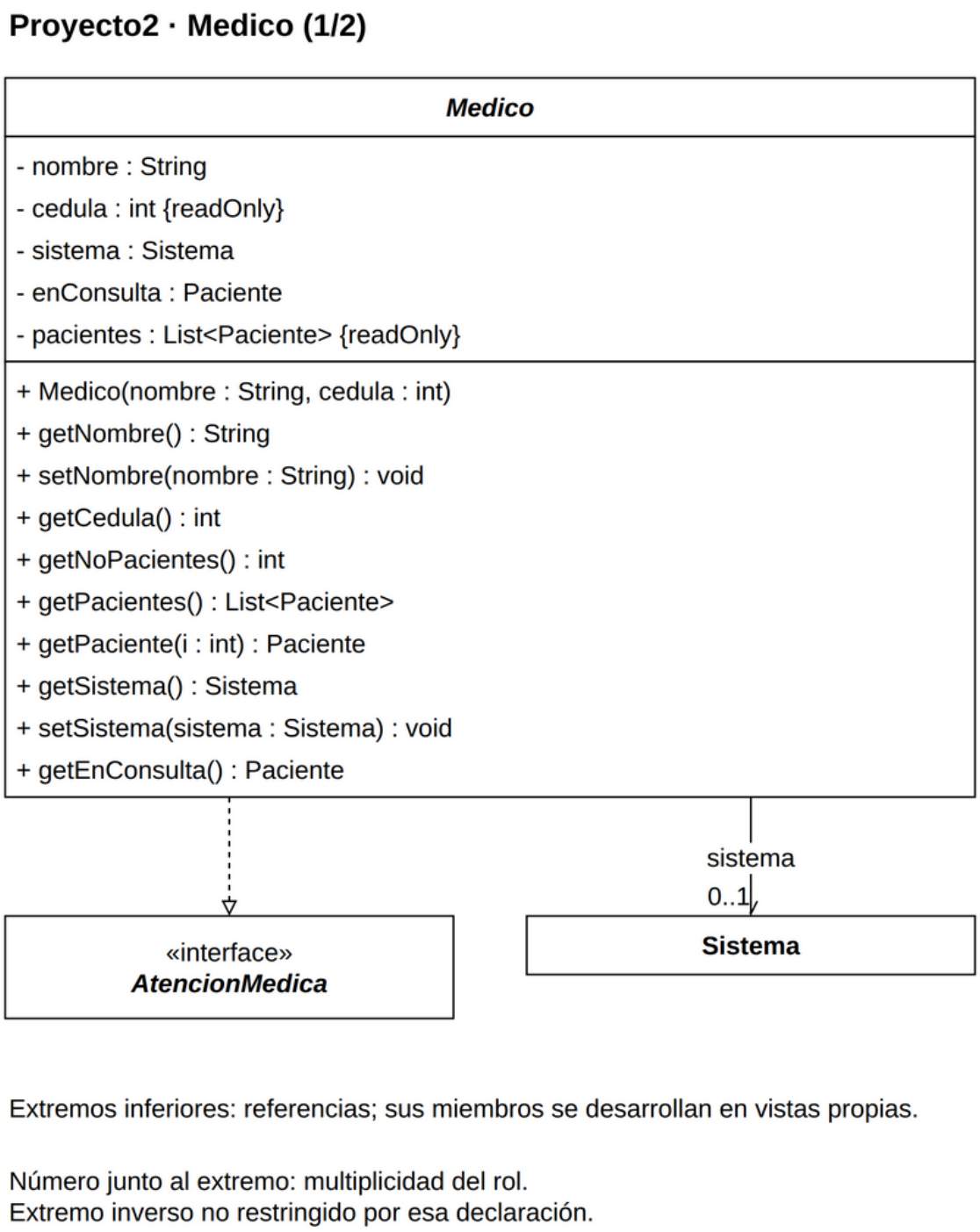


Figura 3.76: Proyecto fase 2 — Medico. Parte 1 de 2.

La figura 3.77 muestra Medico, parte 2 de 2.

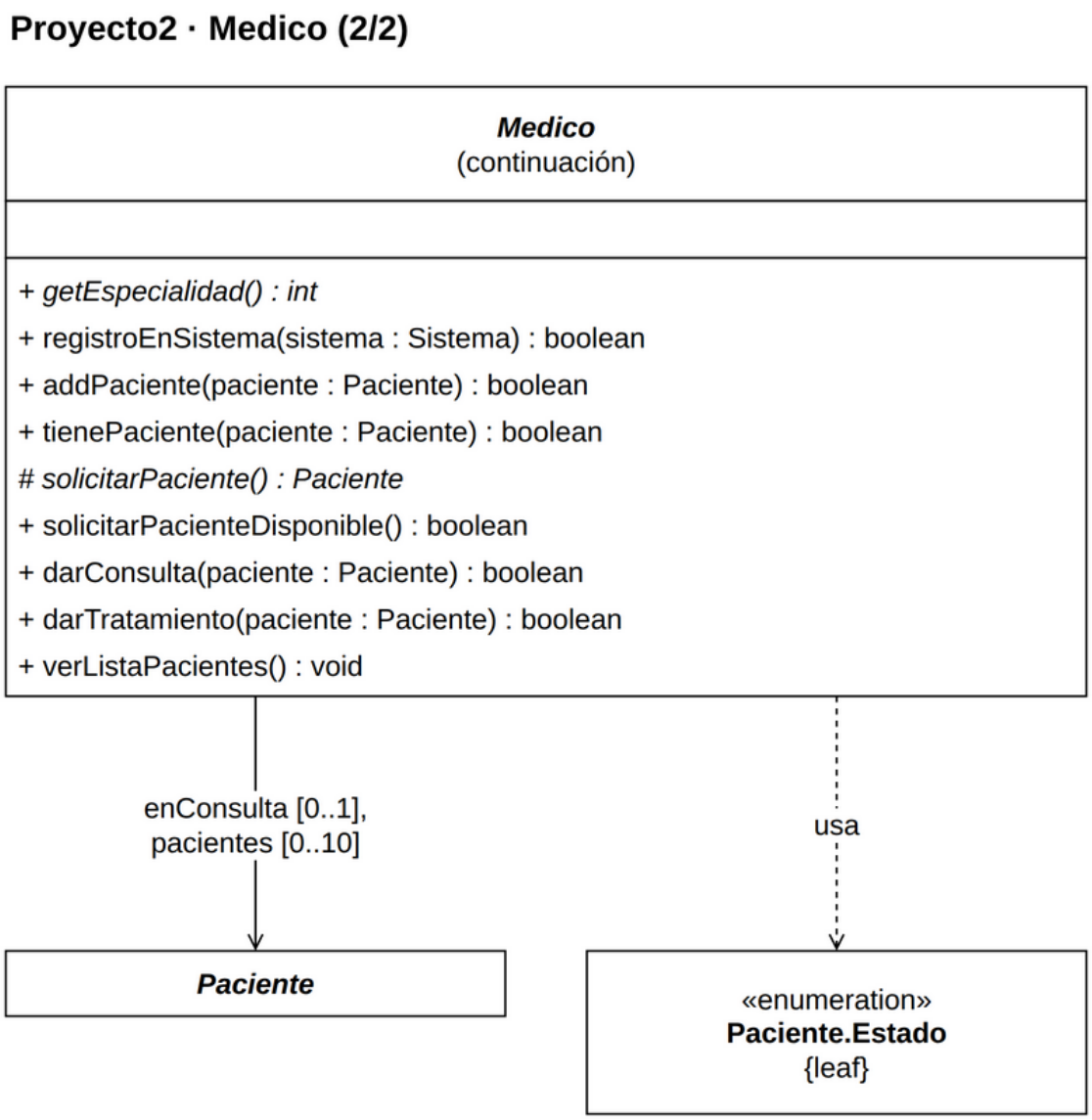


Figura 3.77: Proyecto fase 2 — Medico. Parte 2 de 2.

La figura 3.78 muestra MedicoEspecialista, parte 1 de 2.

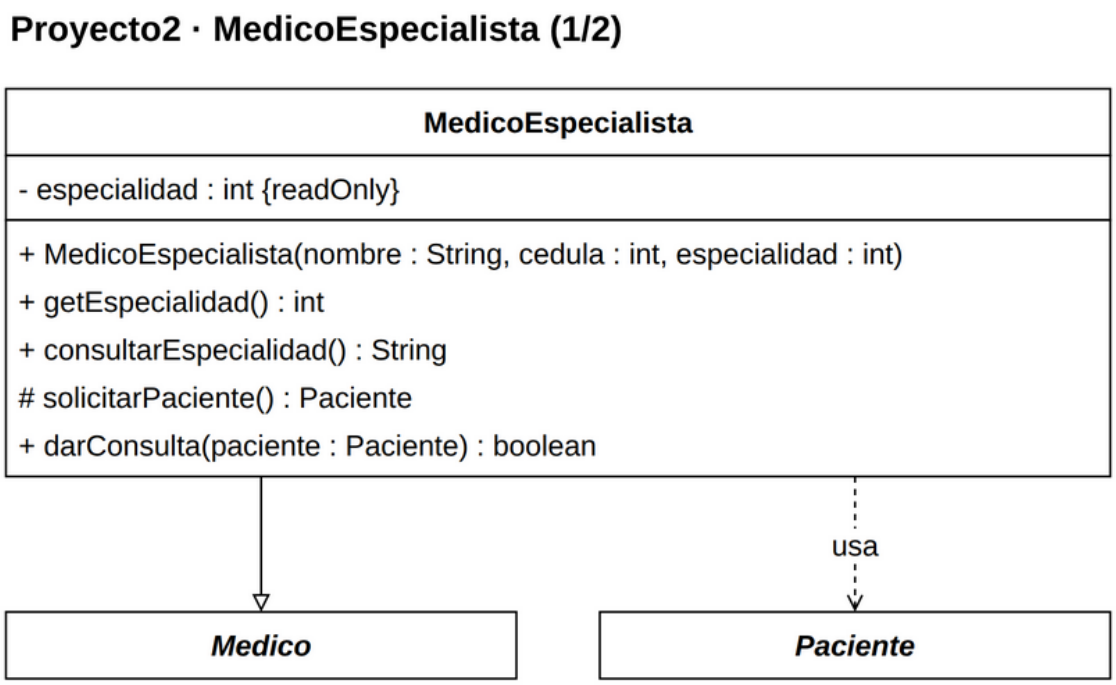


Figura 3.78: Proyecto fase 2 — MedicoEspecialista. Parte 1 de 2.

La figura 3.79 muestra MedicoEspecialista, parte 2 de 2.

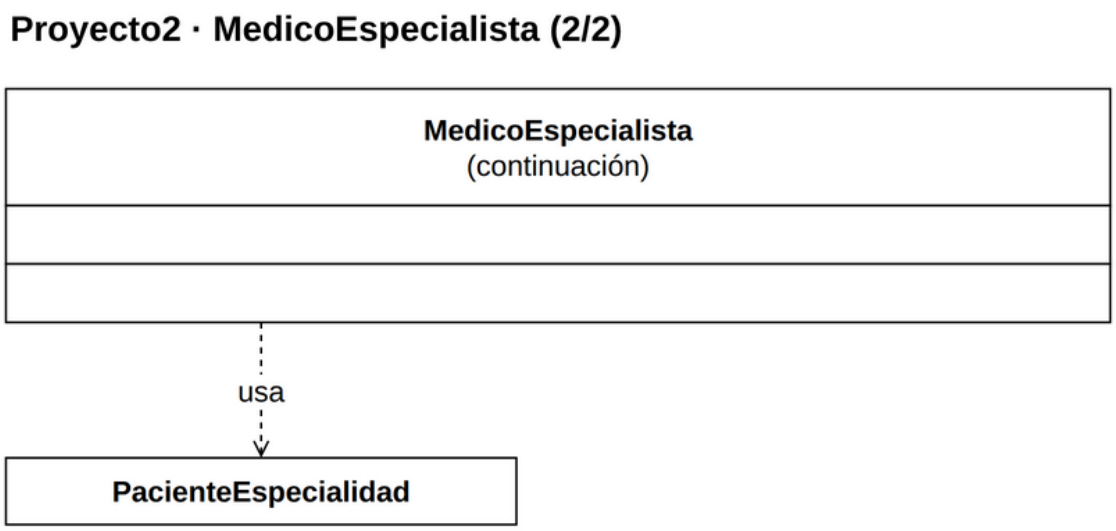


Figura 3.79: Proyecto fase 2 — MedicoEspecialista. Parte 2 de 2.

La figura 3.80 muestra MedicoGeneral, parte 1 de 2.

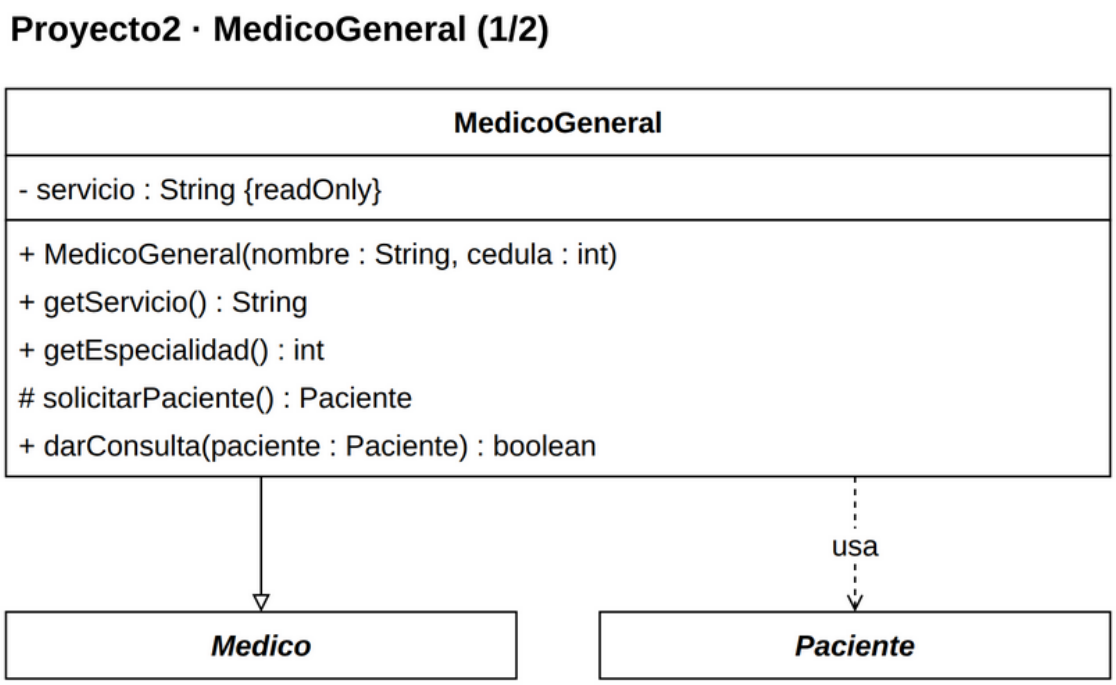


Figura 3.80: Proyecto fase 2 — MedicoGeneral. Parte 1 de 2.

La figura 3.81 muestra MedicoGeneral, parte 2 de 2.

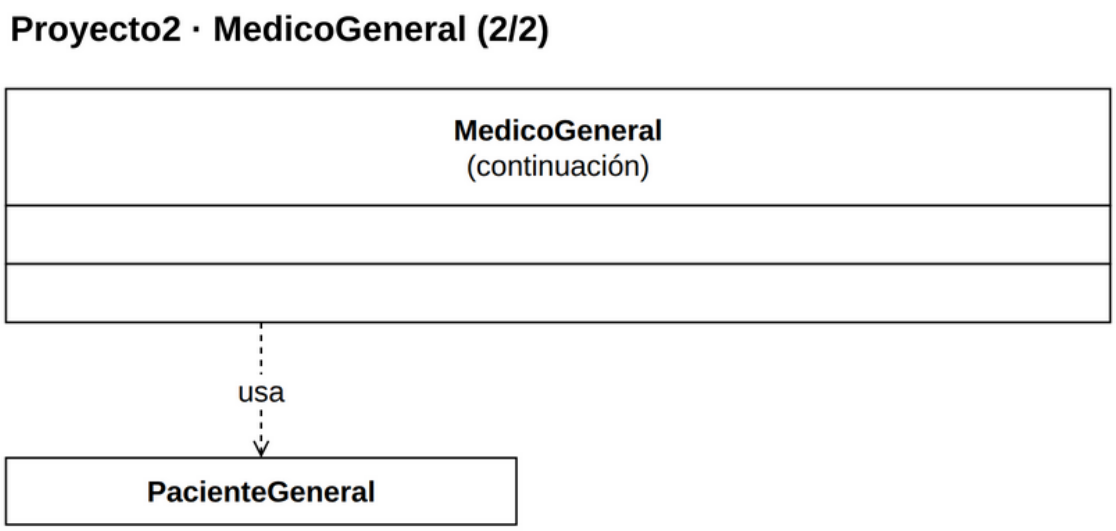


Figura 3.81: Proyecto fase 2 — MedicoGeneral. Parte 2 de 2.

La figura 3.82 muestra Sistema, parte 1 de 5.

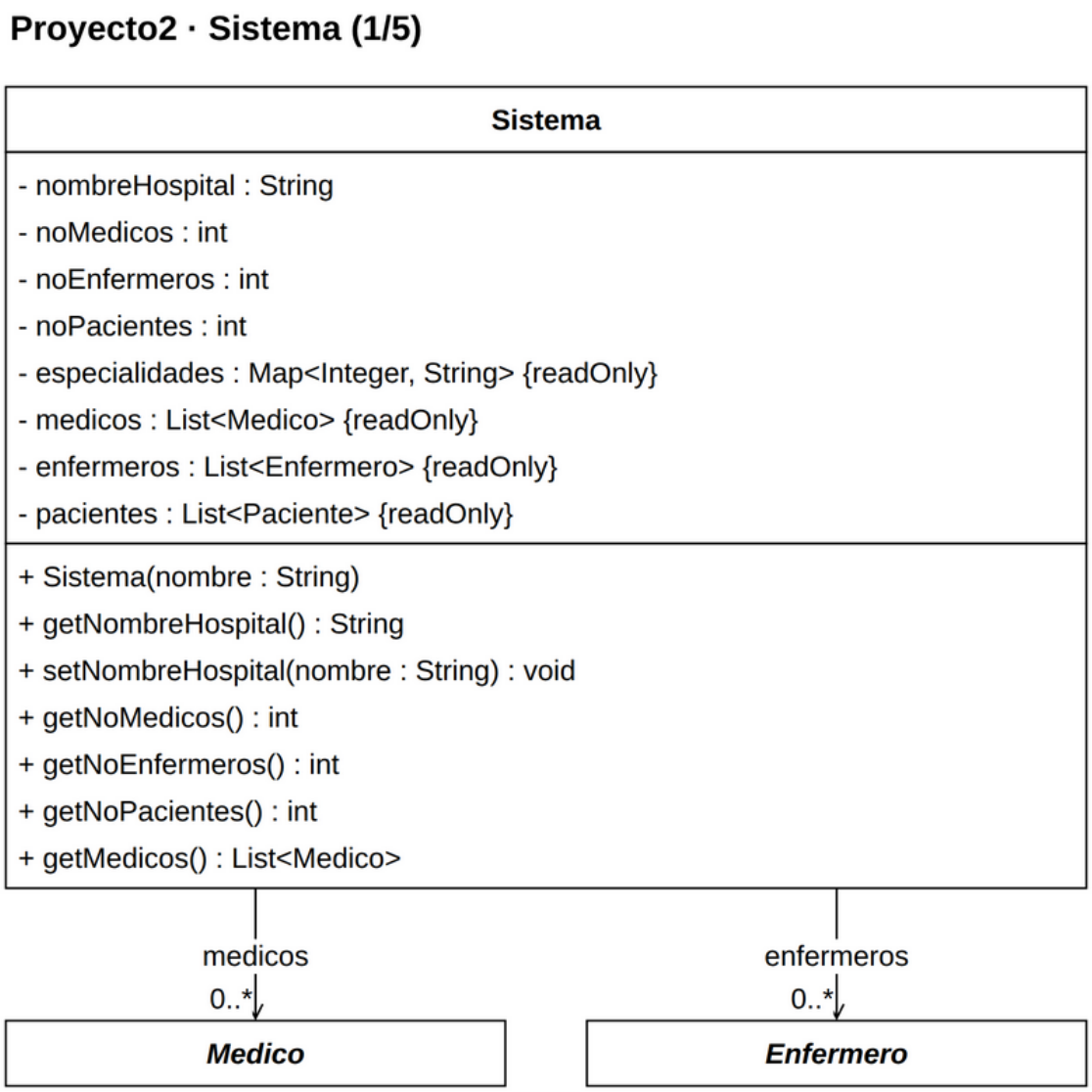


Figura 3.82: Proyecto fase 2 — Sistema. Parte 1 de 5.

La figura 3.83 muestra Sistema, parte 2 de 5.

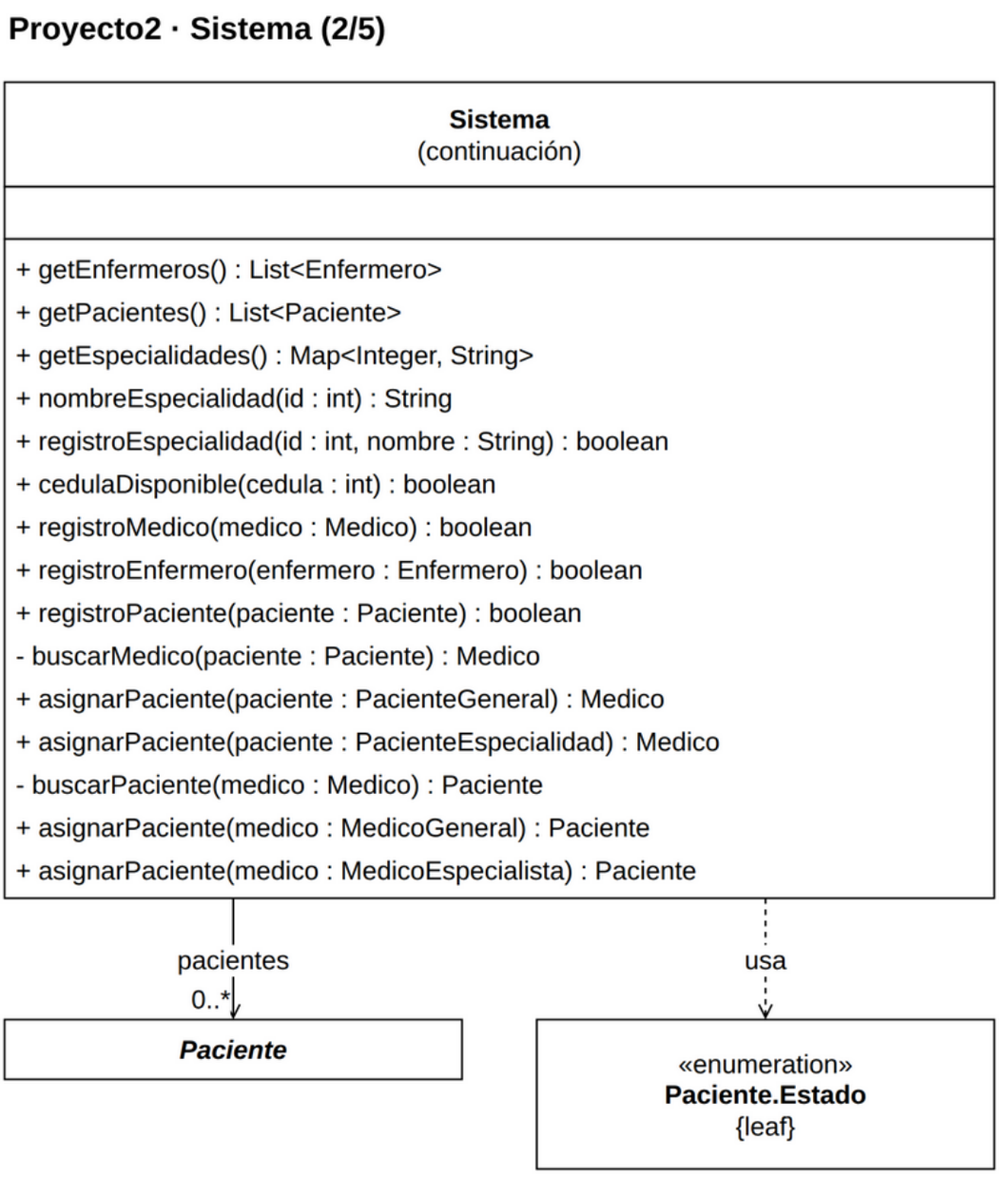


Figura 3.83: Proyecto fase 2 — Sistema. Parte 2 de 5.

La figura 3.84 muestra Sistema, parte 3 de 5.

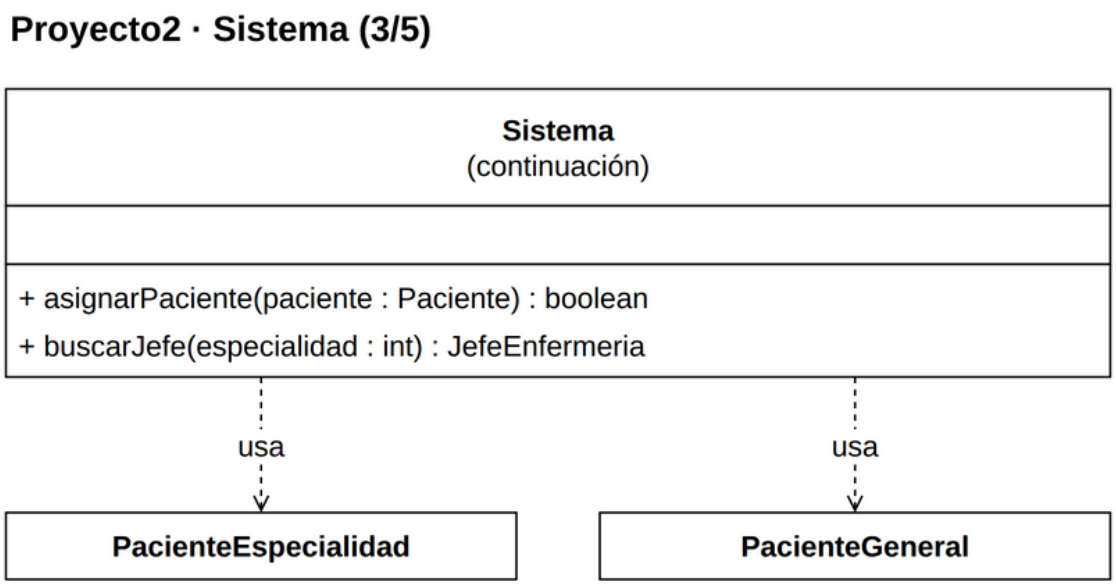


Figura 3.84: Proyecto fase 2 — Sistema. Parte 3 de 5.

La figura 3.85 muestra Sistema, parte 4 de 5.

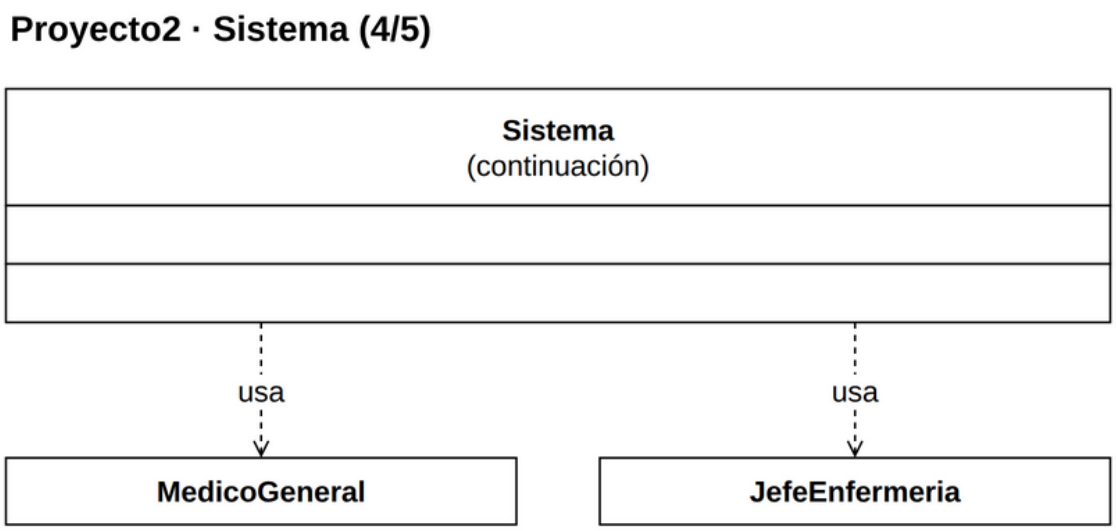


Figura 3.85: Proyecto fase 2 — Sistema. Parte 4 de 5.

La figura 3.86 muestra Sistema, parte 5 de 5.

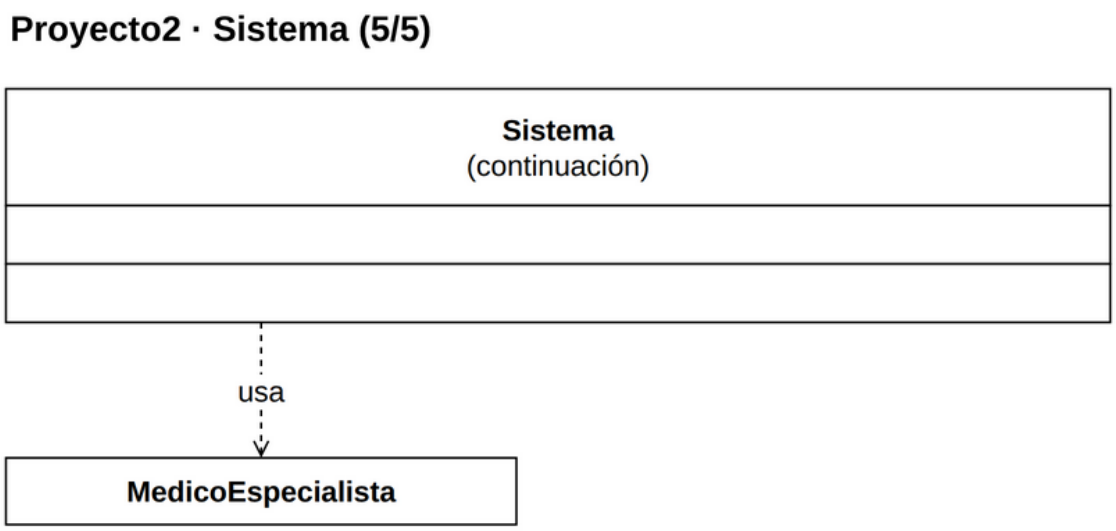


Figura 3.86: Proyecto fase 2 — Sistema. Parte 5 de 5.

4. Resultados

El insumo de esta actividad fue el código existente, no una entrada de consola. El resultado esperado consistió en cubrir todos los tipos propios y conservar sus firmas y relaciones sin alterar las prácticas. El resultado observado fue la compilación correcta de los cinco conjuntos y la obtención de 58 tipos: clases, interfaces, enumeraciones, un record y un tipo anónimo. La descomposición de vistas mantiene la correspondencia entre cada tipo y su archivo fuente.

Tabla 1: Cobertura estructural verificada por conjunto.

Conjunto	Fuentes	Tipos	Compilación
Práctica 5	4	4	Correcta
Práctica 6	25	28	Correcta
Práctica 7	7	7	Correcta
Práctica 8	6	6	Correcta
Proyecto fase 2	12	13	Correcta

Se generaron cinco maestros globales y 86 vistas de detalle, cada uno con un archivo editable de draw.io y su PNG exportado. Las figuras del reporte se incorporaron con tipografía equivalente mínima de 10.93 puntos. Las continuaciones permiten conservar las operaciones extensas sin comprimir la letra. No se ejecutaron programas para producir resultados numéricos nuevos, porque esta actividad corresponde al modelado de su estructura.

Como casos de contraste, MainEmpleado conserva una dependencia de creación y Cuadrilatero una asociación mediante atributos. La posibilidad de recibir null en los cuatro roles de puntos representa una frontera relevante: el modelo registra referencias opcionales, aunque un cálculo geométrico posterior pueda requerir objetos válidos. En el hospital de segunda fase, los máximos de diez y tres proceden de comprobaciones de asignación; no se trasladan automáticamente al hospital anterior de la práctica 6.

5. Conclusiones

La representación de clases permite distinguir el estado permanente de los usos temporales. Esta diferencia explica por qué un programa principal puede depender de una clase sin tener una asociación con ella, y por qué compartir un paquete no establece comunicación entre todos sus tipos. Al conservar clases desconectadas se describe la estructura real del trabajo.

La herencia y la realización expresan compromisos distintos: la primera organiza una jerarquía y la segunda identifica un contrato. Las multiplicidades complementan esa información cuando proceden de atributos y validaciones concretas. Una lectura fiel del código evita atribuir propiedad exclusiva, objetos obligatorios o límites que el programa no garantiza.

El contraste con las firmas compiladas ayuda a mantener consistencia entre las fuentes y sus diagramas. La combinación de maestros editables y vistas de detalle permite estudiar el conjunto y revisar miembros concretos, manteniendo el nivel de precisión necesario para explicar el diseño de los programas existentes.

Referencias

[1] Facultad de Ingeniería, UNAM. *Manual de prácticas del Laboratorio de Programación orientada a objetos*, código MADO-22, versión 01, emisión del 20 de enero de 2017. Apartado UML, páginas 142–150. Extracto local proporcionado para la actividad.

[2] *Diagrama de clases de referencia*. Imagen local proporcionada para la Práctica 10; ejemplo de títulos, ejemplares, préstamos y reservas.

[3] *equipo2-poo: fuentes Java y reportes de las prácticas 5, 6, 7 y 8*. Material del repositorio del equipo, directorios de cada práctica. Fuentes y firmas compiladas consultadas para este modelado.

[4] *equipo2-poo: Proyecto2*. Documento de uso y fuentes en `src/hospital`; segunda fase del sistema hospitalario. Material del repositorio del equipo.